AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC4118: COMPUTER GRAPHICS

Summer 2024-2025

Section: [*F*]

Group No: 09

Project Title

Futuristic Space City

Supervised By

DIPTA JUSTIN GOMES

Submitted By:

Name	ID	Contributions
1. MD. TAZUARE RAHMAN RAHAD	**23-52216-2**	Space Metro Station Scenario and Report Writing
2. SATADRU BARUA JEET	**23-52224-2**	Futuristic City (Partial) and Destroyed City Scene and Code merging
3. RAFAYET HOSSAIN ZANITH	**23-52178-2**	Futuristic City (Partial) and Destroyed City Scene
4. SAMIYA SIDDIQUA SYMPA	**23-52240-2**	Space City Farmland Scenario

Introduction

The idea of a futuristic space city has made people wonder for decades. Many stories, films, and games have tried to imagine how life might exist beyond Earth. Such cities are often shown as symbols of human progress and survival. The concept combines imagination with science and technology. A futuristic space city represents the next stage of civilization, where humans have built advanced settlements in outer space. The project titled "Futuristic Space City" was developed to imagine and visualize what such a place might look like. It focuses on the structure, atmosphere, and culture of a city built outside Earth's environment. The purpose of this project is not only to show an artistic idea but also to reflect on how humanity might adapt to new worlds in the future.

The idea for this project came from observing how modern science fiction has explored the future of humanity. Films such as Interstellar (2014), Blade Runner 2049 (2017), and the video game Cyberpunk 2077 (2020) have deeply influenced its concept. Each of these works presents a different vision of the future. Interstellar focuses on human survival beyond Earth. It shows large Space Metro Stations, rotating habitats, and distant planets where people search for new homes [1]. Blade Runner 2049 shows a futuristic but damaged world on Earth. It depicts tall buildings, flying vehicles, and dark city lights that create a sense of mystery and loneliness [2]. Cyberpunk 2077 takes these ideas further by allowing people to explore a digital city full of neon lights, machines, and humans living closely with technology [3]. These works inspired the design and mood of the Futuristic Space City project. They helped to shape its architecture, color tone, and story.

The main objective of this project is to represent ˙life in a futuristic space environment. It aims to show how cities might evolve when Earth can no longer meet human needs. In such a setting, survival would depend on innovation, cooperation, and technology. The project explores how buildings, transport, and agriculture could function in space. It reflects on how people might create a comfortable and beautiful life even in an artificial environment. It also raises questions about the balance between progress and nature. A futuristic city can be both a dream of hope and a warning about dependence on technology. By visualizing this concept, the project tries to encourage both imagination and awareness. Another objective of this project is to link imagination with education. By visualizing a futuristic city, it encourages thinking about urban design, architecture, sustainability, and digital art. Students and viewers can learn how human creativity and science can work together. The project also introduces how environments can be designed for extreme conditions such as low gravity, isolation, or limited resources. It connects artistic ideas with practical thinking. In this way, it acts as both a creative and educational experience.

The project includes three main scenarios that together build the whole city. The first is the Space Metro Station. It represents the outer area of the city where space travel and transport take place. This station acts as the main connection between the city and other planets or colonies. It shows human control over technology and communication. The second scenario is the City View and Destroyed City. This part of the project shows two sides of civilization one part where life is still active, full of lights and energy, and another part that has fallen into ruin. The destroyed section reminds viewers that every civilization faces challenges, even in the most advanced future. The third scenario is the Space City Farmland. This area symbolizes sustainability and survival. It includes fields, domes, and artificial systems that allow food to grow and air to recycle. Together, these three scenes build a complete picture of a futuristic city that is both alive and self-dependent.

The concept of a futuristic space city also reflects real scientific progress. Today, many space agencies and private companies are planning missions to the Moon and Mars. Projects like SpaceX's Mars Colony or NASA's plans for a lunar base show that the idea of living in space

is no longer just science fiction [4]. Researchers are studying how humans can grow food, produce oxygen, and create safe shelters in extreme environments. The project draws inspiration from these real-world efforts. It tries to imagine how cities in space might look and function if these plans become real. By showing different parts of the city, it helps people visualize what living beyond Earth could mean.

The cultural inspiration behind the project also plays an important role. In Blade Runner 2049, the design of tall, crowded, and glowing buildings creates a powerful emotional effect. It shows how technology can shape human feelings and society. In Cyberpunk 2077, the idea of a city that never sleeps, filled with machines and advertisements, represents a future where humans and technology are deeply connected. In Interstellar, space itself becomes a symbol of hope and challenge. These influences helped build the tone of the Futuristic Space City. The city is imagined as a place of both beauty and struggle full of light yet surrounded by darkness. It expresses the dual nature of human progress: the desire to advance, and the fear of losing humanity in the process.

The idea of dividing the project into three scenarios also follows a symbolic meaning. The Space Metro Station represents human ambition and technological strength. It shows how humans have learned to travel and live in space. The City View and Destroyed City represent civilization's dual face one that builds and one that collapses. It asks whether humanity can maintain balance as it advances. The Space City Farmland represents hope and rebirth. It shows that even in a mechanical world, nature and life must survive. These three scenes together tell a story of survival, growth, and reflection. They form a cycle technology, destruction, and renewal which can be seen in human history as well.

The project is designed not just as a one-time creation but as a foundation for future work. The city structure and scenarios can later be expanded into an interactive game. The idea is to allow users to explore the city, interact with robots, and experience the environment through a player's perspective. This plan connects the project with modern digital entertainment, where virtual cities and worlds are common. Turning it into a game would make it more engaging and educational. It would also allow people to experience a futuristic city not only as viewers but as participants. The concept follows the model seen in Cyberpunk 2077, where players explore a living city full of visual and narrative details. Building such a game in the future would combine education, storytelling, and creativity. The Futuristic Space City is not only an artistic idea but also a reflection of human potential. It brings together imagination, science, and emotion. It shows how digital visualization can turn complex ideas into simple visual forms that anyone can understand. It builds a bridge between fiction and reality, between dreams and technology. It also gives an opportunity to think about how design can affect the way people live, feel, and connect with the world even if that world is in space. By using familiar references like Interstellar, Blade Runner 2049, and Cyberpunk 2077, the project connects imagination with recognizable visual styles and ideas. These influences make it easier for people to connect emotionally and understand the vision behind the work. In the future, this project can grow in many directions. It can become an interactive game, a 3D simulation, or even a virtual reality experience. Sound, lighting, and artificial intelligence could be added to make the environment more realistic. Robots could move, vehicles could fly, and users could explore every corner of the space city. It could be used for education, entertainment, or even as a tool for future urban research. The core idea of imagining life in space will remain the same, but technology will allow deeper exploration. The project provides a strong base for those future steps.

In summary, the Futuristic Space City represents a vision of what life beyond Earth might look like. It explores themes of progress, survival, and creativity. It was inspired by science fiction but guided by real scientific goals. Through three scenarios: Space Metro Station, City View and Destroyed City, and Space City Farmland it tells a story of human achievement and reflection. It connects technology with emotion and future with responsibility. The project

stands as a creative step toward understanding what it means to live and build in the unknown. It is a reminder that imagination and science together can shape the future of humanity.

Methodology

The project was developed using C++ and OpenGL library. The OpenGL library was used because it is free, flexible, and supports both 2D and 3D graphics. The GLUT (OpenGL Utility Toolkit) helped to manage windows, handle input, and control events. The programming work was done in Code::Blocks IDE on Windows 11. Code::Blocks made it easier to compile and test OpenGL programs in one place. The project can also run on Linux or macOS with small changes in settings.

The project followed a modular structure. Each part of the code was divided into small functions. Each function had one task, such as drawing buildings, roads, robots, or stars. For example, a rectangle-drawing function was used to make tall buildings. A circle function was used for domes and satellite shapes. Lines were used for paths and flying routes. This modular structure helped to organize the work and made debugging easier. If a part did not work, it could be fixed without touching other parts.

The project used 2D Orthographic Projection (gluOrtho2D). It was chosen because the aim was to build a two-dimensional visualization, not a 3D one. The 2D projection created a flat look but still allowed depth through colors and animation. It also made the project easier to understand for beginners. The main idea was not to build a realistic 3D city but to visualize a future world using simple 2D graphics.

The full project had three main scenarios: Space Metro Station, City View and Destroyed City, and Space City Farmland. Each scenario was created separately by different team members. They were later connected as a combined futuristic environment.

In the Space Metro Station scenario, different objects were made to show a high-tech area in outer space. The background was black, representing the universe. Many stars were drawn at random positions using random number functions. This gave a natural space effect. Small circles were used to draw planets and satellites. The Space Metro Station itself was built using rectangles and polygons with bright color combinations like blue, white, and silver. A space train was animated to move slowly across the screen. The movement was done using translation transformations. It made the space train look like it was floating in zero gravity. This part was inspired by Interstellar (2014), where the Endurance spacecraft travels through deep space [1]. The same idea of calm and vast outer space was applied. The lighting and color tone of this scenario were kept soft and blue to give a peaceful feeling.

The City View and Destroyed City scenario focused on showing a contrast between progress and destruction. It started with a normal futuristic city full of lights, tall buildings, and moving vehicles. The city was drawn using rectangles with glowing colors to look like glass towers. Windows and light reflections were created using small color variations. The streets were made with long black rectangles and white lines as lanes. Small cars and trains were animated to move continuously using transformation functions. Some glowing arrows were added on the roads to show traffic direction. Later in the same scene, parts of the city were shown as destroyed. Buildings were drawn as broken pieces using irregular polygons. Smoke and fire effects were created using small triangles and color changes from red to orange. This transformation from a glowing city to a ruined one represented a post-apocalyptic world, much

like Blade Runner 2049(2017) [5]. The inspiration came from its visual style where bright neon lights mix with dark, destroyed environments. The destroyed city also symbolizes the danger of technology and environmental collapse.

The third part was Space City Farmland. This scenario showed how people in the future might grow food in space. It was imagined as a green area inside the space city. Rectangles were used to draw small buildings and glass domes where plants grow. The farmland section had patterns like grids to show crops or greenhouses. The soil and grass colors were kept green and brown to give contrast to the metallic parts of the city. A sun or artificial light source was drawn to show that plants get light energy even in space. Some parts had robots drawn using basic shapes, showing that farming was done by machines. This idea came from Interstellar again, where farming technology and sustainability were major themes [1]. The farmland represented the hope and survival of humans in space when Earth becomes unlivable.

Each of these three scenarios had its own main function in the code. The display functions were managed through GLUT, and different scenes could be switched by modifying display callbacks. The modular coding approach allowed all three scenes to work together smoothly. Team members used the same projection setup, coordinate range, and color schemes so that the visuals looked connected. The project also included small animations in each scene. Stars twinkled, robots moved, and spaceships floated across the screen. These effects were done using time-based updates in the main loop.

The project used basic OpenGL primitives like points, lines, and polygons. The points made stars, the lines made roads and paths, and polygons formed large surfaces like buildings or spaceships. Transformations such as translation, scaling, and rotation gave motion to these elements. The project also used RGB color blending to make glowing light effects that resembled cyberpunk visuals. The glowing lights were mainly inspired by Cyberpunk 2077 (2020) [3], where cities shine with neon signs and holographic effects. These styles gave the project a strong futuristic mood even in 2D form.

Some interactivity was also added. Mouse and keyboard inputs were used to control parts of the animation. The mouse could change the speed of moving vehicles, and the keyboard could trigger the robot's motion or spaceship launch. These small interactions showed the early stage of game development. In the future, the project can be developed further into a complete 2D space-themed game. The goal would be to let players explore the city, control robots, and complete missions like repairing a destroyed part or growing plants in the farmland. This future plan is inspired by open-world games like Cyberpunk 2077, where the player freely explores a large futuristic environment [2].

Testing was done after each module was finished. Each function was run separately to check if it drew correctly. Then all were combined and tested together. Adjustments were made to timing, coordinates, and colors to keep the animation smooth. Debugging was done in Code::Blocks, and errors were fixed step by step. The project ran successfully, and all three scenarios were displayed clearly. The animation was simple but effective, showing a lively futuristic space environment.

Overall, the methods used in this project were simple but creative. The use of OpenGL and C++ made the development process flexible and educational. Each part of the code taught something new about computer graphics, such as how shapes are drawn, how transformations work, and how colors can change the feeling of a scene. The modular design helped the team

work independently on different parts and combine them later. The final result showed that even a 2D project can represent complex futuristic ideas with good design and imagination. The combination of the three scenarios gave the project a complete story. The Space Metro Station showed technology and progress. The City View and Destroyed City showed both the beauty and danger of technology. The Space City Farmland showed hope and life beyond Earth. Together, they made a full vision of the future. The use of simple graphics did not reduce the impact; instead, it made the project easier to understand and more creative. The techniques used in this project can help in building larger projects later, including 3D games or educational simulations.

Background of the Problem

The world of technology and design is moving fast. Every year, people imagine new ideas about how the future will look. Modern movies and games show cities in space, robots working with humans, and new ways of living. These ideas inspire many students to build creative visual projects. The Futuristic Space City project was made to show a similar imagination through computer graphics. It was developed to visualize what a future city in space might look like and how it might work. The background of this project comes from the dream of building a human world beyond Earth. The project represents a city in space that is made of three different scenarios. These are the Space Metro Station, the City View and Destroyed City, and the Space City Farmland. Each part tells a story about human life, technology, and survival. The Space Metro Station shows the progress of technology and human settlement in space. The City View and Destroyed City show how technology can create beauty but also bring destruction. The Space City Farmland shows the need for food and sustainability even in outer space. Together, these scenarios form the complete vision of the Futuristic Space City.

The idea came from observing modern science fiction. Movies like Interstellar (2014) show the need for humans to leave Earth and build new homes in space [1]. Blade Runner 2049 (2017) shows the dark side of modern cities and technology [5]. Games like Cyberpunk 2077 (2020) present detailed futuristic cities full of machines, robots, and glowing lights [3]. All of these influenced the theme of this project. The aim was to combine beauty, technology, and imagination through computer graphics. It also helped to learn how OpenGL can be used to visualize complex ideas in a simple 2D form.

The background problem that inspired this project is the lack of interactive, creative, and visually appealing 2D environments that explain future concepts easily. Many beginners in computer graphics find it hard to design something futuristic without using heavy 3D tools. Most existing examples are complex and require high-level programming knowledge. This project tried to fill that gap by creating a futuristic space city using simple 2D OpenGL features. It showed that even simple primitives like points, lines, and polygons can create impressive results when used creatively.

The project also focuses on the educational side of computer graphics. Many students learn about shapes, transformations, and color theory but cannot connect them to real-world visual design. This project made that connection. It used modular functions to build each part of the city and showed how coding and imagination can work together. The modular approach also allowed team members to work separately on their parts. Each scene the Space Metro Station, the City View and Destroyed City, and the Space City Farmland was coded as a different module. Then all the modules were joined together to form one complete environment.

The Space Metro Station scenario shows the problem of human expansion and survival. As the Earth becomes overpopulated and polluted, people may need to move to space one day. The Space Metro Station in the project represents that dream. It shows large structures floating in dark space, connected by satellites and ships. Stars shine randomly in the background, giving

the scene a real space feeling. This idea was directly inspired by Interstellar, where humanity tries to survive by finding another planet [1]. The same emotional and visual feeling was represented here. The problem that inspired this scenario is not only technical but also social how humans will adapt when the Earth cannot support them anymore.

The City View and Destroyed City scenario addresses another important problem the balance between technology and destruction. Modern cities often depend too much on technology, which may lead to pollution, overuse of energy, and even wars. In Blade Runner 2049, the city is filled with neon lights, holograms, and tall towers, but the environment is dark and damaged [5]. This image was recreated in the project through bright lights and broken structures. The transition from a glowing city to a destroyed one represents both progress and failure. It reflects how human greed and carelessness can destroy what was once beautiful. This scenario warns about the dangers of uncontrolled technology.

The Space City Farmland scenario was developed to show the importance of sustainability. Even in the most advanced future, food and farming remain essential. In Interstellar, farming was a key part of the story because food shortages forced humans to search for new worlds [1]. This project used the same idea and showed green farmlands inside the space city. It was drawn with simple rectangles and grids, but it carried a deep meaning. It suggested that even in space, humans must find ways to produce food and live close to nature. This part of the project connects futuristic design with environmental awareness.

Another important problem that inspired the project is the lack of creative 2D simulation in learning. Most computer graphics courses focus on basic shapes and algorithms but do not often encourage creative scene design. The Futuristic Space City project aimed to change that. It was made to teach both programming and creativity. Students could learn how to use functions like gluOrtho2D for projection and transformations for movement. They could also learn how to use colors and animations to make a scene look alive. The idea was to show that creativity and code can be combined even in simple 2D programs.

The background of this project is also linked with the idea of future game design. Modern games like Cyberpunk 2077 use highly detailed environments with light effects and animations [3]. The Futuristic Space City project is a small step in that direction. The future plan is to turn this 2D environment into an interactive game where players can control robots, vehicles, or spaceships. The game could allow users to explore the city, fix broken systems, or grow food in the farmland. This plan adds another layer of motivation behind the project to transform learning projects into playable experiences.

From a technical point of view, the background problem also includes understanding how to make smooth 2D animation. Many students face difficulty in synchronizing motion, timing, and drawing in OpenGL. This project solved that by using the display loop efficiently. Each frame updated positions of objects like spaceships or cars. The stars twinkled using random functions. Robots and vehicles moved using translation. The simple but effective animation showed that smooth movement is possible even without complex physics. It also helped to understand the importance of real-time rendering in games and simulations.

Another motivation behind the project was to experiment with lighting and color. In movies like Blade Runner 2049 and games like Cyberpunk 2077, lights are not only decoration but a way to express emotion [3], [5]. Neon lights show energy, hope, and chaos at the same time. The project applied the same concept. Buildings were given glowing edges using bright colors like cyan and purple. The space background used deep blue and black to show mystery. This kind of color use helped to create a strong mood even without textures or 3D depth. It showed that emotional design can exist in code-based graphics too.

The Futuristic Space City project is more than a collection of code. It represents the connection between science fiction and learning. It was built from inspiration but aimed to teach real programming and design skills. The background of the problem was about how to represent

future life using simple tools. It was also about how to make learning more creative. In many ways, the project stands between imagination and technology just like how science fiction inspires real inventions.

The three scenarios together also reflect a timeline of human progress. The Space Metro Station shows the beginning of expansion. The City View and Destroyed City show the rise and fall of a civilization. The Space City Farmland shows recovery and sustainability. This storytelling pattern connects with both Interstellar and Blade Runner 2049 [1], [5]. It makes the project not only visual but also meaningful. The viewer can feel the story without any words.

In short, the background of the Futuristic Space City project lies in the idea of visualizing a possible future through simple computer graphics. It was made to solve the problem of limited creativity in 2D projects, to teach modular design, to explore the beauty of futuristic art, and to prepare for future game development. The project links science fiction imagination with real coding practice. It shows that even with simple shapes and colors, one can express complex ideas about technology, nature, and human survival.

Feature Set

First Scenario: Space City Station

1. Real-time metro train animation with start and stop.
2. Countdown and multiple trains with different lights.
3. Night sky with stars, meteors, UFOs, and rocks.
4. Astronauts, robots, and robot dogs moving and jumping.
5. Glowing lights, reflections, and transparent effects.
6. Background color changes for day and night feel.
7. Smooth wave motion for natural movement.
8. Random speed and position for sky objects.
9. Animation runs with a global time counter.
10. Keyboard used to control robots, astronauts, and robot dogs.
11. Mouse used to change train speed.
12. Text shows labels and on-screen messages.
13. Objects drawn with simple shapes like circles and lines.
14. Rotation and movement for all objects.
15. Blinking train lights and glowing roads.
16. Continuous animation using timers.
17. Modern futuristic buildings.
18. Falling meteors.
19. Small bushes and Mars-like red ground.
20. Constellation archive memory showing sky patterns.
21. Modular code with separate parts for train, stars, and robots.
22. Passenger seats on the station.

Second Scenario: Space City Farmland

1. Dynamic metro system
2. Animated night sky
3. Skyline and futuristic city silhouette
4. Futuristic agricultural field
5. Small futuristic dome farms (left and right)
6. Tall vertical farm tower

7. Solar panel arrays
8. Energy tower
9. Drones and system
10. Smart wetness system
11. Agricultural robots
12. Group vehicles.
13. Yellow rice grain particles
14. Interactivity and keyboard controls
15. Animation system
16. Lighting and transparency effects
17. Procedural randomization
18. Fully layered scene composition
19. Modern OpenGL utility functions

Third Scenario: Futuristic City and Destroyed City

1. Night sky filled with about 200 twinkling stars.
2. Sleek rocket launching into the air.
3. Falling meteor with a fiery trail.
4. Futuristic buildings in various unique shapes.
5. Rocky curved ground with a rugged texture.
6. Smooth road winding through the scene.
7. Narrow footpath for walking.
8. Ferrari-inspired car with a sleek, red design.
9. Lamborghini-inspired car with sharp, yellow lines.
10. Cybertruck-inspired car with a bold, angular look.
11. Sporty car with a low, fast design.
12. Wispy smoke drifting in the air.
13. Scattered fire debris on the ground.
14. Repair drones hovering with glowing tools.
15. Survivors standing amid the chaos.
16. Destroyed parts of cars scattered around.
17. Cracked, destroyed sections of the ground.
18. Mouse and Keyboard Handler for last two scenes.

Future Direction

The futuristic space city project has many possible paths for future development. The current version is a simple 2D visualization that shows the main idea of life in space. In the future, it can grow into a more advanced and interactive design. The next big goal is to develop a full 3D version of the project. This will make the city more realistic and allow users to explore it in detail. A major aim is to turn this project into a playable game. In that version, users can move through the Space Metro Station, explore the farmland, and walk around the city area. Robots, astronauts, flying cars, and other advanced machines can be added to make the world more active. The player will feel part of the futuristic city. The project has taken strong inspiration from science fiction works. The movie Interstellar presents rotating space habitats and realistic space environments [1]. Blade Runner 2049 shows dark futuristic cities filled with lights and tall buildings [5]. The game Cyberpunk 2077 displays crowded streets, holograms, and dynamic lighting [2]. These examples will help in the next stage to create a more detailed and emotional environment. The future version can combine the calm beauty

of Interstellar with the busy and bright look of Cyberpunk 2077. Artificial intelligence will also play a big role. Robots and drones can be made to move and react automatically. The city may include smart energy systems and digital assistants. These will make the city feel alive. Adding these features will also help students learn about real-world applications of AI and automation. Sound and motion will be improved in later versions. Future updates will include background music, city sounds, and space effects. The farmland will have soft natural sounds. The Space Metro Station will have mechanical noises. Such small details will make the world feel real and full of life.

The project can also become a strong educational tool. It can help students understand urban planning, renewable energy, and the balance between technology and nature. Teachers can use it to show how life in space might look. The city can be a model to teach environmental design and sustainability. Virtual reality can also be added later. It will allow users to enter the space city using VR headsets. They will be able to explore the Space Metro Station, city, and farmland as if they are really there. Multiplayer features can also be added so that many users can visit the same space city at the same time.

In summary, the project will move from a simple 2D model to a smart, interactive, and realistic experience. Each step will bring it closer to becoming a full educational and entertainment platform. It will help people imagine, learn, and explore the possible future of human life in space.

Conclusion

The futuristic space city project represents the dream of human life beyond Earth. It combines science, creativity, and imagination into one complete idea. The project is made up of three connected parts. The first part is the Space Metro Station, which represents science and progress. The second part is the City View and Destroyed City, which shows both the growth and the risks of technology. The third part is the Space City Farmland, which shows nature and peace. Together, these three parts form a complete vision of a balanced future city. The idea came from popular movies and games such as Interstellar, Blade Runner 2049, and Cyberpunk 2077. Each of these inspired parts of the city design. Interstellar encouraged the scientific and realistic design of space living. Blade Runner 2049 gave the dark, futuristic look of the urban area. Cyberpunk 2077 influenced the lighting, atmosphere, and city details. These inspirations helped build a unique and creative project.

The 2D version may look simple, but it expresses a big idea. It shows how humans can imagine new ways of living in space. The modular design and animations helped create different parts of the city step by step. Each building and area were made with care to make the world understandable and meaningful. The simple graphics show the concept clearly, and the next versions can expand this world even more. The project also teaches an important lesson about the future of technology. It shows that technology can help humans survive and grow, but it must be used with care. The City View and Destroyed City remind us of the risks of overusing technology. The Farmland shows that human life always needs nature. The Space Metro Station shows that humans always look forward to discovery and progress. In academic terms, the project is a strong example of creative thinking, teamwork, and technical learning. It uses imagination to express real ideas about sustainability, urban life, and future challenges. It can also motivate students to work on creative computer graphics and environmental design. The future plan to turn this work into a game makes it even more exciting. A game will let people explore the city, interact with the environment, and learn through experience. It can become a useful tool for education, science learning, and digital art.

In conclusion, the futuristic space city project is a small but meaningful step toward imagining life beyond Earth. It connects creativity and technology in a simple but powerful way. It helps

people think about how cities in space may look and how humans can live in balance with machines and nature. It is a project built with imagination but directed toward the future of real human exploration.

Implementation code

```
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <sstream>
#include <string>
#include <algorithm>
#define _USE_MATH_DEFINES
#include <OpenGL/glu.h>
#include <OpenGL/gl.h>
#include <GLUT/glut.h>
int currentScene = 1; // 1: scene1, 2: scene2, 3: scene3, 4: scene4
float commonTrainX = -1.5f;
float commonTrainSpeed = 0.008f;
float timeCounter = 0.0f;
// Scene 1 globals
float zoom1 = 1.0f;
float countdown1 = 10.0f;
int currentTrain1 = 1;
bool trainActive1 = false;
bool trainPaused1 = false;
bool trainStopped1 = false;
long long stopStartTime1 = 0;
long long pauseStartTime1 = 0;
long long totalPausedTime1 = 0;
float starX1[200];
float starY1[200];
float starAlpha1[200];
float rockX1[20];
float rockY1[20];
float rockS1[20];
float astronaut1X1 = -1.0f, astronaut2X1 = 0.0f, astronaut3X1 = 0.9f, astronaut4X1 = -0.4f;
float astronaut5X1 = 1.2f, astronaut6X1 = -1.2f;
float robotDog1X1 = -0.5f, robotDog2X1 = 1.0f;
float robot1X1 = 0.3f;
float robot2X1 = -0.3f, robot3X1 = 0.6f, robot4X1 = -0.6f;
float astronautVel1 = 0.0f;
float dogVel1 = 0.0f;
float robot1XVel1 = 0.0f;
float robot1YOffset1 = 0.0f;
float robot1JumpVel1 = 0.0f;
int numAstronauts1 = 6;
int numRobotDogs1 = 2;
int numRobots1 = 4;
float ufoX1[3];
float ufoY1[3];
float ufoSpeed1[3];
bool nightMode1 = false;
struct Meteor1 {
    float x, y, speed;
    bool active;
};
Meteor1 meteors1[2];
// Scene 2 globals
```

```
float rocketX2 = -1.5f;
float rocketSpeed2 = 0.01f;
float starX2[200];
float starY2[200];
float starAlpha2[200];
float tractorX2[2] = {-1.2f, 1.2f};
float tractorSpeed2[2] = {0.006f, 0.007f};
bool tractorFlip2[2] = {false,true};
float truckX2[2] = {1.4f,-1.4f};
float truckSpeed2[2] = {0.009f, 0.008f};
bool truckFlip2[2] = {true,false};
float robotArmAngle2 = 0.0f;
bool armUp2 = true;
int sunTick2 = 0;
float droneT2 = 0.0f;
float droneT22 = 0.0f;
float droneT32 = 0.0f;
const int NUM_WDRONES2 = 3;
float droneX2[NUM_WDRONES2] = {-1.10f, 0.05f, 1.00f};
float droneY2[NUM_WDRONES2] = {-0.12f, -0.15f, -0.12f};
float dronePhase2[NUM_WDRONES2] = {0.0f, 1.3f, 2.6f};
bool enableWaterDrone2[NUM_WDRONES2] = {true,true,true};
bool enableSky12 = true, enableSky22 = true;
int rainIntensity2 = 42;
float truckRobotArmAngle2 = 0.0f;
bool truckArmUp2 = true;
float driverHeadAngle2 = 0.0f;
bool driverHeadLeft2 = true;
const int NUM_GRAINS2 = 3200;
float grainX2[NUM_GRAINS2], grainY2[NUM_GRAINS2], grainA2[NUM_GRAINS2],
grainS2[NUM_GRAINS2], grainTint2[NUM_GRAINS2];
const int NUM_PATCHES2 = 3;
float patch2[NUM_PATCHES2][4] = {
   {-1.40f, -0.70f, -0.80f, -0.55f},
   {-0.20f, -0.70f, 0.30f, -0.55f},
   { 0.70f, -0.70f, 1.30f, -0.55f}
};
float wetness2[NUM_PATCHES2] = {0.0f, 0.0f, 0.0f};
// Scene 3 globals
bool windowsOn3 = true;
float carSpeed3 = 0.008f;
float rocketX3 = -1.5f;
float rocketSpeed3 = 0.01f;
bool ferrariMoving3 = true;
bool rocketMoving3 = true;
float starX3[200];
float starY3[200];
float starAlpha3[200];
struct Vehicle3 {
   float x;
   float y;
   float speed;
   bool flip;
   float scale;
   float r, g, b;
   bool isBus;
   int type;
};
Vehicle3 vehicles3[8];
float laneY3[4] = {-0.12f, -0.27f, -0.42f, -0.57f};
```

```
// Scene 4 globals
bool windowsOn4 = true;
float carSpeed4 = 0.008f;
float rocketX4 = -1.5f;
float rocketSpeed4 = 0.01f;
bool rocketMoving4 = true;
float rocketBaseSpeed4 = 0.01f;
bool meteorFalling4 = false;
bool meteorHit4 = false;
float meteorX4 = 0.0f;
float meteorY4 = 1.0f;
float meteorSize4 = 0.01f;
float meteorSpeed4 = -0.003f;
bool meteorPaused4 = false;
float debrisX4[100];
float debrisY4[100];
float debrisSpeed4[100];
float smokeAlpha4[50];
float smokeX4[50];
float smokeY4[50];
float fireAlpha4[50];
float fireX4[50];
float fireY4[50];
float fireColorR4[50];
float fireColorG4[50];
struct Drone4 {
    float x, y;
    float angle;
    float speed;
    float armAngle;
};
Drone4 drones4[10];
struct Survivor4 {
    float x, y;
    float anim;
};
Survivor4 survivors4[3];
float starX4[200];
float starY4[200];
float starAlpha4[200];
struct Vehicle4 {
    float x;
    float y;
    float speed;
    bool flip;
    float scale;
    float r, g, b;
    bool isBus;
    int type;
    bool destroyed;
};
Vehicle4 vehicles4[8];
float laneY4[4] = {-0.12f, -0.27f, -0.42f, -0.57f};
bool cityDestroyed4 = false;
bool metroDestroyed4 = false;
bool groundDestroyed4 = false;
bool roadDestroyed4 = false;
// Scene 1 functions
long long getAdjustedTime1() {
    long long now = glutGet(GLUT_ELAPSED_TIME);
```

```
    return trainPaused1 ? (pauseStartTime1 - totalPausedTime1) : (now - totalPausedTime1);
}
void init1() {
    for (int i = 0; i < 200; i++) {
        starX1[i] = (rand() % 300 - 150) / 100.0f;
        starY1[i] = (rand() % 200 - 100) / 100.0f;
        starAlpha1[i] = 0.5f + (rand() % 50) / 100.0f;
    }
    for (int i = 0; i < 20; i++) {
        rockX1[i] = -1.5f + (rand() % 300) / 100.0f;
        rockY1[i] = -1.0f + (rand() % 40) / 100.0f;
        rockS1[i] = 0.01f + (rand() % 10) / 1000.0f;
    }
    for (int i = 0; i < 3; i++) {
        ufoX1[i] = (rand() % 300 - 150) / 100.0f;
        ufoY1[i] = 0.5f + (rand() % 50) / 100.0f;
        ufoSpeed1[i] = 0.002f + (rand() % 30) / 10000.0f;
    }
    for (int i = 0; i < 2; i++) {
        meteors1[i].active = false;
    }
}
void drawStars1() {
    glPointSize(3);
    glBegin(GL_POINTS);
    for (int i = 0; i < 200; i++) {
        float alpha = starAlpha1[i] + 0.3f * sin(timeCounter + i);
        glColor4f(1.0f, 1.0f, 1.0f, nightMode1 ? alpha : alpha * 0.5f);
        glVertex2f(starX1[i], starY1[i]);
    }
    glEnd();
}
void drawCircle(float cx, float cy, float r, int num_segments = 100) {
    glBegin(GL_POLYGON);
    for (int i = 0; i < num_segments; i++) {
        float theta = 2.0f * M_PI * float(i) / float(num_segments);
        float x = r * cos(theta);
        float y = r * sin(theta);
        glVertex2f(x + cx, y + cy);
    }
    glEnd();
}
void drawEllipse(float cx, float cy, float rx, float ry, int num_segments = 100) {
    glBegin(GL_POLYGON);
    for (int i = 0; i < num_segments; i++) {
        float theta = 2.0f * M_PI * float(i) / float(num_segments);
        float x = rx * cos(theta);
        float y = ry * sin(theta);
        glVertex2f(x + cx, y + cy);
    }
    glEnd();
}
void drawString(float x, float y, const char* str) {
    glRasterPos2f(x, y);
    for (int i = 0; str[i] != '\0'; i++)
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, str[i]);
}
void drawMetroTrain1(float x, float y, float width, float height) {
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

```
float flicker = 0.8f + 0.2f * sin(timeCounter * 5.0f);
float segmentWidth = width / 4.0f;
float segmentHeight = height * 1.6f;
glColor4f(1.0f, 0.6f, 0.0f, 0.98f);
glBegin(GL_QUADS);
glVertex2f(x - width / 2, y - segmentHeight / 2);
glVertex2f(x + width / 2, y - segmentHeight / 2);
glVertex2f(x + width / 2, y + segmentHeight / 2);
glVertex2f(x - width / 2, y + segmentHeight / 2);
glEnd();
for (int i = 0; i < 4; i++) {
    float segX = x - width / 2 + segmentWidth * i + segmentWidth / 2;
    glColor4f(1.0f, 0.6f, 0.0f, 0.98f);
    glBegin(GL_POLYGON);
    glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 2);
    glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 2);
    glVertex2f(segX + segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
    glVertex2f(segX - segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
    glEnd();
    glColor4f(1.0f, 0.9f, 0.2f, 0.8f * flicker);
    glBegin(GL_LINE_LOOP);
    glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 2);
    glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 2);
    glVertex2f(segX + segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
    glVertex2f(segX - segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
    glEnd();
    glColor4f(0.0f, 0.9f, 1.0f, 0.95f);
    int numWindows = (i == 0 || i == 3) ? 2 : 3;
    float startX = segX - segmentWidth / 2 + 0.03f;
    float stepX = (segmentWidth - 0.06f) / (numWindows - 1);
    for (int j = 0; j < numWindows; j++) {
        float wx = startX + j * stepX;
        drawCircle(wx, y, 0.035f, 20);
        glColor4f(1.0f, 1.0f, 1.0f, 0.3f * flicker);
        drawCircle(wx + 0.01f, y + 0.01f, 0.01f, 10);
    }
}
glColor4f(1.0f, 0.6f, 0.0f, 0.98f);
glBegin(GL_POLYGON);
glVertex2f(x + width / 2, y - segmentHeight / 3);
glVertex2f(x + width / 2 + 0.2f, y - segmentHeight / 3);
glVertex2f(x + width / 2 + 0.2f, y + segmentHeight / 3);
glVertex2f(x + width / 2, y + segmentHeight / 3);
glEnd();
glPushMatrix();
glTranslatef(x + width / 2 + 0.2f, y, 0.0f);
glScalef(0.9f, 0.9f, 1.0f);
drawCircle(0.0f, 0.0f, 0.14f, 30);
glPopMatrix();
glBegin(GL_POLYGON);
glVertex2f(x - width / 2, y - segmentHeight / 3);
glVertex2f(x - width / 2 - 0.2f, y - segmentHeight / 3);
glVertex2f(x - width / 2 - 0.2f, y + segmentHeight / 3);
glVertex2f(x - width / 2, y + segmentHeight / 3);
glEnd();
glPushMatrix();
glTranslatef(x - width / 2 - 0.2f, y, 0.0f);
glScalef(-0.9f, 0.9f, 1.0f);
drawCircle(0.0f, 0.0f, 0.14f, 30);
glPopMatrix();
```

```
    glColor4f(1.0f, 1.0f, 0.8f, 0.95f * flicker);
    drawCircle(x + width / 2 + 0.2f, y, 0.04f, 20);
    glColor4f(1.0f, 1.0f, 0.8f, 0.3f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(x + width / 2 + 0.2f, y);
    glVertex2f(x + width / 2 + 0.5f, y + 0.1f);
    glVertex2f(x + width / 2 + 0.5f, y - 0.1f);
    glEnd();
    if (trainActive1 && !trainStopped1 && !trainPaused1) {
        glColor4f(0.0f, 0.5f, 1.0f, 0.7f * (0.5f + 0.5f * sin(timeCounter * 12.0f)));
        glBegin(GL_TRIANGLES);
        glVertex2f(x - width / 2 - 0.2f, y);
        glVertex2f(x - width / 2 - 0.3f, y + 0.06f);
        glVertex2f(x - width / 2 - 0.3f, y - 0.06f);
        glEnd();
    }
    glDisable(GL_BLEND);
}
void drawRobot1(float rx, float ry_offset, float move, bool isControllable = false) {
    glColor3f(0.7f, 0.7f, 0.7f);
    glBegin(GL_QUADS);
    glVertex2f(rx + move - 0.05f, -0.5f + ry_offset);
    glVertex2f(rx + move + 0.05f, -0.5f + ry_offset);
    glVertex2f(rx + move + 0.05f, -0.45f + ry_offset);
    glVertex2f(rx + move - 0.05f, -0.45f + ry_offset);
    glEnd();
    drawCircle(rx + move, -0.43f + ry_offset, 0.02f, 20);
    glColor3f(0.0f, 1.0f, 0.0f);
    drawCircle(rx + move, -0.43f + ry_offset, 0.005f, 10);
}
void drawPassengers1() {
    float move1 = 0.03f * sin(timeCounter * 0.6f);
    float move2 = 0.04f * sin(timeCounter * 0.7f + 1.0f);
    float move3 = 0.035f * sin(timeCounter * 0.8f + 2.0f);
    float move4 = 0.045f * sin(timeCounter * 0.4f + 3.0f);
    float move5 = 0.02f * sin(timeCounter * 0.5f + 4.0f);
    float move6 = 0.025f * sin(timeCounter * 0.9f + 5.0f);
    float move7 = 0.03f * sin(timeCounter * 0.6f + 6.0f);
    float move8 = 0.04f * sin(timeCounter * 0.7f + 7.0f);
    float move9 = 0.035f * sin(timeCounter * 0.8f + 8.0f);
    float move10 = 0.045f * sin(timeCounter * 0.4f + 9.0f);
    if (numAstronauts1 >= 1) {
        glColor3f(1.0f, 1.0f, 1.0f);
        drawCircle(astronaut1X1, -0.4f, 0.035f);
        glBegin(GL_QUADS);
        glVertex2f(astronaut1X1 - 0.02f, -0.45f);
        glVertex2f(astronaut1X1 + 0.02f, -0.45f);
        glVertex2f(astronaut1X1 + 0.02f, -0.5f);
        glVertex2f(astronaut1X1 - 0.02f, -0.5f);
        glEnd();
        glColor3f(0.2f, 0.8f, 1.0f);
        drawCircle(astronaut1X1 + 0.01f, -0.39f, 0.01f, 10);
    }
    if (numAstronauts1 >= 2) {
        glColor3f(1.0f, 1.0f, 1.0f);
        drawCircle(astronaut2X1 + move2, -0.4f, 0.035f);
        glBegin(GL_QUADS);
        glVertex2f(astronaut2X1 + move2 - 0.02f, -0.45f);
        glVertex2f(astronaut2X1 + move2 + 0.02f, -0.45f);
        glVertex2f(astronaut2X1 + move2 + 0.02f, -0.5f);
```

```
    glVertex2f(astronaut2X1 + move2 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(astronaut2X1 + move2 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 3) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(astronaut3X1 + move3, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(astronaut3X1 + move3 - 0.02f, -0.45f);
    glVertex2f(astronaut3X1 + move3 + 0.02f, -0.45f);
    glVertex2f(astronaut3X1 + move3 + 0.02f, -0.5f);
    glVertex2f(astronaut3X1 + move3 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(astronaut3X1 + move3 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 4) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(astronaut4X1 + move4, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(astronaut4X1 + move4 - 0.02f, -0.45f);
    glVertex2f(astronaut4X1 + move4 + 0.02f, -0.45f);
    glVertex2f(astronaut4X1 + move4 + 0.02f, -0.5f);
    glVertex2f(astronaut4X1 + move4 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(astronaut4X1 + move4 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 5) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(astronaut5X1 + move5, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(astronaut5X1 + move5 - 0.02f, -0.45f);
    glVertex2f(astronaut5X1 + move5 + 0.02f, -0.45f);
    glVertex2f(astronaut5X1 + move5 + 0.02f, -0.5f);
    glVertex2f(astronaut5X1 + move5 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(astronaut5X1 + move5 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 6) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(astronaut6X1 + move6, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(astronaut6X1 + move6 - 0.02f, -0.45f);
    glVertex2f(astronaut6X1 + move6 + 0.02f, -0.45f);
    glVertex2f(astronaut6X1 + move6 + 0.02f, -0.5f);
    glVertex2f(astronaut6X1 + move6 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(astronaut6X1 + move6 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 7) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(-0.7f + move7, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(-0.7f + move7 - 0.02f, -0.45f);
    glVertex2f(-0.7f + move7 + 0.02f, -0.45f);
    glVertex2f(-0.7f + move7 + 0.02f, -0.5f);
```

```
    glVertex2f(-0.7f + move7 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(-0.7f + move7 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 8) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(0.7f + move8, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(0.7f + move8 - 0.02f, -0.45f);
    glVertex2f(0.7f + move8 + 0.02f, -0.45f);
    glVertex2f(0.7f + move8 + 0.02f, -0.5f);
    glVertex2f(0.7f + move8 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(0.7f + move8 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 9) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(-0.8f + move9, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(-0.8f + move9 - 0.02f, -0.45f);
    glVertex2f(-0.8f + move9 + 0.02f, -0.45f);
    glVertex2f(-0.8f + move9 + 0.02f, -0.5f);
    glVertex2f(-0.8f + move9 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(-0.8f + move9 + 0.01f, -0.39f, 0.01f, 10);
}
if (numAstronauts1 >= 10) {
    glColor3f(1.0f, 1.0f, 1.0f);
    drawCircle(0.8f + move10, -0.4f, 0.035f);
    glBegin(GL_QUADS);
    glVertex2f(0.8f + move10 - 0.02f, -0.45f);
    glVertex2f(0.8f + move10 + 0.02f, -0.45f);
    glVertex2f(0.8f + move10 + 0.02f, -0.5f);
    glVertex2f(0.8f + move10 - 0.02f, -0.5f);
    glEnd();
    glColor3f(0.2f, 0.8f, 1.0f);
    drawCircle(0.8f + move10 + 0.01f, -0.39f, 0.01f, 10);
}
 if (numRobotDogs1 >= 1) {
    glColor3f(0.5f, 0.5f, 0.5f);
    glBegin(GL_QUADS);
    glVertex2f(robotDog1X1 - 0.04f, -0.47f);
    glVertex2f(robotDog1X1 + 0.04f, -0.47f);
    glVertex2f(robotDog1X1 + 0.04f, -0.45f);
    glVertex2f(robotDog1X1 - 0.04f, -0.45f);
    glEnd();
    drawCircle(robotDog1X1 + 0.04f, -0.46f, 0.015f);
    glBegin(GL_LINES);
    glVertex2f(robotDog1X1 - 0.03f, -0.47f);
    glVertex2f(robotDog1X1 - 0.03f, -0.5f);
    glVertex2f(robotDog1X1 - 0.01f, -0.47f);
    glVertex2f(robotDog1X1 - 0.01f, -0.5f);
    glVertex2f(robotDog1X1 + 0.01f, -0.47f);
    glVertex2f(robotDog1X1 + 0.01f, -0.5f);
    glVertex2f(robotDog1X1 + 0.03f, -0.47f);
    glVertex2f(robotDog1X1 + 0.03f, -0.5f);
    glEnd();
```

```
        glColor3f(1.0f, 0.0f, 0.0f);
        drawCircle(robotDog1X1 + 0.045f, -0.46f, 0.005f);
    }
    if (numRobotDogs1 >= 2) {
        glColor3f(0.5f, 0.5f, 0.5f);
        glBegin(GL_QUADS);
        glVertex2f(robotDog2X1 + move1 - 0.04f, -0.47f);
        glVertex2f(robotDog2X1 + move1 + 0.04f, -0.47f);
        glVertex2f(robotDog2X1 + move1 + 0.04f, -0.45f);
        glVertex2f(robotDog2X1 + move1 - 0.04f, -0.45f);
        glEnd();
        drawCircle(robotDog2X1 + move1 + 0.04f, -0.46f, 0.015f);
        glBegin(GL_LINES);
        glVertex2f(robotDog2X1 + move1 - 0.03f, -0.47f);
        glVertex2f(robotDog2X1 + move1 - 0.03f, -0.5f);
        glVertex2f(robotDog2X1 + move1 - 0.01f, -0.47f);
        glVertex2f(robotDog2X1 + move1 - 0.01f, -0.5f);
        glVertex2f(robotDog2X1 + move1 + 0.01f, -0.47f);
        glVertex2f(robotDog2X1 + move1 + 0.01f, -0.5f);
        glVertex2f(robotDog2X1 + move1 + 0.03f, -0.47f);
        glVertex2f(robotDog2X1 + move1 + 0.03f, -0.5f);
        glEnd();
        glColor3f(1.0f, 0.0f, 0.0f);
        drawCircle(robotDog2X1 + move1 + 0.045f, -0.46f, 0.005f);
    }
    if (numRobots1 >= 1) {
        drawRobot1(robot1X1, robot1YOffset1, 0.0f, true);
    }
    if (numRobots1 >= 2) {
        drawRobot1(robot2X1, 0.0f, move2);
    }
    if (numRobots1 >= 3) {
        drawRobot1(robot3X1, 0.0f, move3);
    }
    if (numRobots1 >= 4) {
        drawRobot1(robot4X1, 0.0f, move4);
    }
    if (numRobots1 >= 5) {
        drawRobot1(-0.9f, 0.0f, move5);
    }
    if (numRobots1 >= 6) {
        drawRobot1(0.9f, 0.0f, move6);
    }
    if (numRobots1 >= 7) {
        drawRobot1(-1.0f, 0.0f, move7);
    }
    if (numRobots1 >= 8) {
        drawRobot1(1.0f, 0.0f, move8);
    }
    if (numRobots1 >= 9) {
        drawRobot1(-0.8f, 0.0f, move9);
    }
    if (numRobots1 >= 10) {
        drawRobot1(0.8f, 0.0f, move10);
    }
}
void drawAdditionalObjects1() {

    glColor3f(0.4f, 0.4f, 0.4f);
    for (float bx = -1.0f; bx <= 1.0f; bx += 0.5f) {
```

```
        glBegin(GL_QUADS);
        glVertex2f(bx - 0.1f, -0.48f);
        glVertex2f(bx + 0.1f, -0.48f);
        glVertex2f(bx + 0.1f, -0.46f);
        glVertex2f(bx - 0.1f, -0.46f);
        glEnd();
        glBegin(GL_LINES);
        glVertex2f(bx - 0.1f, -0.48f);
        glVertex2f(bx - 0.1f, -0.5f);
        glVertex2f(bx + 0.1f, -0.48f);
        glVertex2f(bx + 0.1f, -0.5f);
        glEnd();
    }
    glColor3f(0.0f, 0.8f, 0.0f);
    for (float px = -1.2f; px <= 1.2f; px += 0.8f) {
        glBegin(GL_LINES);
        glVertex2f(px, -0.5f);
        glVertex2f(px, -0.4f);
        glEnd();
        drawCircle(px - 0.02f, -0.42f, 0.01f, 20);
        drawCircle(px + 0.02f, -0.42f, 0.01f, 20);
    }
}
void drawStation1() {

    glColor3f(0.1f, 0.1f, 0.2f);
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.5f);
    glVertex2f(1.5f, -0.5f);
    glVertex2f(1.5f, -0.55f);
    glVertex2f(-1.5f, -0.55f);
    glEnd();

    glEnable(GL_BLEND);
    glColor4f(0.0f, 0.5f, 1.0f, 0.5f * (0.5f + 0.5f * sin(timeCounter * 2.0f)));
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.55f);
    glVertex2f(1.5f, -0.55f);
    glVertex2f(1.5f, -0.6f);
    glVertex2f(-1.5f, -0.6f);
    glEnd();
    float glowAlpha = 0.5f * (0.5f + 0.5f * sin(timeCounter * 2.0f));
    if (trainActive1 && !trainStopped1 && !trainPaused1) {
        glColor4f(1.0f, 0.0f, 0.0f, glowAlpha);
    } else {
        glColor4f(0.0f, 1.0f, 0.0f, glowAlpha);
    }
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.51f);
    glVertex2f(1.5f, -0.51f);
    glVertex2f(1.5f, -0.54f);
    glVertex2f(-1.5f, -0.54f);
    glEnd();
    glColor3f(0.3f, 0.3f, 0.6f);
    glLineWidth(3.0f);
    glBegin(GL_LINES);
    glVertex2f(-1.5f, -0.51f);
    glVertex2f(1.5f, -0.51f);
    glVertex2f(-1.5f, -0.54f);
    glVertex2f(1.5f, -0.54f);
```

```
glEnd();
glLineWidth(1.0f);
glColor3f(0.4f, 0.4f, 0.8f);
for (float sx = -1.4f; sx < 1.4f; sx += 0.1f) {
   glBegin(GL_QUADS);
   glVertex2f(sx - 0.02f, -0.505f);
   glVertex2f(sx + 0.02f, -0.505f);
   glVertex2f(sx + 0.02f, -0.55f);
   glVertex2f(sx - 0.02f, -0.55f);
   glEnd();
}
//
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glColor4f(0.1f, 0.1f, 0.15f, 0.3f); // Transparent dark color for roof
glBegin(GL_POLYGON);
glVertex2f(-1.5f, -0.5f); // Left base
glVertex2f(-1.3f, 0.5f); // Left curve up
glVertex2f(0.0f, 0.7f); // Center peak
glVertex2f(1.3f, 0.5f); // Right curve up
glVertex2f(1.5f, -0.5f); // Right base
glEnd();
// Roof outline with glowing effect
glColor4f(0.0f, 0.8f, 1.0f, 0.8f * (0.5f + 0.5f * sin(timeCounter * 2.0f)));
glBegin(GL_LINE_LOOP);
glVertex2f(-1.5f, -0.5f);
glVertex2f(-1.3f, 0.5f);
glVertex2f(0.0f, 0.7f);
glVertex2f(1.3f, 0.5f);
glVertex2f(1.5f, -0.5f);
glEnd();
// Additional roof details (cross beams)
glColor4f(0.3f, 0.3f, 0.4f, 0.5f); // Slightly transparent beams
for (float x = -1.2f; x <= 1.2f; x += 0.4f) {
   glBegin(GL_LINES);
   glVertex2f(x, -0.5f);
   glVertex2f(x, 0.5f + (x / 1.3f) * 0.2f); // Adjust height based on curve
   glEnd();
}
// Draw ellipse shape at the middle and top of the polygon roof
glColor4f(0.0f, 0.2f, 0.4f, 0.3f); // Reduced opacity
drawEllipse(0.0f, 0.6f, 0.8f, 0.15f, 50); // Wide and short ellipse at top center
// Glowing outline for ellipse
glColor4f(0.0f, 1.0f, 1.0f, 0.8f * (0.5f + 0.5f * sin(timeCounter * 3.0f)));
glLineWidth(2.0f);
glBegin(GL_LINE_LOOP);
for (int i = 0; i < 50; i++) {
   float theta = 2.0f * M_PI * float(i) / 50.0f;
   float x = 0.8f * cos(theta);
   float y = 0.15f * sin(theta);
   glVertex2f(x + 0.0f, y + 0.6f);
}
glEnd();
glLineWidth(1.0f);
// Draw "Rahad's Space Station" text centered inside the ellipse
float flicker = 0.8f + 0.2f * sin(timeCounter * 3.0f);
glColor3f(0.0f, flicker, 1.0f);
drawString(-0.38f, 0.58f, "Rahad's Space Station");
// Draw pillars (all black)
glColor3f(0.0f, 0.0f, 0.0f); // Black color for all pillars
```

```
    for (float px = -1.2f; px <= 1.2f; px += 0.4f) {
        glBegin(GL_QUADS);
        glVertex2f(px - 0.01f, -0.5f);
        glVertex2f(px + 0.01f, -0.5f);
        glVertex2f(px + 0.01f, 0.2f);
        glVertex2f(px - 0.01f, 0.2f);
        glEnd();
        glColor3f(1.0f, 1.0f, 0.0f);
        drawCircle(px, 0.1f, 0.005f, 10);
    }
   std::ostringstream oss;
std::string displayText;
if (trainActive1 && trainStopped1) {
    long long remainingTime = 5 - (getAdjustedTime1() - stopStartTime1) / 1000;
    remainingTime = (std::max)(0LL, (std::min)(5LL, remainingTime));
    oss << "Train will depart in " << static_cast<int>(remainingTime) << " sec";
    displayText = oss.str();
    oss.str("");
} else if (trainActive1) {
    oss << "Train Rahad0A" << currentTrain1 << " Passing";
    displayText = oss.str();
    oss.str("");
} else if (countdown1 > 0) {
    oss << "Next Train: " << static_cast<int>(countdown1) << "s";
    displayText = oss.str();
    oss.str("");
} else {
    displayText = "Train Arriving";
}
    glColor3f(0.0f, 0.4f, 0.8f);
    glBegin(GL_QUADS);
    glVertex2f(-0.5f, 0.05f);
    glVertex2f(0.5f, 0.05f);
    glVertex2f(0.5f, 0.15f);
    glVertex2f(-0.5f, 0.15f);
    glEnd();
    glColor3f(1.0f, 1.0f, 0.0f);
    drawString(-0.4f, 0.09f, displayText.c_str());
    glDisable(GL_BLEND);
}
void drawFuturisticBackground1() {

    float flicker = 0.8f + 0.2f * sin(timeCounter * 4.0f);
    glColor3f(0.0f, 0.8f, 1.0f);
    glBegin(GL_QUADS);
    glVertex2f(-1.4f, -0.55f);
    glVertex2f(-1.2f, -0.55f);
    glVertex2f(-1.2f, 0.9f);
    glVertex2f(-1.4f, 0.9f);
    glEnd();

    // Antenna on top
    glColor3f(0.5f, 0.5f, 0.5f);
    glBegin(GL_LINES);
    glVertex2f(-1.3f, 0.9f);
    glVertex2f(-1.3f, 1.1f);
    glEnd();
    drawCircle(-1.3f, 1.1f, 0.02f);
    // Windows
    glColor3f(1.0f, 1.0f, 0.0f); // Yellow lights
```

```
for (float ly = -0.55f; ly < 0.9f; ly += 0.1f) {
    drawCircle(-1.3f, ly, 0.01f, 10);
}
// Building 2: Polygonal structure with dome
glColor3f(0.2f, 0.3f, 0.5f); // Opaque
glBegin(GL_POLYGON);
glVertex2f(0.0f, -0.55f);
glVertex2f(0.3f, -0.55f);
glVertex2f(0.4f, 0.4f);
glVertex2f(0.2f, 0.5f);
glVertex2f(0.0f, 0.4f);
glEnd();
// Dome on top
glColor3f(0.1f, 0.1f, 0.3f);
drawCircle(0.2f, 0.5f, 0.15f, 50);
// Outline with flicker
glColor4f(1.0f, 0.9f, 0.2f, flicker);
glBegin(GL_LINE_LOOP);
glVertex2f(0.0f, -0.55f);
glVertex2f(0.3f, -0.55f);
glVertex2f(0.4f, 0.4f);
glVertex2f(0.2f, 0.5f);
glVertex2f(0.0f, 0.4f);
glEnd();
// Building 3: Rectangular with horizontal lines and solar panels
glColor3f(0.4f, 0.4f, 0.6f); // Opaque
glBegin(GL_QUADS);
glVertex2f(1.0f, -0.55f);
glVertex2f(1.3f, -0.55f);
glVertex2f(1.3f, 0.5f);
glVertex2f(1.0f, 0.5f);
glEnd();
// Solar panels
glColor3f(0.0f, 0.0f, 0.8f);
glBegin(GL_QUADS);
glVertex2f(1.0f, 0.5f);
glVertex2f(1.3f, 0.5f);
glVertex2f(1.4f, 0.6f);
glVertex2f(1.1f, 0.6f);
glEnd();
// Horizontal lines with flicker
glColor4f(1.0f, 1.0f, 0.0f, flicker);
for (float ly = -0.55f; ly < 0.5f; ly += 0.05f) {
    glBegin(GL_LINES);
    glVertex2f(1.0f, ly);
    glVertex2f(1.3f, ly);
    glEnd();
}
// Building 4: Central command center with beams
glColor3f(0.3f, 0.3f, 0.5f); // Opaque
glBegin(GL_QUADS);
glVertex2f(-0.5f, -0.55f);
glVertex2f(0.5f, -0.55f);
glVertex2f(0.5f, 0.3f);
glVertex2f(-0.5f, 0.3f);
glEnd();
// Vertical beams
glColor4f(1.0f, 0.8f, 1.0f, flicker);
for (float lx = -0.5f; lx < 0.5f; lx += 0.1f) {
    glBegin(GL_LINES);
```

```
        glVertex2f(lx, -0.55f);
        glVertex2f(lx, 0.3f);
        glEnd();
    }
    // Top dish
    glColor3f(0.6f, 0.6f, 0.6f);
    drawCircle(0.0f, 0.3f, 0.1f, 30);
    // Building 5: Slanted research lab with lights
    glColor3f(0.2f, 0.4f, 0.6f); // Opaque
    glBegin(GL_POLYGON);
    glVertex2f(-0.8f, -0.55f);
    glVertex2f(-0.6f, -0.55f);
    glVertex2f(-0.5f, 0.6f);
    glVertex2f(-0.7f, 0.6f);
    glEnd();
    // Lights
    glColor3f(1.0f, 0.0f, 0.0f); // Red lights
    for (float ly = -0.55f; ly < 0.6f; ly += 0.15f) {
        drawCircle(-0.65f, ly, 0.01f, 10);
    }
    // Slanted lines with flicker
    glColor4f(1.0f, 1.0f, 1.0f, flicker);
    for (float ly = -0.55f; ly < 0.6f; ly += 0.1f) {
        glBegin(GL_LINES);
        glVertex2f(-0.8f, ly);
        glVertex2f(-0.6f, ly + 0.05f);
        glEnd();
    }
    // Additional Building 6: Habitat module on left
    glColor3f(0.1f, 0.5f, 0.7f); // Opaque
    glBegin(GL_QUADS);
    glVertex2f(-1.0f, -0.55f);
    glVertex2f(-0.8f, -0.55f);
    glVertex2f(-0.8f, 0.3f);
    glVertex2f(-1.0f, 0.3f);
    glEnd();
    // Windows
    glColor3f(0.8f, 0.8f, 1.0f);
    for (float ly = -0.4f; ly < 0.3f; ly += 0.1f) {
        drawCircle(-0.9f, ly, 0.02f, 20);
    }
    // Additional Building 7: Power generator on right
    glColor3f(0.5f, 0.2f, 0.2f); // Opaque
    glBegin(GL_QUADS);
    glVertex2f(0.6f, -0.55f);
    glVertex2f(0.9f, -0.55f);
    glVertex2f(0.9f, 0.2f);
    glVertex2f(0.6f, 0.2f);
    glEnd();
    // Glowing core
    glEnable(GL_BLEND);
    glColor4f(1.0f, 0.5f, 0.0f, 0.8f * flicker);
    drawCircle(0.75f, -0.2f, 0.05f, 30);
    glDisable(GL_BLEND);
}
void drawMarsGround1() {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glBegin(GL_QUADS);
```

```
    glColor4f(0.3f, 0.15f, 0.1f, 1.0f);
    glVertex2f(-1.5f, -1.0f);
    glVertex2f(1.5f, -1.0f);
    glColor4f(0.4f, 0.2f, 0.1f, 1.0f);
    glVertex2f(1.5f, -0.6f);
    glVertex2f(-1.5f, -0.6f);
    glEnd();

    glColor3f(0.25f, 0.12f, 0.08f); // Darker rock color
    for (int i = 0; i < 10; i++) {
        float rx = rockX1[i % 20];
        float ry = rockY1[i % 20] - 0.1f;
        float rs = rockS1[i % 20] * 5.0f;
        glBegin(GL_POLYGON);
        for (int j = 0; j < 5; j++) {
            float theta = 2.0f * M_PI * float(j) / 5.0f;
            float x = rx + rs * cos(theta) * (0.8f + 0.2f * rand() / RAND_MAX);
            float y = ry + rs * sin(theta) * (0.8f + 0.2f * rand() / RAND_MAX);
            glVertex2f(x, y);
        }
        glEnd();
    }
    // Constellation archive lines glowing with memory
    float pulse = 0.5f + 0.5f * sin(timeCounter * 1.7f);
    glColor4f(0.6f, 0.8f, 1.0f, 0.7f * pulse); // Soft blue celestial glow
    glLineWidth(2.0f);
    glBegin(GL_LINES);
    // Orion constellation
    glVertex2f(-1.2f, -0.9f); glVertex2f(-0.9f, -0.85f);
    glVertex2f(-0.9f, -0.85f); glVertex2f(-0.6f, -0.8f);
    // Big Dipper constellation
    glVertex2f(-0.3f, -0.9f); glVertex2f(0.0f, -0.85f);
    glVertex2f(0.0f, -0.85f); glVertex2f(0.3f, -0.8f);
    glVertex2f(0.3f, -0.8f); glVertex2f(0.6f, -0.75f);
    // Custom Martian constellation
    glVertex2f(0.9f, -0.9f); glVertex2f(1.1f, -0.85f);
    glVertex2f(1.1f, -0.85f); glVertex2f(1.3f, -0.8f);
    glEnd();
    glLineWidth(1.0f);
    // Memory nodes glowing at constellation points
    glColor4f(1.0f, 0.9f, 0.6f, 0.8f * pulse); // Warm golden glow
    drawCircle(-1.2f, -0.9f, 0.015f, 10); // Orion start
    drawCircle(-0.9f, -0.85f, 0.015f, 10);
    drawCircle(-0.6f, -0.8f, 0.015f, 10);
    drawCircle(-0.3f, -0.9f, 0.015f, 10); // Dipper start
    drawCircle(0.0f, -0.85f, 0.015f, 10);
    drawCircle(0.3f, -0.8f, 0.015f, 10);
    drawCircle(0.6f, -0.75f, 0.015f, 10);
    drawCircle(0.9f, -0.9f, 0.015f, 10); // Martian start
    drawCircle(1.1f, -0.85f, 0.015f, 10);
    drawCircle(1.3f, -0.8f, 0.015f, 10);
    // Central archive core symbolizing preserved memory
    glColor4f(0.8f, 0.6f, 1.0f, 0.9f * pulse); // Lavender core glow
    drawCircle(0.0f, -0.75f, 0.03f, 15);
    glDisable(GL_BLEND);
}
void drawGlowingSky1() {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

```
    glBegin(GL_QUADS);
    if (nightMode1) {
        glColor4f(0.0f, 0.0f, 0.1f, 1.0f);
        glVertex2f(-1.5f, 1.0f);
        glColor4f(0.0f, 0.0f, 0.1f, 1.0f);
        glVertex2f(1.5f, 1.0f);
        glColor4f(0.0f, 0.1f, 0.3f, 1.0f);
        glVertex2f(1.5f, -0.6f);
        glColor4f(0.0f, 0.1f, 0.3f, 1.0f);
        glVertex2f(-1.5f, -0.6f);
    } else {
        glColor4f(0.1f, 0.3f, 0.6f, 1.0f);
        glVertex2f(-1.5f, 1.0f);
        glColor4f(0.1f, 0.3f, 0.6f, 1.0f);
        glVertex2f(1.5f, 1.0f);
        glColor4f(0.2f, 0.6f, 1.0f, 1.0f);
        glVertex2f(1.5f, -0.6f);
        glColor4f(0.2f, 0.6f, 1.0f, 1.0f);
        glVertex2f(-1.5f, -0.6f);
    }
    glEnd();
    glDisable(GL_BLEND);
}
void drawUFOs1() {

    for (int i = 0; i < 3; i++) {
        glColor3f(0.5f, 0.5f, 0.5f);
        drawCircle(ufoX1[i], ufoY1[i], 0.05f, 20);
        glColor3f(0.8f, 0.8f, 1.0f);
        glBegin(GL_POLYGON);
        for (int j = 0; j < 20; j++) {
            float theta = M_PI * float(j) / 19.0f;
            float dx = 0.03f * cos(theta);
            float dy = 0.03f * sin(theta);
            glVertex2f(ufoX1[i] + dx, ufoY1[i] + dy);
        }
        glEnd();
        float flicker = 0.5f + 0.5f * sin(timeCounter * 10 + i);
        glColor4f(1.0f, 1.0f, 0.0f, flicker);
        drawCircle(ufoX1[i], ufoY1[i] - 0.05f, 0.01f, 10);
    }
}
void drawMeteors1() {
    glLineWidth(2.0f);
    glBegin(GL_LINES);
    for (int i = 0; i < 2; i++) {
        if (meteors1[i].active) {
            glColor3f(1.0f, 1.0f, 0.8f);
            glVertex2f(meteors1[i].x, meteors1[i].y);
            glColor3f(1.0f, 0.5f, 0.0f);
            glVertex2f(meteors1[i].x - 0.02f, meteors1[i].y + 0.1f);
        }
    }
    glEnd();
    glLineWidth(1.0f);
}
void drawScene1() {
    glScalef(zoom1, zoom1, 1.0f);
    drawGlowingSky1();
    drawStars1();
```

```
    drawMeteors1();
    drawUFOs1();
    drawFuturisticBackground1();
    drawMarsGround1();
    drawStation1();
    drawAdditionalObjects1();
    drawPassengers1();
    if (trainActive1)
        drawMetroTrain1(commonTrainX, -0.45f, 1.4f, 0.14f);
}
void updateScene1() {
    long long adjustedTime1 = getAdjustedTime1();
    if (trainActive1) {
        if (!trainStopped1) {
            if (!trainPaused1) {
                commonTrainX += commonTrainSpeed;
            }
            if ((commonTrainX - commonTrainSpeed) < 0.0f && commonTrainX >= 0.0f) {
                commonTrainX = 0.0f;
                trainStopped1 = true;
                stopStartTime1 = adjustedTime1;
            }
        } else {
            if (adjustedTime1 - stopStartTime1 >= 5000) {
                trainStopped1 = false;
                if (currentTrain1 == 1) {
                    numAstronauts1 = 10;
                    numRobots1 = 10;
                    numRobotDogs1 = 2;
                } else if (currentTrain1 == 2) {
                    numAstronauts1 = 5;
                    numRobots1 = 5;
                    numRobotDogs1 = 1;
                } else if (currentTrain1 == 3) {
                    numAstronauts1 = 8;
                    numRobots1 = 8;
                    numRobotDogs1 = 2;
                } else if (currentTrain1 == 4) {
                    numAstronauts1 = 3;
                    numRobots1 = 3;
                    numRobotDogs1 = 0;
                } else if (currentTrain1 == 5) {
                    numAstronauts1 = 6;
                    numRobots1 = 6;
                    numRobotDogs1 = 1;
                }
            }
        }
        if (commonTrainX > 2.0f) {
            trainActive1 = false;
            commonTrainX = -1.5f;
            countdown1 = 10.0f;
            currentTrain1++;
            if (currentTrain1 > 5) currentTrain1 = 1;
            trainStopped1 = false;
            currentScene = 2; // Shift to scene 2
        }
    } else {
        if (!trainPaused1) {
            countdown1 -= 0.016f;
```

```
        }
        if (countdown1 <= 0.0f) {
            trainActive1 = true;
            commonTrainX = -1.5f;
            trainStopped1 = false;
        }
    }
    timeCounter += 0.05f;

    astronaut1X1 += astronautVel1;
    astronaut1X1 = (std::max)(-1.5f, (std::min)(1.5f, astronaut1X1));
    robotDog1X1 += dogVel1;
    robotDog1X1 = (std::max)(-1.5f, (std::min)(1.5f, robotDog1X1));
    robot1X1 += robot1XVel1;
    robot1X1 = (std::max)(-1.5f, (std::min)(1.5f, robot1X1));
    robot1YOffset1 += robot1JumpVel1;
    robot1JumpVel1 -= 0.001f;
    if (robot1YOffset1 < 0.0f) {
        robot1YOffset1 = 0.0f;
        robot1JumpVel1 = 0.0f;
    }
    for (int i = 0; i < 3; i++) {
        ufoX1[i] += ufoSpeed1[i];
        if (ufoX1[i] > 1.5f) ufoX1[i] = -1.5f;
    }
    for (int i = 0; i < 2; i++) {
        if (meteors1[i].active) {
            meteors1[i].y -= meteors1[i].speed;
            if (meteors1[i].y < -1.0f) meteors1[i].active = false;
        } else {
            if (rand() % 200 < 1) {
                meteors1[i].active = true;
                meteors1[i].x = (rand() % 300 - 150) / 100.0f;
                meteors1[i].y = 1.0f;
                meteors1[i].speed = 0.01f + (rand() % 20) / 1000.0f;
            }
        }
    }
}
// Scene 2 functions
void drawCircle2(float cx,float cy,float r,int seg=80){
    drawCircle(cx, cy, r, seg); // Use common drawCircle
}
void drawBeautifulMoon2(float x, float y, float r) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glColor4f(1.0f, 1.0f, 0.9f, 0.15f);
    for (int i = 0; i < 5; i++) {
        drawCircle2(x, y, r * (1.2f + i * 0.1f), 60);
    }

    glBegin(GL_POLYGON);
    for (int i = 0; i < 120; i++) {
        float theta = 2.0f * M_PI * i / 120.0f;
        float shade = 0.9f + 0.1f * sinf(theta * 4.0f);
        glColor4f(0.95f * shade, 0.95f * shade, 0.9f * shade, 1.0f);
        glVertex2f(x + r * cos(theta), y + r * sin(theta));
    }
    glEnd();
```

```
    glColor4f(0.8f, 0.8f, 0.8f, 0.9f);
    drawCircle(x - r * 0.3f, y + r * 0.25f, r * 0.15f, 30);
    drawCircle(x + r * 0.2f, y - r * 0.15f, r * 0.10f, 30);
    drawCircle(x - r * 0.1f, y - r * 0.2f, r * 0.08f, 30);
    glColor4f(1.0f, 1.0f, 0.95f, 0.4f);
    drawCircle(x + r * 0.1f, y + r * 0.1f, r * 0.35f, 40);
    glDisable(GL_BLEND);
}
void drawEllipse2(float cx,float cy,float rx,float ry,int seg=48){
    drawEllipse(cx, cy, rx, ry, seg); // Use common
}
void drawRoundedRect2(float x1,float y1,float x2,float y2,float r){

    float w = x2-x1, h = y2-y1;
    r = fminf(r, fminf(fabsf(w), fabsf(h))*0.5f);
    glBegin(GL_QUADS);
    glVertex2f(x1+r,y1); glVertex2f(x2-r,y1); glVertex2f(x2-r,y2); glVertex2f(x1+r,y2);
    glEnd();

    glBegin(GL_QUADS);
    glVertex2f(x1,y1+r); glVertex2f(x1+r,y1+r); glVertex2f(x1+r,y2-r); glVertex2f(x1,y2-r);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(x2-r,y1+r); glVertex2f(x2,y1+r); glVertex2f(x2,y2-r); glVertex2f(x2-r,y2-r);
    glEnd();
    int seg = 14;
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(x1+r,y2-r);
    for(int i=0;i<=seg;i++){ float th = float(M_PI)/2 + float(M_PI)/2*i/seg; glVertex2f(x1+r + r*cosf(th), y2-r +
r*sinf(th)); }
    glEnd();
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(x2-r,y2-r);
    for(int i=0;i<=seg;i++){ float th = 0 + float(M_PI)/2*i/seg; glVertex2f(x2-r + r*cosf(th), y2-r + r*sinf(th)); }
    glEnd();
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(x2-r,y1+r);
    for(int i=0;i<=seg;i++){ float th = -float(M_PI)/2 + float(M_PI)/2*i/seg; glVertex2f(x2-r + r*cosf(th), y1+r +
r*sinf(th)); }
    glEnd();
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(x1+r,y1+r);
    for(int i=0;i<=seg;i++){ float th = float(M_PI) + float(M_PI)/2*i/seg; glVertex2f(x1+r + r*cosf(th), y1+r +
r*sinf(th)); }
    glEnd();
}
void initStars2(){
    for(int i=0;i<200;i++){
        starX2[i] = (rand() % 300 - 150) / 100.0f;
        starY2[i] = (rand() % 200 - 100) / 100.0f;
        starAlpha2[i] = 0.5f + (rand() % 50) / 100.0f;
    }
}
void drawStars2(){
    glPointSize(2);
    glBegin(GL_POINTS);
    for(int i=0; i<200; i++) {
    float a = 0.6f + 0.4f * sinf(sunTick2 * 0.03f + starAlpha2[i] * 6.28318f);
    glColor4f(1, 1, 1, a);
    glVertex2f(starX2[i], starY2[i]);
```

```
}
   glEnd();
}
void drawSun2(float x, float y, float r) {

   glEnable(GL_BLEND);
   glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
   glColor4f(1.0f, 0.85f, 0.5f, 0.9f);
   drawCircle2(x, y, r, 100);

   glColor4f(1.0f, 0.95f, 0.7f, 0.7f * (0.8f + 0.2f * sin(timeCounter)));
   for (int i = 0; i < 12; i++) {
      float theta = 2.0f * M_PI * float(i) / 12.0f;
      glBegin(GL_TRIANGLES);
      glVertex2f(x, y);
      glVertex2f(x + r * 1.4f * cos(theta), y + r * 1.4f * sin(theta));
      glVertex2f(x + r * 1.4f * cos(theta + 0.2f), y + r * 1.4f * sin(theta + 0.2f));
      glEnd();
   }
   glDisable(GL_BLEND);
}
void drawRingedPlanet2(float x,float y,float r){

   glColor3f(0.9f,0.85f,0.7f);
   drawCircle2(x,y,r,60);
   glPushMatrix(); glTranslatef(x,y,0); glRotatef(25,1,0,0);
   glColor4f(0.95f,0.95f,0.85f,0.8f);
   glBegin(GL_LINE_LOOP);
   for(int i=0;i<100;i++){ float th=2.0f*M_PI*i/100.0f; glVertex2f(r*1.8f*cosf(th), r*0.5f*sinf(th)); }
   glEnd(); glPopMatrix();
}
void drawSkyline2(){

   glBegin(GL_QUADS);
   glColor4f(0.05f,0.07f,0.10f,0.55f);
   glVertex2f(-1.5f,-0.05f); glVertex2f(1.5f,-0.05f);
   glColor4f(0.03f,0.04f,0.06f,0.0f);
   glVertex2f(1.5f,0.35f); glVertex2f(-1.5f,0.35f);
   glEnd();

   glColor4f(0.06f,0.08f,0.11f,0.7f);
   float bx=-1.4f;
   for(int i=0;i<10;i++,bx+=0.3f){
      float w=0.18f+(rand()%30)/300.0f, h=0.12f+(rand()%60)/300.0f;
      glBegin(GL_QUADS);
      glVertex2f(bx,-0.05f); glVertex2f(bx+w,-0.05f); glVertex2f(bx+w,-0.05f+h); glVertex2f(bx,-0.05f+h);
      glEnd();
   }
}
void drawField2(){

   glBegin(GL_QUADS);
   glColor3f(0.10f,0.40f,0.12f);
   glVertex2f(-1.5f,-0.2f); glVertex2f(1.5f,-0.2f);
   glColor3f(0.05f,0.25f,0.07f);
   glVertex2f(1.5f,-0.55f); glVertex2f(-1.5f,-0.55f);
   glEnd();
}
void drawGreenRoad2(){
```

```
    glBegin(GL_QUADS);
    glColor3f(0.2f,0.5f,0.20f);
    glVertex2f(-1.5f,-0.55f); glVertex2f(1.5f,-0.55f);
    glColor3f(0.10f,0.30f,0.12f);
    glVertex2f(1.5f,-0.70f); glVertex2f(-1.5f,-0.70f);
    glEnd();
}
void drawGround2(){

    glBegin(GL_QUADS);
    glColor3f(0.75f,0.30f,0.12f); glVertex2f(-1.5f,-0.70f); glVertex2f(1.5f,-0.70f);
    glColor3f(0.35f,0.10f,0.06f); glVertex2f(1.5f,-1.00f); glVertex2f(-1.5f,-1.00f);
    glEnd();
}
void drawRobot2(float x,float y){

    glColor3f(0.72f,0.72f,0.82f);
    glBegin(GL_QUADS);
    glVertex2f(x-0.03f,y); glVertex2f(x+0.03f,y);
    glVertex2f(x+0.03f,y+0.08f); glVertex2f(x-0.03f,y+0.08f);
    glEnd();
    drawCircle2(x,y+0.12f,0.04f,20);

    glColor3f(0.0f,0.9f,1.0f); drawCircle(x-0.015f,y+0.12f,0.007f,10);
    glColor3f(0.0f,0.9f,1.0f); drawCircle(x+0.015f,y+0.12f,0.007f,10);
    glPushMatrix();
    glTranslatef(x-0.03f,y+0.06f,0);
    glRotatef(robotArmAngle2,0,0,1);
    glBegin(GL_LINES); glVertex2f(0,0); glVertex2f(-0.06f,0); glEnd();
    glPopMatrix();
    glPushMatrix();
    glTranslatef(x+0.03f,y+0.06f,0);
    glRotatef(-robotArmAngle2,0,0,1);
    glBegin(GL_LINES); glVertex2f(0,0); glVertex2f(0.06f,0); glEnd();
    glPopMatrix();
}
void drawSemiCircle2(float cx,float cy,float r,int seg=80){

    glBegin(GL_POLYGON);
    for(int i=0;i<=seg;i++){
        float th = M_PI * i / seg;
        glVertex2f(cx + r*cosf(th), cy + r*sinf(th));
    }
    glVertex2f(cx,cy);
    glEnd();
}
void drawFuturisticTreeTiny2(float x,float y,float scale=0.5f){

    glColor3f(0.45f,0.28f,0.18f);
    glBegin(GL_QUADS);
    glVertex2f(x-0.008f*scale,y); glVertex2f(x+0.008f*scale,y);
    glVertex2f(x+0.008f*scale,y+0.06f*scale); glVertex2f(x-0.008f*scale,y+0.06f*scale);
    glEnd();

    glEnable(GL_BLEND);
    glColor4f(0.10f,0.85f,0.35f,0.75f);
    drawEllipse(x, y+0.10f*scale, 0.035f*scale, 0.028f*scale, 36);
    drawEllipse(x-0.018f*scale, y+0.11f*scale, 0.028f*scale, 0.024f*scale, 36);
    drawEllipse(x+0.018f*scale, y+0.11f*scale, 0.028f*scale, 0.024f*scale, 36);
    glDisable(GL_BLEND);
```

```
}
void drawFarm2(float x,float y,float w,float h){

    float r = w/2.0f;
    glEnable(GL_BLEND);
    glColor4f(0.6f,0.9f,1.0f,0.18f);
    drawSemiCircle2(x+w/2, y, r, 100);

    {
        float domeBaseY = y;
        glColor4f(0.20f, 0.55f, 0.95f, 0.35f);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(x + w*0.50f, domeBaseY + 0.02f);
        int seg = 90;
        for(int i=0;i<=seg;i++){
            float th = (float)M_PI * i / seg;
            float px = x + w*0.50f + (w*0.42f)*cosf(th);
            float py = domeBaseY + 0.03f + (0.04f)*sinf(th);
            glVertex2f(px,py);
        }
        glEnd();
    }
    for(float tx=x+0.11f; tx<x+w-0.11f; tx+=0.06f) drawFuturisticTreeTiny2(tx, y+0.05f, 0.50f);
    drawRobot2(x+w*0.65f,y+0.06f);
    glDisable(GL_BLEND);
}
void drawEarth2(float x, float y, float r) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glBegin(GL_POLYGON);
    for (int i = 0; i < 100; i++) {
        float theta = 2.0f * M_PI * float(i) / 100.0f;
        float t = float(i) / 100.0f;
        float blue = t < 0.7f ? 0.7f : 0.2f;
        float green = t < 0.7f ? 0.3f : (t < 0.85f ? 0.6f : 0.4f);
        float red = t < 0.7f ? 0.1f : 0.3f;
        float intensity = 1.0f + 0.2f * cos(theta);
        glColor4f(red * intensity, green * intensity, blue * intensity, 0.9f);
        float vx = r * cos(theta);
        float vy = r * sin(theta);
        glVertex2f(x + vx, y + vy);
    }
    glEnd();

    glColor4f(0.3f, 0.6f, 0.2f, 0.8f);
    drawCircle(x + r * 0.3f, y, r * 0.25f, 20);
    drawCircle(x - r * 0.2f, y + r * 0.3f, r * 0.2f, 20);
    glColor4f(1.0f, 1.0f, 1.0f, 0.7f);
    drawCircle(x + r * 0.1f, y - r * 0.2f, r * 0.15f, 20);
    drawCircle(x - r * 0.3f, y + r * 0.1f, r * 0.1f, 20);
    glDisable(GL_BLEND);
}
void drawFarmTall2(float cx, float baseY, float w, float h) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

     float bodyHeight = h * 0.5f;
    float domeHeight = h * 0.3f;
```

```
    w *= 0.65f;
    glColor3f(0.25f, 0.25f, 0.3f);
    drawRoundedRect2(cx - w/2 - 0.07f, baseY - 0.05f, cx + w/2 + 0.07f, baseY, 0.03f);
    glBegin(GL_QUADS);
    glColor3f(0.10f, 0.12f, 0.16f);
    glVertex2f(cx - w / 2, baseY);
    glVertex2f(cx + w / 2, baseY);
    glColor3f(0.55f, 0.58f, 0.62f);
    glVertex2f(cx + w / 2, baseY + bodyHeight);
    glVertex2f(cx - w / 2, baseY + bodyHeight);
    glEnd();
    for (int i=0;i<4;i++){
        float gx = cx - w/2 + (i+1)*(w/5);
        glColor4f(0.0f, 1.0f, 0.9f, 0.22f);
        glBegin(GL_QUADS);
        glVertex2f(gx-0.003f, baseY);
        glVertex2f(gx+0.003f, baseY);
        glVertex2f(gx+0.003f, baseY + bodyHeight);
        glVertex2f(gx-0.003f, baseY + bodyHeight);
        glEnd();
    }
    glColor4f(0.0f, 0.8f, 1.0f, 0.15f);
    drawRoundedRect2(cx - w/2 - 0.01f, baseY, cx + w/2 + 0.01f, baseY + bodyHeight, 0.04f);
    int steps = 85;
    float radius = 0.45f * w;
    float startY = baseY + 0.02f;
    float endY = baseY + bodyHeight * 0.95f;
    float totalH = endY - startY;
    for (int i = 0; i < steps; i++) {
        float t = (float)i / steps;
        float ang = t * 2.8f * M_PI;
        float sx = cx + cosf(ang) * (radius * 1.1f);
        float sy = startY + totalH * t;
        glColor4f(0.3f, 0.9f, 1.0f, 0.25f);
        drawRoundedRect2(sx-0.018f, sy-0.004f, sx+0.018f, sy+0.002f, 0.0025f);
    }
float domeBase = baseY + bodyHeight - 0.02f;
int segs = 200;
glBegin(GL_TRIANGLE_FAN);
glColor4f(0.6f, 0.9f, 1.0f, 0.20f);
glVertex2f(cx, domeBase);
for (int i = 0; i <= segs; i++) {
    float th = M_PI * i / segs;
    float px = cx + (w * 0.95f) * cos(th);
    float py = domeBase + (w * 0.5f) * sin(th);
    glVertex2f(px, py);
}
glEnd();
glColor4f(0.8f, 1.0f, 1.0f, 0.25f);
drawEllipse(cx, domeBase + (w * 0.25f), w * 0.6f, 0.03f, 90);
    glColor4f(0.8f, 1.0f, 1.0f, 0.25f);
    drawEllipse(cx, domeBase + domeHeight*0.4f, w*0.6f, 0.04f, 90);
    float innerTop = domeBase + (w * 0.4f);
    for (float px = cx - w*0.4f; px <= cx + w*0.4f; px += 0.035f) {
        for (int layer = 0; layer < 5; layer++) {
            float py = domeBase + 0.05f + layer * 0.04f;
            if (py > innerTop) break;
            glColor4f(0.1f, 0.9f, 0.3f, 0.8f);
            drawEllipse(px, py, 0.015f, 0.022f, 24);
            glColor4f(0.2f, 1.0f, 0.4f, 0.5f);
```

```
            drawEllipse(px, py + 0.01f, 0.012f, 0.018f, 24);
        }
    }
    glColor4f(0.0f, 0.7f, 1.0f, 0.08f);
    drawEllipse(cx, domeBase + domeHeight * 0.4f, w*0.8f, domeHeight*0.7f, 60);
    glDisable(GL_BLEND);
}
void drawSolarPanel2(float x,float y,float w,float h){

    float follow = -15.0f + 12.0f * sinf(sunTick2*0.01f);
    glPushMatrix(); glTranslatef(x,y,0); glRotatef(-20 + follow*0.2f, 1,0,0);

    glColor3f(0.16f,0.16f,0.17f);
    drawRoundedRect2(-w/2-0.015f, -0.01f, w/2+0.015f, h+0.015f, 0.015f);
    glColor3f(0.07f,0.09f,0.11f);
    glBegin(GL_QUADS);
    glVertex2f(-w/2,0); glVertex2f(w/2,0); glVertex2f(w/2,h); glVertex2f(-w/2,h);
    glEnd();
    int cols=6, rows=4; float gap=0.006f;
    float cellW=(w - gap*(cols+1))/cols, cellH=(h - gap*(rows+1))/rows;
    for(int i=0;i<cols;i++){
        for(int j=0;j<rows;j++){
            float cx = -w/2 + gap + i*(cellW+gap);
            float cy = gap + j*(cellH+gap);
            float tint = 0.50f + 0.05f*((i+j)%2);
            glColor3f(0.02f, 0.20f + 0.6f*tint, 0.35f + 0.45f*tint);
            glBegin(GL_QUADS);
            glVertex2f(cx,cy); glVertex2f(cx+cellW,cy);
            glVertex2f(cx+cellW,cy+cellH); glVertex2f(cx,cy+cellH);
            glEnd();
        }
    }
    glPopMatrix();
    glColor3f(0.30f,0.30f,0.32f);
    glBegin(GL_QUADS);
    glVertex2f(x-0.01f,y-0.1f); glVertex2f(x+0.01f,y-0.1f);
    glVertex2f(x+0.01f,y); glVertex2f(x-0.01f,y);
    glEnd();
}
void drawTractor2(float x,float y,bool flip){

    glPushMatrix(); glTranslatef(x,y,0); if(flip) glScalef(-1,1,1);
    glColor3f(0.2f,0.6f,0.2f);
    glBegin(GL_QUADS);
    glVertex2f(-0.12f,-0.05f); glVertex2f(0.12f,-0.05f);
    glVertex2f(0.12f,0.05f); glVertex2f(-0.12f,0.05f);
    glEnd();

    glColor3f(0.9f,0.9f,0.05f);
    drawCircle(-0.08f,-0.05f,0.04f,20); drawCircle(0.08f,-0.05f,0.04f,20);
    glColor3f(0.3f,0.5f,0.95f);
    glBegin(GL_QUADS);
    glVertex2f(-0.08f,0.0f); glVertex2f(0.05f,0.0f);
    glVertex2f(0.05f,0.04f); glVertex2f(-0.08f,0.04f);
    glEnd();
    glPopMatrix();
}
void drawTruckRobot2(float rx,float ry){

    glColor3f(0.7f,0.75f,0.85f);
```

```
    glBegin(GL_QUADS);
    glVertex2f(rx-0.015f,ry); glVertex2f(rx+0.015f,ry);
    glVertex2f(rx+0.015f,ry+0.05f); glVertex2f(rx-0.015f,ry+0.05f);
    glEnd();

    glColor3f(0.6f,0.65f,0.72f);
    drawCircle(rx,ry+0.07f,0.02f,16);
    glColor3f(0.15f,0.15f,0.15f);
    glBegin(GL_QUADS);
    glVertex2f(rx-0.018f,ry+0.065f); glVertex2f(rx+0.018f,ry+0.065f);
    glVertex2f(rx+0.018f,ry+0.075f); glVertex2f(rx-0.018f,ry+0.075f);
    glEnd();
    glColor3f(0.0f,1.0f,0.9f);
    drawCircle(rx-0.007f,ry+0.075f,0.005f,8);
    drawCircle(rx+0.007f,ry+0.075f,0.005f,8);
    glPushMatrix();
    glTranslatef(rx-0.015f,ry+0.04f,0);
    glRotatef(truckRobotArmAngle2,0,0,1);
    glBegin(GL_LINES); glVertex2f(0,0); glVertex2f(-0.03f,0.0f); glEnd();
    glPopMatrix();
    glPushMatrix();
    glTranslatef(rx+0.015f,ry+0.04f,0);
    glRotatef(-truckRobotArmAngle2,0,0,1);
    glBegin(GL_LINES); glVertex2f(0,0); glVertex2f(0.03f,0.0f); glEnd();
    glPopMatrix();
}
void drawTruck2(float x,float y,bool flip){

    glPushMatrix(); glTranslatef(x,y,0); if(flip) glScalef(-1,1,1);
    glColor3f(0.18f,0.18f,0.22f);
    drawRoundedRect2(-0.28f,-0.06f,0.14f,0.06f,0.02f);

    glColor3f(0.85f,0.10f,0.10f);
    drawRoundedRect2(0.14f,-0.06f,0.28f,0.05f,0.02f);
    glColor3f(0.0f,0.85f,1.0f);
    drawRoundedRect2(0.17f,-0.01f,0.26f,0.04f,0.01f);
    glColor3f(0.05f,0.05f,0.05f);
    drawCircle(-0.22f,-0.07f,0.035f,20);
    drawCircle( 0.00f,-0.07f,0.035f,20);
    drawCircle( 0.22f,-0.07f,0.035f,20);
    glEnable(GL_BLEND);
    glColor4f(1.0f,1.0f,0.7f,0.9f); drawCircle(0.285f, -0.025f, 0.012f, 14);
    glColor4f(1.0f,0.1f,0.05f,0.9f); drawCircle(-0.285f,-0.02f, 0.010f, 14);
    glDisable(GL_BLEND);
    glColor3f(0.95f,0.85f,0.05f);
    for(float cx=-0.26f; cx<-0.05f; cx+=0.03f){
      glBegin(GL_QUADS);
      glVertex2f(cx,-0.02f); glVertex2f(cx+0.02f,-0.02f);
      glVertex2f(cx+0.02f,0.04f); glVertex2f(cx,0.04f);
      glEnd();
    }
    drawTruckRobot2(-0.20f,0.02f);
    drawTruckRobot2(-0.13f,0.02f);
    float dx = 0.21f, dy = 0.00f;
    glColor3f(0.7f,0.75f,0.85f);
    glBegin(GL_QUADS);
    glVertex2f(dx-0.015f,dy-0.02f); glVertex2f(dx+0.015f,dy-0.02f);
    glVertex2f(dx+0.015f,dy+0.02f); glVertex2f(dx-0.015f,dy+0.02f);
    glEnd();
    glPushMatrix();
```

```
    glTranslatef(dx,dy+0.04f,0);
    glRotatef(driverHeadAngle2,0,0,1);
    glColor3f(0.6f,0.65f,0.72f); drawCircle(0,0,0.015f,16);
    glColor3f(0.15f,0.15f,0.15f);
    glBegin(GL_QUADS);
    glVertex2f(-0.012f,-0.005f); glVertex2f(0.012f,-0.005f);
    glVertex2f(0.012f,0.005f); glVertex2f(-0.012f,0.005f);
    glEnd();
    glColor3f(0.0f,1.0f,0.9f);
    drawCircle(-0.006f,0.005f,0.004f,8);
    drawCircle( 0.006f,0.005f,0.004f,8);
    glPopMatrix();
    glPopMatrix();
}
void drawDrone2(float x,float y){

    glPushMatrix(); glTranslatef(x,y,0);
    glColor3f(0.85f,0.85f,0.9f);
    drawRoundedRect2(-0.03f,-0.01f,0.03f,0.01f,0.005f);

    glColor3f(0.1f, 0.1f, 0.1f);
    drawCircle(0.012f, 0.0f, 0.007f, 16);
    drawCircle(-0.012f, 0.0f, 0.007f, 16);
    float pulse = 0.6f + 0.4f * sinf(sunTick2 * 0.2f);
    glColor3f(0.0f, 1.0f * pulse, 0.9f * pulse);
    drawCircle(0.012f, 0.0f, 0.004f, 10);
    drawCircle(-0.012f, 0.0f, 0.004f, 10);
    glEnable(GL_BLEND);
    glColor4f(0.0f, 1.0f, 0.9f, 0.15f);
    glBegin(GL_TRIANGLES);
    glVertex2f(0.0f, 0.0f);
    glVertex2f(0.04f, -0.005f);
    glVertex2f(0.04f, 0.005f);
    glEnd();
    float r=0.015f;
    for(int i=0;i<4;i++){
        float ox=(i<2? -0.04f:0.04f);
        float oy=(i%2==0? 0.02f:-0.02f);
        glColor4f(1,1,1,0.25f);
        drawEllipse(ox,oy, r*(1.0f+0.6f*fabsf(sinf(sunTick2*0.25f))), r*0.35f, 24);
    }
    glDisable(GL_BLEND);
    glPopMatrix();
}
void drawWaterSpray2(float x, float y, int count){

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glPointSize(2.0f);
    glBegin(GL_POINTS);
    glColor4f(0.45f, 0.85f, 1.0f, 0.80f);
    for(int i=0;i<count;i++){
        float ang=((rand()%100)/100.0f-0.5f)*0.22f;
        float len=0.18f+((rand()%100)/100.0f)*0.20f;
        float drift = 0.02f*sinf(sunTick2*0.03f + i);
        glVertex2f(x + sinf(ang)*0.05f + drift, y - len);
    }
    glEnd();
    glDisable(GL_BLEND);
}
```

```
void drawRocket2(float x, float y) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);
    glColor4f(0.8f, 0.8f, 0.9f, 1.0f);
    glBegin(GL_POLYGON);
    glVertex2f(x - 0.15f, y - 0.03f);
    glVertex2f(x + 0.15f, y - 0.02f);
    glVertex2f(x + 0.15f, y + 0.02f);
    glVertex2f(x - 0.15f, y + 0.03f);
    glEnd();

    glColor4f(0.7f, 0.7f, 0.8f, 1.0f);
    glBegin(GL_TRIANGLES);
    glVertex2f(x - 0.15f, y + 0.03f);
    glVertex2f(x - 0.1f, y + 0.06f);
    glVertex2f(x - 0.05f, y + 0.03f);
    glVertex2f(x - 0.15f, y - 0.03f);
    glVertex2f(x - 0.1f, y - 0.06f);
    glVertex2f(x - 0.05f, y - 0.03f);
    glEnd();
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f * flicker);
    drawCircle(x + 0.1f, y, 0.015f, 20);
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(x - 0.15f, y);
    glVertex2f(x - 0.25f, y + 0.04f);
    glVertex2f(x - 0.25f, y - 0.04f);
    glEnd();
    glDisable(GL_BLEND);
}
void drawMetroPillar2(float x, float yBottom, float width, float height) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float topWidth = width;
    glBegin(GL_QUADS);
    glColor4f(0.7f, 0.7f, 0.8f, 0.95f);
    glVertex2f(x - width / 2, yBottom);
    glVertex2f(x + width / 2, yBottom);
    glColor4f(0.8f, 0.8f, 0.9f, 0.95f);
    glVertex2f(x + topWidth / 2, yBottom + height);
    glVertex2f(x - topWidth / 2, yBottom + height);
    glEnd();

    glColor4f(0.0f, 1.0f, 1.0f, 0.8f);
    glBegin(GL_QUADS);
    glVertex2f(x - width / 2, yBottom);
    glVertex2f(x - width / 2 + 0.005f, yBottom);
    glVertex2f(x - topWidth / 2 + 0.005f, yBottom + height);
    glVertex2f(x - topWidth / 2, yBottom + height);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(x + width / 2 - 0.005f, yBottom);
    glVertex2f(x + width / 2, yBottom);
    glVertex2f(x + topWidth / 2, yBottom + height);
    glVertex2f(x + topWidth / 2 - 0.005f, yBottom + height);
    glEnd();
    glDisable(GL_BLEND);
```

```
}
void drawVacuumTube2(float xStart, float xEnd, float y, float height) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glColor4f(0.75f, 0.85f, 1.0f, 0.25f);
    glBegin(GL_QUADS);
    glVertex2f(xStart, y - height / 2);
    glVertex2f(xEnd, y - height / 2);
    glVertex2f(xEnd, y + height / 2);
    glVertex2f(xStart, y + height / 2);
    glEnd();

    glColor4f(0.5f, 0.8f, 1.0f, 0.35f);
    drawCircle(xStart, y, height / 2, 40);
    drawCircle(xEnd, y, height / 2, 40);
    glColor4f(1.0f, 1.0f, 1.0f, 0.3f);
    glBegin(GL_LINES);
    glVertex2f(xStart, y + height * 0.3f);
    glVertex2f(xEnd, y + height * 0.3f);
    glEnd();
    glDisable(GL_BLEND);
}
void drawMetroRail2(float xStart, float xEnd, float y, float thickness) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glColor4f(0.25f, 0.25f, 0.3f, 0.95f);
    glBegin(GL_QUADS);
    glVertex2f(xStart, y - thickness / 2);
    glVertex2f(xEnd, y - thickness / 2);
    glVertex2f(xEnd, y + thickness / 2);
    glVertex2f(xStart, y + thickness / 2);
    glEnd();

    glColor4f(1.0f, 0.6f, 0.1f, 0.8f);
    glBegin(GL_QUADS);
    glVertex2f(xStart, y - thickness / 6);
    glVertex2f(xEnd, y - thickness / 6);
    glVertex2f(xEnd, y + thickness / 6);
    glVertex2f(xStart, y + thickness / 6);
    glEnd();
    glColor4f(1.0f, 0.7f, 0.3f, 0.9f);
    drawCircle(xStart, y, thickness / 2, 40);
    drawCircle(xEnd, y, thickness / 2, 40);
    glDisable(GL_BLEND);
}
void drawMetroTrain2(float x, float y, float width, float height) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);
    float segmentWidth = width / 4.0f;
    float segmentHeight = height * 1.2f;

    glColor4f(1.0f, 0.55f, 0.05f, 0.95f);
    glBegin(GL_QUADS);
    glVertex2f(x - width / 2, y - segmentHeight / 2);
    glVertex2f(x + width / 2, y - segmentHeight / 2);
    glVertex2f(x + width / 2, y + segmentHeight / 2);
```

```
glVertex2f(x - width / 2, y + segmentHeight / 2);
glEnd();
for (int i = 0; i < 4; i++) {
    float segX = x - width / 2 + segmentWidth * i + segmentWidth / 2;
    glColor4f(1.0f, 0.55f, 0.05f, 0.95f);
    glBegin(GL_POLYGON);
    glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 2);
    glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 2);
    glVertex2f(segX + segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
    glVertex2f(segX - segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
    glEnd();
    glColor4f(1.0f, 0.7f, 0.2f, 0.7f * flicker);
    glBegin(GL_QUADS);
    glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 4);
    glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 4);
    glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 4 + 0.01f);
    glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 4 + 0.01f);
    glEnd();
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
    int numWindows = (i == 0 || i == 3) ? 2 : 3;
    float startX = segX - segmentWidth / 2 + 0.03f;
    float stepX = (segmentWidth - 0.06f) / (numWindows - 1);
    for (int j = 0; j < numWindows; j++) {
        drawCircle(startX + j * stepX, y, 0.02f, 24);
    }
}
glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
glBegin(GL_POLYGON);
glVertex2f(x + width / 2, y - segmentHeight / 3);
glVertex2f(x + width / 2 + 0.1f, y - segmentHeight / 3);
glVertex2f(x + width / 2 + 0.1f, y + segmentHeight / 3);
glVertex2f(x + width / 2, y + segmentHeight / 3);
glEnd();
glPushMatrix();
glTranslatef(x + width / 2 + 0.1f, y, 0.0f);
drawCircle(0.0f, 0.0f, 0.08f, 40);
glPopMatrix();
glColor4f(0.0f, 0.9f, 1.0f, 0.8f);
glBegin(GL_QUADS);
glVertex2f(x + width / 2 + 0.05f, y - 0.035f);
glVertex2f(x + width / 2 + 0.12f, y - 0.035f);
glVertex2f(x + width / 2 + 0.12f, y + 0.035f);
glVertex2f(x + width / 2 + 0.05f, y + 0.035f);
glEnd();
glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
glBegin(GL_POLYGON);
glVertex2f(x - width / 2, y - segmentHeight / 3);
glVertex2f(x - width / 2 - 0.1f, y - segmentHeight / 3);
glVertex2f(x - width / 2 - 0.1f, y + segmentHeight / 3);
glVertex2f(x - width / 2, y + segmentHeight / 3);
glEnd();
glPushMatrix();
glTranslatef(x - width / 2 - 0.1f, y, 0.0f);
drawCircle(0.0f, 0.0f, 0.08f, 40);
glPopMatrix();
glColor4f(0.0f, 0.9f, 1.0f, 0.8f);
glBegin(GL_QUADS);
glVertex2f(x - width / 2 - 0.12f, y - 0.035f);
glVertex2f(x - width / 2 - 0.05f, y - 0.035f);
glVertex2f(x - width / 2 - 0.05f, y + 0.035f);
```

```
    glVertex2f(x - width / 2 - 0.12f, y + 0.035f);
    glEnd();
    glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
    drawCircle(x + width / 2 + 0.1f, y, 0.02f, 20);
    drawCircle(x - width / 2 - 0.1f, y, 0.02f, 20);
    glColor4f(1.0f, 0.7f, 0.2f, 0.6f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(x - width / 2 - 0.12f, y);
    glVertex2f(x - width / 2 - 0.18f, y + 0.03f);
    glVertex2f(x - width / 2 - 0.18f, y - 0.03f);
    glEnd();
    glDisable(GL_BLEND);
}
void drawMetro2(float trainX) {

    float yMid = (-0.05f + -0.60f) / 2.0f;
    float yBottom = yMid - 0.35f;
    float height = 0.98f;
    float railY = yBottom + height;
    float thickness = 0.06f;
    float tubeHeight = 0.12f;
    drawVacuumTube2(-1.5f, 1.5f, railY, tubeHeight);
    drawMetroRail2(-1.5f, 1.5f, railY, thickness);
    for (float x = -1.4f; x <= 1.4f; x += 0.4f) {
       drawMetroPillar2(x, yBottom, 0.05f, height);
    }
    drawMetroTrain2(trainX, railY, 0.8f, 0.08f);
}
static void drawGrainSquare2(float x, float y, float angleDeg, float scale, float tint){

    float s = 0.0105f * scale;
    float half = s * 0.5f;
    glPushMatrix();
    glTranslatef(x, y, 0);
    glRotatef(angleDeg, 0,0,1);
    glEnable(GL_BLEND);
    glColor4f(0,0,0,0.10f);
    glBegin(GL_QUADS);
    glVertex2f(+half*0.9f, -half*0.5f);
    glVertex2f(+half*1.1f, -half*0.5f);
    glVertex2f(+half*1.1f, +half*0.5f);
    glVertex2f(+half*0.9f, +half*0.5f);
    glEnd();

    float r = 0.98f, g = 0.85f + 0.10f*tint, b = 0.18f + 0.05f*tint;
    glColor3f(r,g,b);
    glBegin(GL_QUADS);
    glVertex2f(-half, -half); glVertex2f(+half, -half);
    glVertex2f(+half, +half); glVertex2f(-half, +half);
    glEnd();
    glColor4f(1,1,1,0.18f);
    glBegin(GL_QUADS);
    glVertex2f(-half*0.5f, +half*0.1f);
    glVertex2f(+half*0.2f, +half*0.1f);
    glVertex2f(+half*0.2f, +half*0.25f);
    glVertex2f(-half*0.5f, +half*0.25f);
    glEnd();
    glDisable(GL_BLEND);
    glPopMatrix();
}
```

```
void initGrains2(){
   for(int i=0;i<NUM_GRAINS2;i++){
      grainX2[i] = (rand()%3000 - 1500) / 1000.0f;
      float ry = -0.70f + (rand()%1500)/1000.0f * (0.70f-0.55f);
      if(ry > -0.55f) ry = -0.55f;
      if(ry < -0.70f) ry = -0.70f;
      grainY2[i] = ry + (rand()%100-50)/10000.0f;
      grainA2[i] = (rand()%18000)/100.0f;
      grainS2[i] = 0.55f + (rand()%60)/100.0f;
      grainTint2[i] = (rand()%100)/100.0f;
   }
}
void drawRoadGrains2(){

   for(int i=0;i<NUM_GRAINS2;i++){
      float gx = grainX2[i];
      float gy = grainY2[i];
      bool inside=false;
      for(int p=0;p<NUM_PATCHES2;p++){
         if(gx>=patch2[p][0] && gx<=patch2[p][2] &&
            gy>=patch2[p][1] && gy<=patch2[p][3]){ inside=true; break; }
      }
      if(!inside) continue;
      float a = grainA2[i] + 2.0f * sinf( (sunTick2*0.007f) + i*0.13f );
      float s = grainS2[i] * (0.98f + 0.02f * cosf( (sunTick2*0.01f) + i*0.29f ));
      drawGrainSquare2(gx, gy, a, s, grainTint2[i]);
   }
}
void drawEnergyTower2(float x, float y){

   glColor3f(0.15f, 0.15f, 0.22f);
   drawRoundedRect2(x-0.08f, y-0.4f, x+0.08f, y+0.4f, 0.05f);

   glEnable(GL_BLEND);
   glColor4f(0.2f, 0.8f, 1.0f, 0.6f);
   drawRoundedRect2(x-0.03f, y-0.35f, x+0.03f, y+0.35f, 0.02f);
   for(int i=0;i<4;i++){
      float ringY = y-0.25f + i*0.18f + 0.01f*sin(sunTick2*0.05f + i);
      glColor4f(0.0f, 1.0f, 0.9f, 0.7f);
      drawEllipse(x, ringY, 0.10f, 0.02f, 40);
   }
   glColor4f(0.2f, 1.0f, 0.9f, 0.8f);
   drawCircle(x, y+0.42f, 0.06f, 24);
   glDisable(GL_BLEND);
}
void drawWetnessOverlay2(){

   for(int p=0;p<NUM_PATCHES2;p++){
      if(wetness2[p] <= 0.001f) continue;
      float alpha = fminf(0.45f, wetness2[p]*0.35f);
      glEnable(GL_BLEND);
      glColor4f(0.05f, 0.10f, 0.20f, alpha);
      glBegin(GL_QUADS);
      glVertex2f(patch2[p][0], patch2[p][1]);
      glVertex2f(patch2[p][2], patch2[p][1]);
      glVertex2f(patch2[p][2], patch2[p][3]);
      glVertex2f(patch2[p][0], patch2[p][3]);
      glEnd();
      glDisable(GL_BLEND);
   }
```

```
}
void drawScene2() {
   drawStars2();
   drawSun2(1.1f,0.9f,0.14f);
   drawRingedPlanet2(-0.9f,0.9f,0.10f);
   drawBeautifulMoon2(-0.3f, 0.75f, 0.08f);
   drawStars2();
   drawSkyline2();
   drawRocket2(rocketX2, 0.7f);
   drawEarth2(0.0f, 0.8f, 0.12f);
   drawMetro2(commonTrainX);
   drawField2();
   drawFarm2(-1.0f,-0.2f,0.5f,0.35f);
   drawFarm2( 0.5f,-0.2f,0.5f,0.35f);
   drawFarmTall2(1.23f,-0.55f,0.42f,0.95f);
   drawSolarPanel2(-1.30f,-0.32f,0.11f,0.07f);
   drawSolarPanel2(-1.10f,-0.34f,0.10f,0.07f);
   drawSolarPanel2(-0.90f,-0.36f,0.09f,0.06f);
   drawSolarPanel2(-0.65f,-0.39f,0.12f,0.08f);
   drawSolarPanel2(-0.15f,-0.35f,0.10f,0.07f);
   drawSolarPanel2( 0.15f,-0.37f,0.11f,0.07f);
   drawSolarPanel2( 0.45f,-0.34f,0.09f,0.06f);
   drawSolarPanel2( 0.75f,-0.36f,0.10f,0.07f);
   drawEnergyTower2(-1.33f,-0.22f);
   drawGreenRoad2();
   drawRoadGrains2();
   drawTractor2(tractorX2[0],-0.63f,false);
   drawTractor2(tractorX2[1],-0.67f,true);
   drawTruck2(truckX2[0],-0.59f,truckFlip2[0]);
   drawTruck2(truckX2[1],-0.66f,truckFlip2[1]);
   if(enableSky12){
      float dx = -0.35f + 0.55f * sinf(droneT2*0.9f);
      float dy = -0.05f + 0.03f * sinf(droneT2*1.7f);
      drawDrone2(dx, dy);
   }
   if(enableSky22){
      float dx2 = 0.30f + 0.52f * sinf(droneT32*0.8f + 2.0f);
      float dy2 = -0.02f + 0.04f * cosf(droneT32*1.2f + 1.5f);
      drawDrone2(dx2, dy2);
   }
   for(int i=0;i<NUM_WDRONES2;i++){
      if(!enableWaterDrone2[i]) continue;
      float wx = droneX2[i] + 0.06f * sinf(droneT22*0.6f + dronePhase2[i]);
      float wy = droneY2[i] + 0.02f * cosf(droneT22*1.3f + dronePhase2[i]);
      drawDrone2(wx, wy);
      drawWaterSpray2(wx, wy - 0.03f, rainIntensity2);
      if(wx >= patch2[i][0] && wx <= patch2[i][2]) {
         wetness2[i] += 0.006f;
         if(wetness2[i] > 1.2f) wetness2[i] = 1.2f;
      }
   }
   drawWetnessOverlay2();
   drawGround2();
}
void updateScene2() {
   sunTick2++;
   timeCounter += 0.05f;
   commonTrainX += commonTrainSpeed;
   if (commonTrainX > 2.0f) {
      commonTrainX = -1.5f;
```

```
        currentScene = 3;
    }
    rocketX2 += rocketSpeed2;
    if (rocketX2 > 1.8f) rocketX2 = -1.8f;
    droneT2 += 0.020f;
    droneT22 += 0.016f;
    droneT32 += 0.018f;
    if(truckArmUp2) truckRobotArmAngle2+=1.0f; else truckRobotArmAngle2-=1.0f;
    if(truckRobotArmAngle2>25) truckArmUp2=false;
    if(truckRobotArmAngle2<-25) truckArmUp2=true;
    if(driverHeadLeft2) driverHeadAngle2+=0.5f; else driverHeadAngle2-=0.5f;
    if(driverHeadAngle2>10) driverHeadLeft2=false;
    if(driverHeadAngle2<-10) driverHeadLeft2=true;
    // Train movement is common, so skip here
    for(int i=0;i<2;i++){
        if(!tractorFlip2[i]) tractorX2[i]+=tractorSpeed2[i]; else tractorX2[i]-=tractorSpeed2[i];
        if(tractorX2[i]>1.6f) tractorX2[i]=-1.6f;
        if(tractorX2[i]<-1.6f) tractorX2[i]= 1.6f;
    }
    for(int i=0;i<2;i++){
        if(!truckFlip2[i]) truckX2[i]+=truckSpeed2[i]; else truckX2[i]-=truckSpeed2[i];
        if(truckX2[i]>1.8f) truckX2[i]=-1.8f;
        if(truckX2[i]<-1.8f) truckX2[i]= 1.8f;
    }
    if(armUp2) robotArmAngle2+=1.0f; else robotArmAngle2-=1.0f;
    if(robotArmAngle2>30) armUp2=false; if(robotArmAngle2<-30) armUp2=true;
    for(int p=0;p<NUM_PATCHES2;p++){
        wetness2[p] -= 0.0025f;
        if(wetness2[p] < 0.0f) wetness2[p] = 0.0f;
    }
    for(int i=0;i<NUM_WDRONES2;i++) dronePhase2[i] += 0.0015f;
}
// Scene 3 functions (kept as is, adjust trainX to commonTrainX)
void drawCircle3(float cx, float cy, float r, int num_segments = 100) {
    drawCircle(cx, cy, r, num_segments); // common
}
void drawRocket3(float x, float y) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);

    // Rocket body
    glColor4f(0.8f, 0.8f, 0.9f, 1.0f); // Silver-gray
    glBegin(GL_POLYGON);
    glVertex2f(x - 0.15f, y - 0.03f); // Rear
    glVertex2f(x + 0.15f, y - 0.02f); // Front tip
    glVertex2f(x + 0.15f, y + 0.02f);
    glVertex2f(x - 0.15f, y + 0.03f);
    glEnd();
    // Fins
    glColor4f(0.7f, 0.7f, 0.8f, 1.0f);
    glBegin(GL_TRIANGLES);
    // Top fin
    glVertex2f(x - 0.15f, y + 0.03f);
    glVertex2f(x - 0.1f, y + 0.06f);
    glVertex2f(x - 0.05f, y + 0.03f);
    // Bottom fin
    glVertex2f(x - 0.15f, y - 0.03f);
    glVertex2f(x - 0.1f, y - 0.06f);
```

```
    glVertex2f(x - 0.05f, y - 0.03f);
    glEnd();
    // Glowing cockpit window
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f * flicker);
    drawCircle(x + 0.1f, y, 0.015f, 20);
    // Neon accents along body
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_QUADS);
    glVertex2f(x - 0.1f, y - 0.01f);
    glVertex2f(x + 0.1f, y - 0.01f);
    glVertex2f(x + 0.1f, y - 0.005f);
    glVertex2f(x - 0.1f, y - 0.005f);
    glEnd();
    // Fiery trail
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(x - 0.15f, y);
    glVertex2f(x - 0.25f, y + 0.04f);
    glVertex2f(x - 0.25f, y - 0.04f);
    glEnd();
    // Secondary trail glow
    glColor4f(1.0f, 0.7f, 0.2f, 0.5f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(x - 0.15f, y);
    glVertex2f(x - 0.3f, y + 0.06f);
    glVertex2f(x - 0.3f, y - 0.06f);
    glEnd();
    glDisable(GL_BLEND);
}
void initStars3() {
    for (int i = 0; i < 200; i++) {
        starX3[i] = (rand() % 300 - 150) / 100.0f;
        starY3[i] = (rand() % 200 - 100) / 100.0f;
        starAlpha3[i] = 0.5f + (rand() % 50) / 100.0f;
    }
}
void drawStars3() {
    glPointSize(4);
    glBegin(GL_POINTS);
    for (int i = 0; i < 200; i++) {
        float alpha = starAlpha3[i] + 0.3f * sin(timeCounter + i);
        glColor4f(1.0f, 1.0f, 1.0f, alpha);
        glVertex2f(starX3[i], starY3[i]);
    }
    glEnd();
}
void drawWindowsStrips3(float x, float y, float w, float h) {

    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    for (float j = y + 0.02f; j < y + h - 0.01f; j += 0.06f) {
        glBegin(GL_QUADS);
        glColor4f(0.0f, 0.9f, 1.0f, 0.85f);
        glVertex2f(x + 0.02f, j);
        glVertex2f(x + w - 0.02f, j);
        glVertex2f(x + w - 0.02f, j + 0.02f);
        glVertex2f(x + 0.02f, j + 0.02f);
        glEnd();
    }
}
void drawWindowsGrid3(float x, float y, float w, float h) {
```

```
    float cellW = 0.04f;
    float cellH = 0.025f;
    for (float i = x + 0.02f; i < x + w - 0.01f; i += cellW + 0.01f) {
        for (float j = y + 0.02f; j < y + h - 0.01f; j += cellH + 0.01f) {
            glBegin(GL_QUADS);
            glColor4f(0.2f, 1.0f, 0.9f, 0.9f);
            glVertex2f(i, j);
            glVertex2f(i + cellW, j);
            glVertex2f(i + cellW, j + cellH);
            glVertex2f(i, j + cellH);
            glEnd();
        }
    }
}
void drawWindowsDots3(float x, float y, float w, float h) {

    for (float i = x + 0.03f; i < x + w - 0.02f; i += 0.06f) {
        for (float j = y + 0.03f; j < y + h - 0.02f; j += 0.06f) {
            glBegin(GL_POLYGON);
            glColor4f(0.8f, 0.9f, 1.0f, 0.9f);
            int segs = 12;
            float r = 0.008f;
            for (int s = 0; s < segs; s++) {
                float th = 2.0f * M_PI * s / segs;
                glVertex2f(i + r * cos(th), j + r * sin(th));
            }
            glEnd();
        }
    }
}
void drawWindowsHex3(float x, float y, float w, float h) {

    float band = 0.06f;
    for (float i = x + 0.02f; i < x + w - 0.01f; i += band) {
        glBegin(GL_QUADS);
        glColor4f(0.0f, 0.8f, 0.9f, 0.9f);
        glVertex2f(i, y + 0.02f);
        glVertex2f(i + band*0.6f, y + 0.02f);
        glVertex2f(i + band*0.6f, y + h - 0.02f);
        glVertex2f(i, y + h - 0.02f);
        glEnd();
    }
}
void drawFrontRect3(float x, float y, float w, float h, float r, float g, float b, int windowStyle) {

    glColor3f(r, g, b);
    glBegin(GL_QUADS);
    glVertex2f(x, y);
    glVertex2f(x + w, y);
    glVertex2f(x + w, y + h);
    glVertex2f(x, y + h);
    glEnd();
    if (windowsOn3) {
        switch (windowStyle) {
            case 0: drawWindowsStrips3(x, y, w, h); break;
            case 1: drawWindowsGrid3(x, y, w, h); break;
            case 2: drawWindowsDots3(x, y, w, h); break;
            case 3: drawWindowsHex3(x, y, w, h); break;
            default: drawWindowsStrips3(x, y, w, h);
```

```
        }
    }
}
void drawFrontTrapezoid3(float x, float y, float w, float h, float r, float g, float b, int windowStyle) {

    glColor3f(r,g,b);
    glBegin(GL_QUADS);
    glVertex2f(x, y);
    glVertex2f(x + w, y);
    glVertex2f(x + w*0.75f, y + h);
    glVertex2f(x + w*0.25f, y + h);
    glEnd();
    if (windowsOn3) drawWindowsStrips3(x + w*0.25f, y, w*0.5f, h);
}
void drawFrontDome3(float x, float y, float w, float h, float r, float g, float b, int windowStyle) {

    glColor3f(r,g,b);
    glBegin(GL_QUADS);
    glVertex2f(x, y);
    glVertex2f(x + w, y);
    glVertex2f(x + w, y + h*0.7f);
    glVertex2f(x, y + h*0.7f);
    glEnd();
    glColor3f(r*1.1f, g*1.1f, b*1.1f);
    drawCircle3(x + w/2, y + h*0.7f, w*0.5f);
    if (windowsOn3) drawWindowsHex3(x, y, w, h*0.7f);
}
void drawFrontSpire3(float x, float y, float w, float h, float r, float g, float b, int windowStyle) {

    glColor3f(r,g,b);
    glBegin(GL_TRIANGLES);
    glVertex2f(x + w/2, y + h);
    glVertex2f(x, y);
    glVertex2f(x + w, y);
    glEnd();
    if (windowsOn3) drawWindowsStrips3(x, y, w, h*0.6f);
}
void drawCity3() {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    drawFrontRect3(-1.4f, -0.05f, 0.18f, 0.6f, 0.12f, 0.26f, 0.6f, 0);
    // ... (full code for all buildings)
    drawFrontTrapezoid3(-1.16f, -0.05f, 0.22f, 0.65f, 0.18f, 0.35f, 0.8f, 1);
    drawFrontDome3(-0.92f, -0.05f, 0.20f, 0.55f, 0.25f, 0.45f, 0.9f, 3);
    drawFrontSpire3(-0.68f, -0.05f, 0.18f, 0.7f, 0.2f, 0.5f, 0.95f, 0);
    drawFrontRect3(-0.46f, -0.05f, 0.2f, 0.58f, 0.22f, 0.5f, 0.9f, 2);
    drawFrontTrapezoid3(-0.22f, -0.05f, 0.18f, 0.62f, 0.28f, 0.55f, 1.0f, 0);
    drawFrontDome3(0.02f, -0.05f, 0.24f, 0.5f, 0.3f, 0.6f, 1.0f, 1);
    drawFrontSpire3(0.28f, -0.05f, 0.18f, 0.66f, 0.28f, 0.58f, 0.98f, 2);
    drawFrontRect3(0.52f, -0.05f, 0.2f, 0.56f, 0.35f, 0.65f, 1.0f, 3);
    drawFrontTrapezoid3(0.76f, -0.05f, 0.18f, 0.6f, 0.38f, 0.68f, 1.0f, 0);
    drawFrontSpire3(1.0f, -0.05f, 0.22f, 0.58f, 0.4f, 0.72f, 1.0f, 1);
    drawFrontRect3(1.26f, -0.05f, 0.18f, 0.54f, 0.45f, 0.75f, 1.0f, 2);
    glDisable(GL_BLEND);
}
void drawVehicle3(const Vehicle3& v) {

    glPushMatrix();
    glTranslatef(v.x, v.y, 0.0f);
```

```
glScalef(v.scale, v.scale, 1.0f);
if(v.flip) glScalef(-1.0f, 1.0f, 1.0f);
float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);
// ... (full code for all types)
glColor3f(v.r, v.g, v.b);
if (!v.isBus) {
   if (v.type == 0) { // Cybertruck-inspired
      glBegin(GL_POLYGON);
      glVertex2f(-0.14f, -0.05f);
      glVertex2f(0.14f, -0.05f);
      glVertex2f(0.11f, -0.02f);
      glVertex2f(0.09f, 0.07f);
      glVertex2f(-0.09f, 0.07f);
      glVertex2f(-0.11f, -0.02f);
      glEnd();
      // Neon underlights
      glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker); // Orange underlights
      glBegin(GL_QUADS);
      glVertex2f(-0.12f, -0.05f);
      glVertex2f(0.12f, -0.05f);
      glVertex2f(0.12f, -0.06f);
      glVertex2f(-0.12f, -0.06f);
      glEnd();
      // Windows
      glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
      glBegin(GL_QUADS);
      glVertex2f(-0.07f, 0.03f);
      glVertex2f(0.07f, 0.03f);
      glVertex2f(0.06f, 0.06f);
      glVertex2f(-0.06f, 0.06f);
      glEnd();
      // Headlights
      glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
      drawCircle(0.13f, -0.015f, 0.015f, 20);
      drawCircle(0.13f, 0.015f, 0.015f, 20);
      // Thrusters
      glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
      drawCircle(-0.12f, -0.03f, 0.02f, 20);
      // Jet flames
      float jetPos = v.flip ? 0.14f : -0.14f;
      glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
      glBegin(GL_TRIANGLES);
      glVertex2f(jetPos, 0.0f);
      glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
      glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
      glEnd();
   } else if (v.type == 1) { // Ferrari-inspired
      glBegin(GL_POLYGON);
      glVertex2f(-0.12f, -0.04f);
      glVertex2f(0.15f, -0.04f);
      glVertex2f(0.13f, -0.01f);
      glVertex2f(0.09f, 0.04f);
      glVertex2f(-0.09f, 0.04f);
      glVertex2f(-0.11f, -0.01f);
      glEnd();
      // Neon underlights
      glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
      glBegin(GL_QUADS);
      glVertex2f(-0.11f, -0.04f);
      glVertex2f(0.14f, -0.04f);
```

```
    glVertex2f(0.14f, -0.05f);
    glVertex2f(-0.11f, -0.05f);
    glEnd();
    // Curved window
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
    drawCircle(0.0f, 0.02f, 0.025f, 20);
    // Headlights
    glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
    drawCircle(0.14f, -0.015f, 0.015f, 20);
    drawCircle(0.14f, 0.015f, 0.015f, 20);
    // Thrusters
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
    drawCircle(-0.11f, -0.03f, 0.02f, 20);
    // Jet flames
    float jetPos = v.flip ? 0.13f : -0.13f;
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(jetPos, 0.0f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
    glEnd();
} else if (v.type == 2) { // Lamborghini-inspired
    glBegin(GL_POLYGON);
    glVertex2f(-0.13f, -0.045f);
    glVertex2f(0.13f, -0.045f);
    glVertex2f(0.11f, -0.015f);
    glVertex2f(0.07f, 0.045f);
    glVertex2f(-0.07f, 0.045f);
    glVertex2f(-0.11f, -0.015f);
    glEnd();
    // Neon underlights
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_QUADS);
    glVertex2f(-0.12f, -0.045f);
    glVertex2f(0.12f, -0.045f);
    glVertex2f(0.12f, -0.055f);
    glVertex2f(-0.12f, -0.055f);
    glEnd();
    // Angular windows
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(-0.06f, 0.02f);
    glVertex2f(0.06f, 0.02f);
    glVertex2f(0.05f, 0.04f);
    glVertex2f(-0.05f, 0.04f);
    glEnd();
    // Headlights
    glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
    drawCircle(0.12f, -0.015f, 0.015f, 20);
    drawCircle(0.12f, 0.015f, 0.015f, 20);
    // Thrusters
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
    drawCircle(-0.12f, -0.03f, 0.02f, 20);
    // Jet flames
    float jetPos = v.flip ? 0.13f : -0.13f;
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(jetPos, 0.0f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
```

```
            glEnd();
        } else { // Generic Sports Car
            glBegin(GL_POLYGON);
            glVertex2f(-0.11f, -0.04f);
            glVertex2f(0.12f, -0.04f);
            glVertex2f(0.11f, -0.01f);
            glVertex2f(0.09f, 0.04f);
            glVertex2f(-0.09f, 0.04f);
            glVertex2f(-0.11f, -0.01f);
            glEnd();
            // Neon underlights
            glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
            glBegin(GL_QUADS);
            glVertex2f(-0.1f, -0.04f);
            glVertex2f(0.11f, -0.04f);
            glVertex2f(0.11f, -0.05f);
            glVertex2f(-0.1f, -0.05f);
            glEnd();
            // Window
            glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
            drawCircle(0.0f, 0.02f, 0.025f, 20);
            // Headlight
            glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
            drawCircle(0.11f, 0.0f, 0.015f, 20);
            // Thrusters
            glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
            drawCircle(-0.1f, -0.03f, 0.02f, 20);
            // Jet flames
            float jetPos = v.flip ? 0.12f : -0.12f;
            glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
            glBegin(GL_TRIANGLES);
            glVertex2f(jetPos, 0.0f);
            glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
            glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
            glEnd();
        }
    } else {
        // Bus
        glBegin(GL_POLYGON);
        glVertex2f(-0.18f, -0.05f);
        glVertex2f(0.18f, -0.05f);
        glVertex2f(0.18f, 0.0f);
        glVertex2f(0.18f, 0.05f);
        glVertex2f(-0.18f, 0.05f);
        glVertex2f(-0.18f, 0.0f);
        glEnd();
        // Neon underlights
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        glBegin(GL_QUADS);
        glVertex2f(-0.17f, -0.05f);
        glVertex2f(0.17f, -0.05f);
        glVertex2f(0.17f, -0.06f);
        glVertex2f(-0.17f, -0.06f);
        glEnd();
        // Dome
        glColor4f(0.0f, 0.4f, 0.8f, 0.9f);
        drawCircle(0.0f, 0.05f, 0.07f, 30);
        // Windows
        glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
        for(float wx = -0.15f; wx < 0.15f; wx += 0.06f){
```

```
            drawCircle(wx, 0.0f, 0.025f, 20);
        }
        // Front light
        glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
        drawCircle(0.17f, 0.0f, 0.02f, 20);
        // Jets
        float jetPos = v.flip ? 0.18f : -0.18f;
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        glBegin(GL_TRIANGLES);
        glVertex2f(jetPos, 0.0f);
        glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
        glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
        glEnd();
    }
    glPopMatrix();
}
void drawRoadAndFootpath3() {

    glColor3f(0.15f, 0.15f, 0.15f);
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.05f);
    glVertex2f(1.5f, -0.05f);
    glVertex2f(1.5f, -0.60f);
    glVertex2f(-1.5f, -0.60f);
    glEnd();

    // Dividers
    glColor3f(1.0f, 1.0f, 1.0f);
    float dividerY[3] = {-0.20f, -0.35f, -0.50f};
    for(int d=0; d<3; d++){
        for(float x=-1.4f; x<=1.4f; x+=0.3f) {
            glBegin(GL_QUADS);
            glVertex2f(x, dividerY[d]+0.01f);
            glVertex2f(x+0.1f, dividerY[d]+0.01f);
            glVertex2f(x+0.1f, dividerY[d]-0.01f);
            glVertex2f(x, dividerY[d]-0.01f);
            glEnd();
        }
    }
    // Footpaths
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    // Top footpath
    glColor4f(0.3f, 0.3f, 0.3f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.02f);
    glVertex2f(1.5f, -0.02f);
    glVertex2f(1.5f, -0.05f);
    glVertex2f(-1.5f, -0.05f);
    glEnd();
    // Bottom footpath
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.60f);
    glVertex2f(1.5f, -0.60f);
    glVertex2f(1.5f, -0.63f);
    glVertex2f(-1.5f, -0.63f);
    glEnd();
    // Glowing markers
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * (0.8f + 0.2f * sin(timeCounter))); // Orange markers
    for (float x = -1.4f; x <= 1.4f; x += 0.2f) {
```

```
        drawCircle(x, -0.035f, 0.01f, 20);
        drawCircle(x, -0.615f, 0.01f, 20);
    }
    // Neon lamplights
    glColor4f(1.0f, 1.0f, 1.0f, 0.8f); // White stands
    for (float x = -1.3f; x <= 1.3f; x += 0.4f) {
        glBegin(GL_QUADS);
        glVertex2f(x - 0.01f, -0.02f);
        glVertex2f(x + 0.01f, -0.02f);
        glVertex2f(x + 0.01f, 0.02f);
        glVertex2f(x - 0.01f, 0.02f);
        glEnd();
        glBegin(GL_QUADS);
        glVertex2f(x - 0.01f, -0.63f);
        glVertex2f(x + 0.01f, -0.63f);
        glVertex2f(x + 0.01f, -0.59f); // Flipped upward
        glVertex2f(x - 0.01f, -0.59f);
        glEnd();
    }
    glColor4f(1.0f, 0.5f, 0.0f, 0.8f * (0.8f + 0.2f * sin(timeCounter))); // Orange lights
    for (float x = -1.3f; x <= 1.3f; x += 0.4f) {
        drawCircle(x, 0.02f, 0.015f, 20); // Top footpath lamp
        drawCircle(x, -0.59f, 0.015f, 20); // Bottom footpath lamp, flipped upward
    }
    glDisable(GL_BLEND);
}
void drawGround3() {

    glBegin(GL_QUADS);
    glColor3f(0.8f, 0.3f, 0.1f);
    glVertex2f(-1.5f, -0.63f);
    glVertex2f(1.5f, -0.63f);
    glColor3f(0.4f, 0.1f, 0.05f);
    glVertex2f(1.5f, -1.0f);
    glVertex2f(-1.5f, -1.0f);
    glEnd();

    glColor3f(0.6f, 0.15f, 0.05f);
    glBegin(GL_POLYGON);
    glVertex2f(-1.5f, -0.63f);
    glVertex2f(-1.2f, -0.58f);
    glVertex2f(-0.9f, -0.63f);
    glVertex2f(-0.6f, -0.61f);
    glVertex2f(-0.3f, -0.63f);
    glVertex2f(0.0f, -0.60f);
    glVertex2f(0.3f, -0.63f);
    glVertex2f(0.7f, -0.59f);
    glVertex2f(1.2f, -0.63f);
    glVertex2f(1.5f, -0.62f);
    glEnd();
    glColor3f(0.3f, 0.1f, 0.05f);
    drawCircle(-0.6f, -0.8f, 0.08f, 40);
    drawCircle(0.4f, -0.75f, 0.06f, 40);
    drawCircle(-0.2f, -0.9f, 0.1f, 40);
}
void drawPlanet3(float x, float y, float r, int type) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    if (type == 0) {
```

```
        // Sun: Bright red-orange with flares
    glColor4f(1.0f, 0.85f, 0.5f, 0.9f);
    drawCircle(x, y, r, 100);
    // Flares
    glColor4f(1.0f, 0.95f, 0.7f, 0.7f * (0.8f + 0.2f * sin(timeCounter)));
    for (int i = 0; i < 12; i++) {
        float theta = 2.0f * M_PI * float(i) / 12.0f;
        glBegin(GL_TRIANGLES);
        glVertex2f(x, y);
        glVertex2f(x + r * 1.3f * cos(theta), y + r * 1.3f * sin(theta));
        glVertex2f(x + r * 1.3f * cos(theta + 0.2f), y + r * 1.3f * sin(theta + 0.2f));
        glEnd();
    }
} else if (type == 1) { // Moon: Bright white-grey with craters
    glBegin(GL_POLYGON);
    for (int i = 0; i < 100; i++) {
        float theta = 2.0f * M_PI * float(i) / 100.0f;
        float intensity = 1.0f + 0.2f * cos(theta);
        glColor4f(1.0f * intensity, 1.0f * intensity, 0.95f * intensity, 0.9f);
        float vx = r * cos(theta);
        float vy = r * sin(theta);
        glVertex2f(x + vx, y + vy);
    }
    glEnd();
    // Craters
    glColor4f(0.85f, 0.85f, 0.85f, 0.8f);
    drawCircle(x - r * 0.4f, y + r * 0.3f, r * 0.2f, 20);
    drawCircle(x + r * 0.3f, y - r * 0.2f, r * 0.15f, 20);
    drawCircle(x - r * 0.2f, y - r * 0.1f, r * 0.1f, 20);
} else if (type == 2) { // Earth: Realistic blue-green with continents and clouds
    glBegin(GL_POLYGON);
    for (int i = 0; i < 100; i++) {
        float theta = 2.0f * M_PI * float(i) / 100.0f;
        float t = float(i) / 100.0f;
        float blue = t < 0.7f ? 0.7f : 0.2f; // More blue for oceans
        float green = t < 0.7f ? 0.3f : (t < 0.85f ? 0.6f : 0.4f); // Green-brown continents
        float red = t < 0.7f ? 0.1f : 0.3f; // Subtle red for land
        float intensity = 1.0f + 0.2f * cos(theta);
        glColor4f(red * intensity, green * intensity, blue * intensity, 0.9f);
        float vx = r * cos(theta);
        float vy = r * sin(theta);
        glVertex2f(x + vx, y + vy);
    }
    glEnd();
    // Continents
    glColor4f(0.3f, 0.6f, 0.2f, 0.8f); // Green-brown continents
    drawCircle(x + r * 0.3f, y, r * 0.25f, 20);
    drawCircle(x - r * 0.2f, y + r * 0.3f, r * 0.2f, 20);
    // Clouds
    glColor4f(1.0f, 1.0f, 1.0f, 0.7f);
    drawCircle(x + r * 0.1f, y - r * 0.2f, r * 0.15f, 20);
    drawCircle(x - r * 0.3f, y + r * 0.1f, r * 0.1f, 20);
} else if (type == 3) { // Saturn: Bright pale yellow with detailed rings
    glBegin(GL_POLYGON);
    for (int i = 0; i < 100; i++) {
        float theta = 2.0f * M_PI * float(i) / 100.0f;
        float intensity = 1.0f + 0.3f * cos(theta);
        glColor4f(1.0f * intensity, 0.95f * intensity, 0.8f * intensity, 0.9f);
        float vx = r * cos(theta);
        float vy = r * sin(theta);
```

```
            glVertex2f(x + vx, y + vy);
        }
        glEnd();
        // Rings
        glColor4f(0.95f, 0.95f, 0.85f, 0.8f);
        glBegin(GL_LINE_LOOP);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float rx = r * 1.6f * cos(theta);
            float ry = r * 0.3f * sin(theta);
            glVertex2f(x + rx, y + ry);
        }
        glEnd();
        glBegin(GL_LINE_LOOP);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float rx = r * 1.4f * cos(theta);
            float ry = r * 0.2f * sin(theta);
            glVertex2f(x + rx, y + ry);
        }
        glEnd();
        glBegin(GL_LINE_LOOP);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float rx = r * 1.2f * cos(theta);
            float ry = r * 0.1f * sin(theta);
            glVertex2f(x + rx, y + ry);
        }
        glEnd();
    }
    glDisable(GL_BLEND);
        // ... (full code for all types)

}
void drawPlanets3() {
    drawPlanet3(-0.6f, 0.85f, 0.15f, 0);
    drawPlanet3(-0.3f, 0.75f, 0.08f, 1);
    drawPlanet3(0.0f, 0.8f, 0.12f, 2);
    drawPlanet3(0.3f, 0.85f, 0.1f, 3);
}
void drawMetroPillar3(float x, float yBottom, float width, float height) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float topWidth = width;
    glBegin(GL_QUADS);
    glColor4f(0.7f, 0.7f, 0.8f, 0.95f);
    glVertex2f(x - width / 2, yBottom);
    glVertex2f(x + width / 2, yBottom);
    glColor4f(0.8f, 0.8f, 0.9f, 0.95f);
    glVertex2f(x + topWidth / 2, yBottom + height);
    glVertex2f(x - topWidth / 2, yBottom + height);
    glEnd();

    glColor4f(0.5f, 0.8f, 1.0f, 0.8f);
    glLineWidth(1.5f);
    glBegin(GL_LINES);
    for (float offset = -width / 2 + 0.01f; offset < width / 2; offset += 0.02f) {
        glVertex2f(x + offset, yBottom);
        glVertex2f(x + offset, yBottom + height);
```

```
    }
    glEnd();
    glColor4f(0.0f, 1.0f, 1.0f, 0.8f); // Neon cyan accents
    glBegin(GL_QUADS);
    glVertex2f(x - width / 2, yBottom);
    glVertex2f(x - width / 2 + 0.005f, yBottom);
    glVertex2f(x - topWidth / 2 + 0.005f, yBottom + height);
    glVertex2f(x - topWidth / 2, yBottom + height);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(x + width / 2 - 0.005f, yBottom);
    glVertex2f(x + width / 2, yBottom);
    glVertex2f(x + topWidth / 2, yBottom + height);
    glVertex2f(x + topWidth / 2 - 0.005f, yBottom + height);
    glEnd();
    glColor4f(0.6f, 0.6f, 0.7f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(x - width / 2, yBottom + height * 0.3f);
    glVertex2f(x - width / 2 + 0.01f, yBottom + height * 0.3f);
    glVertex2f(x + topWidth / 2 - 0.01f, yBottom + height * 0.7f);
    glVertex2f(x + topWidth / 2, yBottom + height * 0.7f);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(x + width / 2 - 0.01f, yBottom + height * 0.3f);
    glVertex2f(x + width / 2, yBottom + height * 0.3f);
    glVertex2f(x - topWidth / 2, yBottom + height * 0.7f);
    glVertex2f(x - topWidth / 2 + 0.01f, yBottom + height * 0.7f);
    glEnd();
    glColor4f(0.5f, 0.5f, 0.6f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(x - topWidth / 2 - 0.05f, yBottom + height - 0.03f);
    glVertex2f(x + topWidth / 2 + 0.05f, yBottom + height - 0.03f);
    glVertex2f(x + topWidth / 2 + 0.05f, yBottom + height + 0.03f);
    glVertex2f(x - topWidth / 2 - 0.05f, yBottom + height + 0.03f);
    glEnd();
    glDisable(GL_BLEND);
}
void drawVacuumTube3(float xStart, float xEnd, float y, float height) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glColor4f(0.75f, 0.75f, 0.85f, 0.5f);
    glBegin(GL_POLYGON);
    glVertex2f(xStart, y - height / 2);
    glVertex2f(xEnd, y - height / 2);
    for (int i = 0; i <= 20; i++) {
        float t = float(i) / 20.0f;
        float x = xStart + (xEnd - xStart) * t;
        float yOffset = height / 2 * cos(M_PI * t);
        glVertex2f(x, y + yOffset);
    }
    glEnd();

    glColor4f(0.2f, 0.2f, 0.3f, 0.9f);
    for (float x = -1.4f; x <= 1.4f; x += 0.4f) {
        glBegin(GL_QUADS);
        glVertex2f(x - 0.06f, y - height / 2);
        glVertex2f(x + 0.06f, y - height / 2);
        glVertex2f(x + 0.06f, y + height / 2);
        glVertex2f(x - 0.06f, y + height / 2);
```

```
        glEnd();
    }
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f); // Orange accents
    for (float x = -1.4f; x <= 1.4f; x += 0.4f) {
        glBegin(GL_QUADS);
        glVertex2f(x - 0.06f, y + height / 2 - 0.01f);
        glVertex2f(x + 0.06f, y + height / 2 - 0.01f);
        glVertex2f(x + 0.06f, y + height / 2);
        glVertex2f(x - 0.06f, y + height / 2);
        glEnd();
    }
    glDisable(GL_BLEND);
}
void drawMetroRail3(float xStart, float xEnd, float y, float thickness) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glColor4f(0.2f, 0.2f, 0.3f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(xStart, y - thickness / 2);
    glVertex2f(xEnd, y - thickness / 2);
    glVertex2f(xEnd, y + thickness / 2);
    glVertex2f(xStart, y + thickness / 2);
    glEnd();

    glColor4f(1.0f, 0.5f, 0.0f, 0.6f); // Orange rail accent
    glBegin(GL_QUADS);
    glVertex2f(xStart, y - thickness / 4);
    glVertex2f(xEnd, y - thickness / 4);
    glVertex2f(xEnd, y + thickness / 4);
    glVertex2f(xStart, y + thickness / 4);
    glEnd();
    glDisable(GL_BLEND);
}
void drawMetroTrain3(float x, float y, float width, float height) {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);
    float segmentWidth = width / 4.0f;
    float segmentHeight = height * 1.2f;
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(x - width / 2, y - segmentHeight / 2);
    glVertex2f(x + width / 2, y - segmentHeight / 2);
    glVertex2f(x + width / 2, y + segmentHeight / 2);
    glVertex2f(x - width / 2, y + segmentHeight / 2);
    glEnd();

    // Draw 4 compartments
    for (int i = 0; i < 4; i++) {
        float segX = x - width / 2 + segmentWidth * i + segmentWidth / 2;
        glColor4f(1.0f, 0.5f, 0.0f, 0.9f); // Orange
        glBegin(GL_POLYGON);
        glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 2);
        glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 2);
        glVertex2f(segX + segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
        glVertex2f(segX - segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
        glEnd();
        // Neon accent
```

```
    glColor4f(1.0f, 0.7f, 0.2f, 0.7f * flicker); // Lighter orange accent
    glBegin(GL_QUADS);
    glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 4);
    glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 4);
    glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 4 + 0.01f);
    glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 4 + 0.01f);
    glEnd();
    // Windows (2-3 per compartment, total 9)
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
    int numWindows = (i == 0 || i == 3) ? 2 : 3; // 2, 3, 2, 2 = 9 windows
    float startX = segX - segmentWidth / 2 + 0.03f;
    float stepX = (segmentWidth - 0.06f) / (numWindows - 1);
    for (int j = 0; j < numWindows; j++) {
        drawCircle(startX + j * stepX, y, 0.02f, 20);
    }
}
// Front and back noses (round + square)
glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
// Front nose
glBegin(GL_POLYGON);
glVertex2f(x + width / 2, y - segmentHeight / 3);
glVertex2f(x + width / 2 + 0.1f, y - segmentHeight / 3);
glVertex2f(x + width / 2 + 0.1f, y + segmentHeight / 3);
glVertex2f(x + width / 2, y + segmentHeight / 3);
glEnd();
glPushMatrix();
glTranslatef(x + width / 2 + 0.1f, y, 0.0f);
drawCircle(0.0f, 0.0f, 0.08f, 40);
glPopMatrix();
glColor4f(0.0f, 0.9f, 1.0f, 0.8f);
glBegin(GL_QUADS);
glVertex2f(x + width / 2 + 0.05f, y - 0.035f);
glVertex2f(x + width / 2 + 0.12f, y - 0.035f);
glVertex2f(x + width / 2 + 0.12f, y + 0.035f);
glVertex2f(x + width / 2 + 0.05f, y + 0.035f);
glEnd();
glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
glBegin(GL_POLYGON);
glVertex2f(x - width / 2, y - segmentHeight / 3);
glVertex2f(x - width / 2 - 0.1f, y - segmentHeight / 3);
glVertex2f(x - width / 2 - 0.1f, y + segmentHeight / 3);
glVertex2f(x - width / 2, y + segmentHeight / 3);
glEnd();
glPushMatrix();
glTranslatef(x - width / 2 - 0.1f, y, 0.0f);
drawCircle(0.0f, 0.0f, 0.08f, 40);
glPopMatrix();
glColor4f(0.0f, 0.9f, 1.0f, 0.8f);
glBegin(GL_QUADS);
glVertex2f(x - width / 2 - 0.12f, y - 0.035f);
glVertex2f(x - width / 2 - 0.05f, y - 0.035f);
glVertex2f(x - width / 2 - 0.05f, y + 0.035f);
glVertex2f(x - width / 2 - 0.12f, y + 0.035f);
glEnd();
glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
drawCircle(x + width / 2 + 0.1f, y, 0.02f, 20);
drawCircle(x - width / 2 - 0.1f, y, 0.02f, 20);
glColor4f(1.0f, 0.7f, 0.2f, 0.6f * flicker);
glBegin(GL_TRIANGLES);
glVertex2f(x - width / 2 - 0.12f, y);
```

```
    glVertex2f(x - width / 2 - 0.18f, y + 0.03f);
    glVertex2f(x - width / 2 - 0.18f, y - 0.03f);
    glEnd();
    glDisable(GL_BLEND);
}
void drawMetro3() {

    float yMid = (-0.05f + -0.60f) / 2.0f;
    float yBottom = yMid - 0.35f;
    float height = 0.76f;
    float railY = yBottom + height;
    float thickness = 0.06f;
    float tubeHeight = 0.12f;
    drawVacuumTube3(-1.5f, 1.5f, railY, tubeHeight);
    drawMetroRail3(-1.5f, 1.5f, railY, thickness);
    for (float x = -1.4f; x <= 1.4f; x += 0.4f) {
        drawMetroPillar3(x, yBottom, 0.05f, height);
    }
    drawMetroTrain3(commonTrainX, railY, 0.8f, 0.08f);
}
void drawScene3() {
    drawStars3();
    drawPlanets3();
    drawRocket3(rocketX3, 0.7f);
    drawGround3();
    drawCity3();
    drawRoadAndFootpath3();
    for (int i = 0; i < 8; i++) {
        drawVehicle3(vehicles3[i]);
    }
    drawMetro3();
}
void updateScene3() {
    timeCounter += 0.05f;
    commonTrainX += commonTrainSpeed;
    // Update vehicles (ferrari logic kept)
    for (int i = 0; i < 8; i++) {
        if (i == 1 && !ferrariMoving3) continue;
        float effectiveSpeed = vehicles3[i].speed;
        float newX = vehicles3[i].x + effectiveSpeed * (vehicles3[i].flip ? -1 : 1);
        float minDist = 0.4f;
        for (int j = 0; j < 8; j++) {
            if (i != j && vehicles3[j].y == vehicles3[i].y && vehicles3[i].flip == vehicles3[j].flip) {
                float dist = vehicles3[i].flip ? (vehicles3[j].x - newX) : (newX - vehicles3[j].x);
                if (dist > 0 && dist < minDist) {
                    newX = vehicles3[i].flip ? (vehicles3[j].x - minDist) : (vehicles3[j].x + minDist);
                    break;
                }
            }
        }
        vehicles3[i].x = newX;
        if (vehicles3[i].flip && vehicles3[i].x < -1.5f) vehicles3[i].x = 1.5f;
        if (!vehicles3[i].flip && vehicles3[i].x > 1.5f) vehicles3[i].x = -1.5f;
    }
    // Train movement is common
    if (commonTrainX > 2.0f) {
        currentScene = 4; // Shift to scene 4
        commonTrainX = -1.5f;
    }
    if (rocketMoving3) {
```

```
        rocketX3 += rocketSpeed3;
        if (rocketX3 > 1.5f) rocketX3 = -1.5f;
    }
}
// Scene 4 functions (kept as is, adjust trainX to commonTrainX, but keep meteor logic)
void drawCircle4(float cx, float cy, float r, int num_segments = 100) {
    drawCircle(cx, cy, r, num_segments);
}
void drawRocket4(float x, float y) {
    // same as scene3, full code
    // ... (copy from scene3)
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);
    // Rocket body
    glColor4f(0.8f, 0.8f, 0.9f, 1.0f); // Silver-gray
    glBegin(GL_POLYGON);
    glVertex2f(x - 0.15f, y - 0.03f); // Rear
    glVertex2f(x + 0.15f, y - 0.02f); // Front tip
    glVertex2f(x + 0.15f, y + 0.02f);
    glVertex2f(x - 0.15f, y + 0.03f);
    glEnd();
    // Fins
    glColor4f(0.7f, 0.7f, 0.8f, 1.0f);
    glBegin(GL_TRIANGLES);
    // Top fin
    glVertex2f(x - 0.15f, y + 0.03f);
    glVertex2f(x - 0.1f, y + 0.06f);
    glVertex2f(x - 0.05f, y + 0.03f);
    // Bottom fin
    glVertex2f(x - 0.15f, y - 0.03f);
    glVertex2f(x - 0.1f, y - 0.06f);
    glVertex2f(x - 0.05f, y - 0.03f);
    glEnd();
    // Glowing cockpit window
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f * flicker);
    drawCircle(x + 0.1f, y, 0.015f, 20);
    // Neon accents along body
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_QUADS);
    glVertex2f(x - 0.1f, y - 0.01f);
    glVertex2f(x + 0.1f, y - 0.01f);
    glVertex2f(x + 0.1f, y - 0.005f);
    glVertex2f(x - 0.1f, y - 0.005f);
    glEnd();
    // Fiery trail
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(x - 0.15f, y);
    glVertex2f(x - 0.25f, y + 0.04f);
    glVertex2f(x - 0.25f, y - 0.04f);
    glEnd();
    // Secondary trail glow
    glColor4f(1.0f, 0.7f, 0.2f, 0.5f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(x - 0.15f, y);
    glVertex2f(x - 0.3f, y + 0.06f);
    glVertex2f(x - 0.3f, y - 0.06f);
    glEnd();
    glDisable(GL_BLEND);
```

```
}
void initStars4() {
    // same as scene3
    // ...
    for (int i = 0; i < 200; i++) {
        starX4[i] = (rand() % 300 - 150) / 100.0f;
        starY4[i] = (rand() % 200 - 100) / 100.0f;
        starAlpha4[i] = 0.5f + (rand() % 50) / 100.0f;
    }
}
void drawStars4() {
    // same
    // ...
    glPointSize(4);
    glBegin(GL_POINTS);
    for (int i = 0; i < 200; i++) {
        float alpha = starAlpha4[i] + 0.3f * sin(timeCounter + i);
        glColor4f(1.0f, 1.0f, 1.0f, alpha);
        glVertex2f(starX4[i], starY4[i]);
    }
    glEnd();
}
void initDebris4() {

    for (int i = 0; i < 100; i++) {
        debrisX4[i] = meteorX4 + (rand() % 100 - 50) / 100.0f;
        debrisY4[i] = -0.63f;
        debrisSpeed4[i] = (rand() % 2 == 0 ? -0.01f : 0.01f) + (rand() % 5) / 1000.0f;
    }
}
void initSmoke4() {

    // ...
    for (int i = 0; i < 50; i++) {
        smokeX4[i] = meteorX4 + (rand() % 200 - 100) / 100.0f;
        smokeY4[i] = -0.63f;
        smokeAlpha4[i] = 0.3f + (rand() % 30) / 100.0f;
    }
}
void initFire4() {

    // ...
    for (int i = 0; i < 50; i++) {
        fireX4[i] = meteorX4 + (rand() % 200 - 100) / 100.0f;
        fireY4[i] = -0.63f;
        fireAlpha4[i] = 0.5f + (rand() % 30) / 100.0f;
        fireColorR4[i] = 1.0f - (rand() % 20) / 100.0f; // Vary red (0.8 to 1.0)
        fireColorG4[i] = 0.3f + (rand() % 30) / 100.0f; // Vary green (0.3 to 0.6)
    }
}
void initDrones4() {

    // ...
    for (int i = 0; i < 10; i++) {
        drones4[i].x = meteorX4 + (rand() % 200 - 100) / 100.0f;
        drones4[i].y = (rand() % 100) / 100.0f;
        drones4[i].angle = (rand() % 360) * M_PI / 180.0f;
        drones4[i].speed = 0.005f + (rand() % 3) / 1000.0f;
        drones4[i].armAngle = (rand() % 90) * M_PI / 180.0f; // Random initial arm angle
    }
```

```
}
void initSurvivors4() {

    // ...
    for (int i = 0; i < 3; i++) {
        survivors4[i].x = meteorX4 + (rand() % 100 - 50) / 100.0f;
        survivors4[i].y = -0.63f;
        survivors4[i].anim = (rand() % 100) / 100.0f; // Random phase
    }
}
void drawWindowsStrips4(float x, float y, float w, float h) {
    // same as scene3
    // ...
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    for (float j = y + 0.02f; j < y + h - 0.01f; j += 0.06f) {
        glBegin(GL_QUADS);
        glColor4f(0.0f, 0.9f, 1.0f, 0.85f);
        glVertex2f(x + 0.02f, j);
        glVertex2f(x + w - 0.02f, j);
        glVertex2f(x + w - 0.02f, j + 0.02f);
        glVertex2f(x + 0.02f, j + 0.02f);
        glEnd();
    }
 // ... (all window functions same as scene3, with prefix 4 if needed)
}
void drawWindowsGrid4(float x, float y, float w, float h) {
    float cellW = 0.04f;
    float cellH = 0.025f;
    for (float i = x + 0.02f; i < x + w - 0.01f; i += cellW + 0.01f) {
        for (float j = y + 0.02f; j < y + h - 0.01f; j += cellH + 0.01f) {
            glBegin(GL_QUADS);
            glColor4f(0.2f, 1.0f, 0.9f, 0.9f);
            glVertex2f(i, j);
            glVertex2f(i + cellW, j);
            glVertex2f(i + cellW, j + cellH);
            glVertex2f(i, j + cellH);
            glEnd();
        }
    }
}
void drawWindowsDots4(float x, float y, float w, float h) {
    for (float i = x + 0.03f; i < x + w - 0.02f; i += 0.06f) {
        for (float j = y + 0.03f; j < y + h - 0.02f; j += 0.06f) {
            glBegin(GL_POLYGON);
            glColor4f(0.8f, 0.9f, 1.0f, 0.9f);
            int segs = 12;
            float r = 0.008f;
            for (int s = 0; s < segs; s++) {
                float th = 2.0f * M_PI * s / segs;
                glVertex2f(i + r * cos(th), j + r * sin(th));
            }
            glEnd();
        }
    }
}
void drawWindowsHex4(float x, float y, float w, float h) {
    float band = 0.06f;
    for (float i = x + 0.02f; i < x + w - 0.01f; i += band) {
        glBegin(GL_QUADS);
        glColor4f(0.0f, 0.8f, 0.9f, 0.9f);
```

```
        glVertex2f(i, y + 0.02f);
        glVertex2f(i + band*0.6f, y + 0.02f);
        glVertex2f(i + band*0.6f, y + h - 0.02f);
        glVertex2f(i, y + h - 0.02f);
        glEnd();
    }
}
void drawFrontRect4(float x, float y, float w, float h, float r, float g, float b, int windowStyle, bool destroyed) {

    if (destroyed) {

        // Draw ruined version: shorter height, jagged top
        h *= 0.6f; // Shorter
        glColor3f(r * 0.5f, g * 0.5f, b * 0.5f); // Darker
        glBegin(GL_POLYGON);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glVertex2f(x + w, y + h * 0.8f);
        glVertex2f(x + w * 0.7f, y + h);
        glVertex2f(x + w * 0.3f, y + h);
        glVertex2f(x, y + h * 0.8f);
        glEnd();
        // Add cracks
        glColor3f(0.2f, 0.2f, 0.2f);
        glLineWidth(1.5f);
        glBegin(GL_LINES);
        glVertex2f(x + w * 0.2f, y);
        glVertex2f(x + w * 0.4f, y + h * 0.5f);
        glVertex2f(x + w * 0.6f, y + h * 0.3f);
        glVertex2f(x + w * 0.8f, y + h * 0.7f);
        glEnd();
    } else {

        glColor3f(r, g, b);
        glBegin(GL_QUADS);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glVertex2f(x + w, y + h);
        glVertex2f(x, y + h);
        glEnd();
        if (windowsOn4) {
            switch (windowStyle) {
                case 0: drawWindowsStrips4(x, y, w, h); break;
                case 1: drawWindowsGrid4(x, y, w, h); break;
                case 2: drawWindowsDots4(x, y, w, h); break;
                case 3: drawWindowsHex4(x, y, w, h); break;
                default: drawWindowsStrips4(x, y, w, h);
            }
        }
    }
}
void drawFrontTrapezoid4(float x, float y, float w, float h, float r, float g, float b, int windowStyle, bool destroyed)
{
    if (destroyed) {
        h *= 0.5f;
        glColor3f(r * 0.5f, g * 0.5f, b * 0.5f);
        glBegin(GL_POLYGON);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glVertex2f(x + w * 0.6f, y + h);
```

```
        glVertex2f(x + w * 0.4f, y + h * 1.2f);
        glVertex2f(x + w * 0.2f, y + h);
        glEnd();
        // Cracks
        glColor3f(0.2f, 0.2f, 0.2f);
        glBegin(GL_LINES);
        glVertex2f(x + w * 0.1f, y);
        glVertex2f(x + w * 0.5f, y + h);
        glEnd();
    } else {
        glColor3f(r,g,b);
        glBegin(GL_QUADS);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glVertex2f(x + w*0.75f, y + h);
        glVertex2f(x + w*0.25f, y + h);
        glEnd();
        if (windowsOn4) drawWindowsStrips4(x + w*0.25f, y, w*0.5f, h);
    }
}
void drawFrontDome4(float x, float y, float w, float h, float r, float g, float b, int windowStyle, bool destroyed) {
    if (destroyed) {
        h *= 0.7f;
        glColor3f(r * 0.5f, g * 0.5f, b * 0.5f);
        glBegin(GL_QUADS);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glVertex2f(x + w, y + h*0.5f);
        glVertex2f(x, y + h*0.5f);
        glEnd();
        // Broken dome
        glBegin(GL_TRIANGLES);
        glVertex2f(x + w/2, y + h*0.5f);
        glVertex2f(x + w*0.3f, y + h*0.7f);
        glVertex2f(x + w*0.7f, y + h*0.7f);
        glEnd();
    } else {
        glColor3f(r,g,b);
        glBegin(GL_QUADS);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glVertex2f(x + w, y + h*0.7f);
        glVertex2f(x, y + h*0.7f);
        glEnd();
        glColor3f(r*1.1f, g*1.1f, b*1.1f);
        drawCircle4(x + w/2, y + h*0.7f, w*0.5f);
        if (windowsOn4) drawWindowsHex4(x, y, w, h*0.7f);
    }
}
void drawFrontSpire4(float x, float y, float w, float h, float r, float g, float b, int windowStyle, bool destroyed) {
    if (destroyed) {
        h *= 0.4f;
        glColor3f(r * 0.5f, g * 0.5f, b * 0.5f);
        glBegin(GL_TRIANGLES);
        glVertex2f(x + w/2, y + h * 0.5f);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glEnd();
        // Add debris-like
        glBegin(GL_TRIANGLES);
```

```
        glVertex2f(x + w/2, y + h);
        glVertex2f(x + w*0.4f, y + h*0.8f);
        glVertex2f(x + w*0.6f, y + h*0.8f);
        glEnd();
    } else {
        glColor3f(r,g,b);
        glBegin(GL_TRIANGLES);
        glVertex2f(x + w/2, y + h);
        glVertex2f(x, y);
        glVertex2f(x + w, y);
        glEnd();
        if (windowsOn4) drawWindowsStrips4(x, y, w, h*0.6f);
    }
}
void drawCity4() {

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    drawFrontRect4(-1.4f, -0.05f, 0.18f, 0.6f, 0.12f, 0.26f, 0.6f, 0, cityDestroyed4);
    // ... (full)
     drawFrontTrapezoid4(-1.16f, -0.05f, 0.22f, 0.65f, 0.18f, 0.35f, 0.8f, 1, cityDestroyed4);
    drawFrontDome4(-0.92f, -0.05f, 0.20f, 0.55f, 0.25f, 0.45f, 0.9f, 3, cityDestroyed4);
    drawFrontSpire4(-0.68f, -0.05f, 0.18f, 0.7f, 0.2f, 0.5f, 0.95f, 0, cityDestroyed4);
    drawFrontRect4(-0.46f, -0.05f, 0.2f, 0.58f, 0.22f, 0.5f, 0.9f, 2, cityDestroyed4);
    drawFrontTrapezoid4(-0.22f, -0.05f, 0.18f, 0.62f, 0.28f, 0.55f, 1.0f, 0, cityDestroyed4);
    drawFrontDome4(0.02f, -0.05f, 0.24f, 0.5f, 0.3f, 0.6f, 1.0f, 1, cityDestroyed4);
    drawFrontSpire4(0.28f, -0.05f, 0.18f, 0.66f, 0.28f, 0.58f, 0.98f, 2, cityDestroyed4);
    drawFrontRect4(0.52f, -0.05f, 0.2f, 0.56f, 0.35f, 0.65f, 1.0f, 3, cityDestroyed4);
    drawFrontTrapezoid4(0.76f, -0.05f, 0.18f, 0.6f, 0.38f, 0.68f, 1.0f, 0, cityDestroyed4);
    drawFrontSpire4(1.0f, -0.05f, 0.22f, 0.58f, 0.4f, 0.72f, 1.0f, 1, cityDestroyed4);
    drawFrontRect4(1.26f, -0.05f, 0.18f, 0.54f, 0.45f, 0.75f, 1.0f, 2, cityDestroyed4);
    glDisable(GL_BLEND);
}
void drawVehicle4(const Vehicle4& v) {
    // same as scene3, but with destroyed check? No, separate drawDestroyedVehicle4

        glPushMatrix();
    glTranslatef(v.x, v.y, 0.0f);
    glScalef(v.scale, v.scale, 1.0f);
    if(v.flip) glScalef(-1.0f, 1.0f, 1.0f);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);
    glColor3f(v.r, v.g, v.b);
    if (!v.isBus) {
        if (v.type == 0) { // Cybertruck-inspired
            glBegin(GL_POLYGON);
            glVertex2f(-0.14f, -0.05f);
            glVertex2f(0.14f, -0.05f);
            glVertex2f(0.11f, -0.02f);
            glVertex2f(0.09f, 0.07f);
            glVertex2f(-0.09f, 0.07f);
            glVertex2f(-0.11f, -0.02f);
            glEnd();
            // Neon underlights
            glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
            glBegin(GL_QUADS);
            glVertex2f(-0.12f, -0.05f);
            glVertex2f(0.12f, -0.05f);
            glVertex2f(0.12f, -0.06f);
            glVertex2f(-0.12f, -0.06f);
            glEnd();
```

```
    // Windows
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(-0.07f, 0.03f);
    glVertex2f(0.07f, 0.03f);
    glVertex2f(0.06f, 0.06f);
    glVertex2f(-0.06f, 0.06f);
    glEnd();
    // Headlights
    glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
    drawCircle(0.13f, -0.015f, 0.015f, 20);
    drawCircle(0.13f, 0.015f, 0.015f, 20);
    // Thrusters
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
    drawCircle(-0.12f, -0.03f, 0.02f, 20);
    // Jet flames
    float jetPos = v.flip ? 0.14f : -0.14f;
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(jetPos, 0.0f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
    glEnd();
} else if (v.type == 1) { // Ferrari-inspired
    glBegin(GL_POLYGON);
    glVertex2f(-0.12f, -0.04f);
    glVertex2f(0.15f, -0.04f);
    glVertex2f(0.13f, -0.01f);
    glVertex2f(0.09f, 0.04f);
    glVertex2f(-0.09f, 0.04f);
    glVertex2f(-0.11f, -0.01f);
    glEnd();
    // Neon underlights
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_QUADS);
    glVertex2f(-0.11f, -0.04f);
    glVertex2f(0.14f, -0.04f);
    glVertex2f(0.14f, -0.05f);
    glVertex2f(-0.11f, -0.05f);
    glEnd();
    // Curved window
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
    drawCircle(0.0f, 0.02f, 0.025f, 20);
    // Headlights
    glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
    drawCircle(0.14f, -0.015f, 0.015f, 20);
    drawCircle(0.14f, 0.015f, 0.015f, 20);
    // Thrusters
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
    drawCircle(-0.11f, -0.03f, 0.02f, 20);
    // Jet flames
    float jetPos = v.flip ? 0.13f : -0.13f;
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(jetPos, 0.0f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
    glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
    glEnd();
} else if (v.type == 2) { // Lamborghini-inspired
    glBegin(GL_POLYGON);
```

```
        glVertex2f(-0.13f, -0.045f);
        glVertex2f(0.13f, -0.045f);
        glVertex2f(0.11f, -0.015f);
        glVertex2f(0.07f, 0.045f);
        glVertex2f(-0.07f, 0.045f);
        glVertex2f(-0.11f, -0.015f);
        glEnd();
        // Neon underlights
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        glBegin(GL_QUADS);
        glVertex2f(-0.12f, -0.045f);
        glVertex2f(0.12f, -0.045f);
        glVertex2f(0.12f, -0.055f);
        glVertex2f(-0.12f, -0.055f);
        glEnd();
        // Angular windows
        glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
        glBegin(GL_QUADS);
        glVertex2f(-0.06f, 0.02f);
        glVertex2f(0.06f, 0.02f);
        glVertex2f(0.05f, 0.04f);
        glVertex2f(-0.05f, 0.04f);
        glEnd();
        // Headlights
        glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
        drawCircle(0.12f, -0.015f, 0.015f, 20);
        drawCircle(0.12f, 0.015f, 0.015f, 20);
        // Thrusters
        glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
        drawCircle(-0.12f, -0.03f, 0.02f, 20);
        // Jet flames
        float jetPos = v.flip ? 0.13f : -0.13f;
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        glBegin(GL_TRIANGLES);
        glVertex2f(jetPos, 0.0f);
        glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
        glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
        glEnd();
    } else { // Generic Sports Car
        glBegin(GL_POLYGON);
        glVertex2f(-0.11f, -0.04f);
        glVertex2f(0.12f, -0.04f);
        glVertex2f(0.11f, -0.01f);
        glVertex2f(0.09f, 0.04f);
        glVertex2f(-0.09f, 0.04f);
        glVertex2f(-0.11f, -0.01f);
        glEnd();
        // Neon underlights
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        glBegin(GL_QUADS);
        glVertex2f(-0.1f, -0.04f);
        glVertex2f(0.11f, -0.04f);
        glVertex2f(0.11f, -0.05f);
        glVertex2f(-0.1f, -0.05f);
        glEnd();
        // Window
        glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
        drawCircle(0.0f, 0.02f, 0.025f, 20);
        // Headlight
        glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
```

```
            drawCircle(0.11f, 0.0f, 0.015f, 20);
            // Thrusters
            glColor4f(1.0f, 0.5f, 0.0f, 0.9f * flicker);
            drawCircle(-0.1f, -0.03f, 0.02f, 20);
            // Jet flames
            float jetPos = v.flip ? 0.12f : -0.12f;
            glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
            glBegin(GL_TRIANGLES);
            glVertex2f(jetPos, 0.0f);
            glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
            glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
            glEnd();
        }
    } else {
        // Bus
        glBegin(GL_POLYGON);
        glVertex2f(-0.18f, -0.05f);
        glVertex2f(0.18f, -0.05f);
        glVertex2f(0.18f, 0.0f);
        glVertex2f(0.18f, 0.05f);
        glVertex2f(-0.18f, 0.05f);
        glVertex2f(-0.18f, 0.0f);
        glEnd();
        // Neon underlights
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        glBegin(GL_QUADS);
        glVertex2f(-0.17f, -0.05f);
        glVertex2f(0.17f, -0.05f);
        glVertex2f(0.17f, -0.06f);
        glVertex2f(-0.17f, -0.06f);
        glEnd();
        // Dome
        glColor4f(0.0f, 0.4f, 0.8f, 0.9f);
        drawCircle(0.0f, 0.05f, 0.07f, 30);
        // Windows
        glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
        for(float wx = -0.15f; wx < 0.15f; wx += 0.06f){
            drawCircle(wx, 0.0f, 0.025f, 20);
        }
        // Front light
        glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
        drawCircle(0.17f, 0.0f, 0.02f, 20);
        // Jets
        float jetPos = v.flip ? 0.18f : -0.18f;
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        glBegin(GL_TRIANGLES);
        glVertex2f(jetPos, 0.0f);
        glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), 0.02f);
        glVertex2f(jetPos + (v.flip ? 0.05f : -0.05f), -0.02f);
        glEnd();
    }
    glPopMatrix();
}
void drawDestroyedVehicle4(const Vehicle4& v) {

    // ...
    glPushMatrix();
    glTranslatef(v.x, v.y, 0.0f);
    glScalef(v.scale, v.scale, 1.0f);
    if(v.flip) glScalef(-1.0f, 1.0f, 1.0f);
```

```
// Draw broken parts with darker colors and jagged shapes
glColor3f(v.r * 0.5f, v.g * 0.5f, v.b * 0.5f);
if (!v.isBus) {
    // Front part
    glBegin(GL_POLYGON);
    glVertex2f(-0.05f, -0.04f);
    glVertex2f(0.05f, -0.04f);
    glVertex2f(0.03f, 0.02f);
    glVertex2f(-0.03f, 0.02f);
    glEnd();
    // Rear part tilted
    glPushMatrix();
    glTranslatef(-0.1f, 0.0f, 0.0f);
    glRotatef(15.0f, 0.0f, 0.0f, 1.0f);
    glBegin(GL_POLYGON);
    glVertex2f(-0.05f, -0.04f);
    glVertex2f(0.05f, -0.04f);
    glVertex2f(0.03f, 0.02f);
    glVertex2f(-0.03f, 0.02f);
    glEnd();
    glPopMatrix();
    // Debris pieces
    glBegin(GL_TRIANGLES);
    glVertex2f(0.1f, -0.05f);
    glVertex2f(0.15f, -0.02f);
    glVertex2f(0.12f, 0.0f);
    glEnd();
    glBegin(GL_TRIANGLES);
    glVertex2f(-0.15f, -0.03f);
    glVertex2f(-0.2f, 0.0f);
    glVertex2f(-0.18f, -0.05f);
    glEnd();
    // Smoke effect on vehicle
    glEnable(GL_BLEND);
    glColor4f(0.3f, 0.3f, 0.3f, 0.5f);
    drawCircle(0.0f, 0.05f, 0.05f, 20);
    glDisable(GL_BLEND);
} else {
    // Bus broken
    glBegin(GL_POLYGON);
    glVertex2f(-0.1f, -0.05f);
    glVertex2f(0.1f, -0.05f);
    glVertex2f(0.08f, 0.03f);
    glVertex2f(-0.08f, 0.03f);
    glEnd();
    // Side part
    glPushMatrix();
    glTranslatef(0.15f, 0.0f, 0.0f);
    glRotatef(-20.0f, 0.0f, 0.0f, 1.0f);
    glBegin(GL_POLYGON);
    glVertex2f(-0.05f, -0.05f);
    glVertex2f(0.05f, -0.05f);
    glVertex2f(0.03f, 0.03f);
    glVertex2f(-0.03f, 0.03f);
    glEnd();
    glPopMatrix();
    // Debris
    glBegin(GL_QUADS);
    glVertex2f(-0.15f, -0.06f);
    glVertex2f(-0.1f, -0.06f);
```

```
        glVertex2f(-0.1f, -0.03f);
        glVertex2f(-0.15f, -0.03f);
        glEnd();
    }
    glPopMatrix();
}
void drawRoadAndFootpath4() {
    // same as scene3
    // ...
    // Road
    glColor3f(0.15f, 0.15f, 0.15f);
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.05f);
    glVertex2f(1.5f, -0.05f);
    glVertex2f(1.5f, -0.60f);
    glVertex2f(-1.5f, -0.60f);
    glEnd();
    // Dividers
    glColor3f(1.0f, 1.0f, 1.0f);
    float dividerY[3] = {-0.20f, -0.35f, -0.50f};
    for(int d=0; d<3; d++){
        for(float x=-1.4f; x<=1.4f; x+=0.3f) {
            glBegin(GL_QUADS);
            glVertex2f(x, dividerY[d]+0.01f);
            glVertex2f(x+0.1f, dividerY[d]+0.01f);
            glVertex2f(x+0.1f, dividerY[d]-0.01f);
            glVertex2f(x, dividerY[d]-0.01f);
            glEnd();
        }
    }
    // Footpaths
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    // Top footpath
    glColor4f(0.3f, 0.3f, 0.3f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.02f);
    glVertex2f(1.5f, -0.02f);
    glVertex2f(1.5f, -0.05f);
    glVertex2f(-1.5f, -0.05f);
    glEnd();
    // Bottom footpath
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.60f);
    glVertex2f(1.5f, -0.60f);
    glVertex2f(1.5f, -0.63f);
    glVertex2f(-1.5f, -0.63f);
    glEnd();
    // Glowing markers
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * (0.8f + 0.2f * sin(timeCounter)));
    for (float x = -1.4f; x <= 1.4f; x += 0.2f) {
        drawCircle(x, -0.035f, 0.01f, 20);
        drawCircle(x, -0.615f, 0.01f, 20);
    }
    // Neon lamplights
    glColor4f(1.0f, 1.0f, 1.0f, 0.8f); // White stands
    for (float x = -1.3f; x <= 1.3f; x += 0.4f) {
        glBegin(GL_QUADS);
        glVertex2f(x - 0.01f, -0.02f);
        glVertex2f(x + 0.01f, -0.02f);
```

```
        glVertex2f(x + 0.01f, 0.02f);
        glVertex2f(x - 0.01f, 0.02f);
        glEnd();
        glBegin(GL_QUADS);
        glVertex2f(x - 0.01f, -0.63f);
        glVertex2f(x + 0.01f, -0.63f);
        glVertex2f(x + 0.01f, -0.59f);
        glVertex2f(x - 0.01f, -0.59f);
        glEnd();
    }
    glColor4f(1.0f, 0.5f, 0.0f, 0.8f * (0.8f + 0.2f * sin(timeCounter)));
    for (float x = -1.3f; x <= 1.3f; x += 0.4f) {
        drawCircle(x, 0.02f, 0.015f, 20);
        drawCircle(x, -0.59f, 0.015f, 20);
    }
    glDisable(GL_BLEND);
}
void drawDestroyedRoadAndFootpath4() {

    // ...
    // Destroyed road with cracks and holes
    glColor3f(0.1f, 0.1f, 0.1f);
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.05f);
    glVertex2f(1.5f, -0.05f);
    glVertex2f(1.5f, -0.60f);
    glVertex2f(-1.5f, -0.60f);
    glEnd();
    // Cracks
    glColor3f(0.05f, 0.05f, 0.05f);
    glLineWidth(2.0f);
    glBegin(GL_LINES);
    for (int i = 0; i < 10; i++) {
        float x1 = meteorX4 + (rand() % 100 - 50) / 100.0f;
        float y1 = -0.05f + (rand() % 55) / 100.0f * -1;
        float x2 = x1 + (rand() % 50 - 25) / 100.0f;
        float y2 = y1 + (rand() % 20 - 10) / 100.0f;
        glVertex2f(x1, y1);
        glVertex2f(x2, y2);
    }
    glEnd();
    // Holes
    glColor3f(0.05f, 0.05f, 0.05f);
    drawCircle(meteorX4 - 0.2f, -0.3f, 0.08f, 50);
    drawCircle(meteorX4 + 0.1f, -0.4f, 0.05f, 50);
    // Destroyed dividers (partial)
    glColor3f(0.8f, 0.8f, 0.8f);
    float dividerY[3] = {-0.20f, -0.35f, -0.50f};
    for(int d=0; d<3; d++){
        for(float x=-1.4f; x<=1.4f; x+=0.3f) {
            if (fabs(x - meteorX4) > 0.3f) { // Skip near impact
                glBegin(GL_QUADS);
                glVertex2f(x, dividerY[d]+0.01f);
                glVertex2f(x+0.1f, dividerY[d]+0.01f);
                glVertex2f(x+0.1f, dividerY[d]-0.01f);
                glVertex2f(x, dividerY[d]-0.01f);
                glEnd();
            }
        }
    }
```

```
    // Destroyed footpaths with debris
    glEnable(GL_BLEND);
    glColor4f(0.2f, 0.2f, 0.2f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.02f);
    glVertex2f(1.5f, -0.02f);
    glVertex2f(1.5f, -0.05f);
    glVertex2f(-1.5f, -0.05f);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(-1.5f, -0.60f);
    glVertex2f(1.5f, -0.60f);
    glVertex2f(1.5f, -0.63f);
    glVertex2f(-1.5f, -0.63f);
    glEnd();
    glDisable(GL_BLEND);
}
void drawGround4() {
    // same as scene3
    // ...
    glBegin(GL_QUADS);
    glColor3f(0.8f, 0.3f, 0.1f);
    glVertex2f(-1.5f, -0.63f);
    glVertex2f(1.5f, -0.63f);
    glColor3f(0.4f, 0.1f, 0.05f);
    glVertex2f(1.5f, -1.0f);
    glVertex2f(-1.5f, -1.0f);
    glEnd();
    glColor3f(0.6f, 0.15f, 0.05f);
    glBegin(GL_POLYGON);
    glVertex2f(-1.5f, -0.63f);
    glVertex2f(-1.2f, -0.58f);
    glVertex2f(-0.9f, -0.63f);
    glVertex2f(-0.6f, -0.61f);
    glVertex2f(-0.3f, -0.63f);
    glVertex2f(0.0f, -0.60f);
    glVertex2f(0.3f, -0.63f);
    glVertex2f(0.7f, -0.59f);
    glVertex2f(1.2f, -0.63f);
    glVertex2f(1.5f, -0.62f);
    glEnd();
    glColor3f(0.3f, 0.1f, 0.05f);
    drawCircle(-0.6f, -0.8f, 0.08f, 40);
    drawCircle(0.4f, -0.75f, 0.06f, 40);
    drawCircle(-0.2f, -0.9f, 0.1f, 40);
}
void drawDestroyedGround4() {

    // ...
    // Draw damaged ground
    glBegin(GL_QUADS);
    glColor3f(0.4f, 0.2f, 0.1f);
    glVertex2f(-1.5f, -0.63f);
    glVertex2f(1.5f, -0.63f);
    glVertex2f(1.5f, -1.0f);
    glVertex2f(-1.5f, -1.0f);
    glEnd();
    // More craters and cracks
    glColor3f(0.2f, 0.1f, 0.05f);
    drawCircle(meteorX4 - 0.5f, -0.8f, 0.1f, 40);
```

```
    drawCircle(meteorX4 + 0.3f, -0.85f, 0.08f, 40);
    drawCircle(meteorX4, -0.75f, 0.12f, 40);
    drawCircle(meteorX4 - 0.2f, -0.9f, 0.07f, 40);
    // Cracks
    glLineWidth(2.0f);
    glBegin(GL_LINES);
    glVertex2f(meteorX4 - 0.4f, -0.63f);
    glVertex2f(meteorX4 - 0.6f, -0.8f);
    glVertex2f(meteorX4 + 0.4f, -0.63f);
    glVertex2f(meteorX4 + 0.6f, -0.8f);
    glEnd();
}
void drawPlanet4(float x, float y, float r, int type) {
    // same as scene3
    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    if (type == 0) { // Sun
        glColor4f(1.0f, 0.85f, 0.5f, 0.9f);
        drawCircle(x, y, r, 100);
        // Flares
        glColor4f(1.0f, 0.95f, 0.7f, 0.7f * (0.8f + 0.2f * sin(timeCounter)));
        for (int i = 0; i < 12; i++) {
            float theta = 2.0f * M_PI * float(i) / 12.0f;
            glBegin(GL_TRIANGLES);
            glVertex2f(x, y);
            glVertex2f(x + r * 1.3f * cos(theta), y + r * 1.3f * sin(theta));
            glVertex2f(x + r * 1.3f * cos(theta + 0.2f), y + r * 1.3f * sin(theta + 0.2f));
            glEnd();
        }
    } else if (type == 1) { // Moon
        glBegin(GL_POLYGON);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float intensity = 1.0f + 0.2f * cos(theta);
            glColor4f(1.0f * intensity, 1.0f * intensity, 0.95f * intensity, 0.9f);
            float vx = r * cos(theta);
            float vy = r * sin(theta);
            glVertex2f(x + vx, y + vy);
        }
        glEnd();
        // Craters
        glColor4f(0.85f, 0.85f, 0.85f, 0.8f);
        drawCircle(x - r * 0.4f, y + r * 0.3f, r * 0.2f, 20);
        drawCircle(x + r * 0.3f, y - r * 0.2f, r * 0.15f, 20);
        drawCircle(x - r * 0.2f, y - r * 0.1f, r * 0.1f, 20);
    } else if (type == 2) { // Earth
        glBegin(GL_POLYGON);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float t = float(i) / 100.0f;
            float blue = t < 0.7f ? 0.7f : 0.2f;
            float green = t < 0.7f ? 0.3f : (t < 0.85f ? 0.6f : 0.4f);
            float red = t < 0.7f ? 0.1f : 0.3f;
            float intensity = 1.0f + 0.2f * cos(theta);
            glColor4f(red * intensity, green * intensity, blue * intensity, 0.9f);
            float vx = r * cos(theta);
            float vy = r * sin(theta);
            glVertex2f(x + vx, y + vy);
        }
```

```
        glEnd();
        // Continents
        glColor4f(0.3f, 0.6f, 0.2f, 0.8f);
        drawCircle(x + r * 0.3f, y, r * 0.25f, 20);
        drawCircle(x - r * 0.2f, y + r * 0.3f, r * 0.2f, 20);
        // Clouds
        glColor4f(1.0f, 1.0f, 1.0f, 0.7f);
        drawCircle(x + r * 0.1f, y - r * 0.2f, r * 0.15f, 20);
        drawCircle(x - r * 0.3f, y + r * 0.1f, r * 0.1f, 20);
    } else if (type == 3) { // Saturn
        glBegin(GL_POLYGON);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float intensity = 1.0f + 0.3f * cos(theta);
            glColor4f(1.0f * intensity, 0.95f * intensity, 0.8f * intensity, 0.9f);
            float vx = r * cos(theta);
            float vy = r * sin(theta);
            glVertex2f(x + vx, y + vy);
        }
        glEnd();
        // Rings
        glColor4f(0.95f, 0.95f, 0.85f, 0.8f);
        glBegin(GL_LINE_LOOP);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float rx = r * 1.6f * cos(theta);
            float ry = r * 0.3f * sin(theta);
            glVertex2f(x + rx, y + ry);
        }
        glEnd();
        glBegin(GL_LINE_LOOP);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float rx = r * 1.4f * cos(theta);
            float ry = r * 0.2f * sin(theta);
            glVertex2f(x + rx, y + ry);
        }
        glEnd();
        glBegin(GL_LINE_LOOP);
        for (int i = 0; i < 100; i++) {
            float theta = 2.0f * M_PI * float(i) / 100.0f;
            float rx = r * 1.2f * cos(theta);
            float ry = r * 0.1f * sin(theta);
            glVertex2f(x + rx, y + ry);
        }
        glEnd();
    }
    glDisable(GL_BLEND);
}
void drawPlanets4() {
    // same
    // ...
    drawPlanet4(-0.6f, 0.85f, 0.15f, 0); // Sun
    drawPlanet4(-0.3f, 0.75f, 0.08f, 1); // Moon
    drawPlanet4(0.0f, 0.8f, 0.12f, 2); // Earth
    drawPlanet4(0.3f, 0.85f, 0.1f, 3); // Saturn
}
void drawMeteor4() {
    if (!meteorFalling4 || meteorHit4) return;
    glEnable(GL_BLEND);
```

```
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    // Meteor core with gradient
    for (int i = 0; i < 20; i++) {
        float frac = float(i) / 20.0f;
        glColor4f(1.0f, 0.3f - frac * 0.2f, 0.1f - frac * 0.05f, 0.9f - frac * 0.4f);
        drawCircle(meteorX4, meteorY4, meteorSize4 * (1.0f - frac * 0.5f), 50);
    }
    // Craters on meteor
    glColor4f(0.5f, 0.1f, 0.05f, 0.8f);
    drawCircle(meteorX4 + meteorSize4 * 0.3f, meteorY4 + meteorSize4 * 0.2f, meteorSize4 * 0.2f, 20);
    drawCircle(meteorX4 - meteorSize4 * 0.4f, meteorY4 - meteorSize4 * 0.1f, meteorSize4 * 0.15f, 20);
    drawCircle(meteorX4 + meteorSize4 * 0.1f, meteorY4 - meteorSize4 * 0.3f, meteorSize4 * 0.1f, 20);
    // Glowing trail
    float flicker = 0.8f + 0.2f * sin(timeCounter * 5.0f);
    glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(meteorX4, meteorY4);
    glVertex2f(meteorX4 - 0.1f, meteorY4 + 0.2f);
    glVertex2f(meteorX4 + 0.1f, meteorY4 + 0.2f);
    glEnd();
    // Secondary trail
    glColor4f(1.0f, 0.7f, 0.2f, 0.4f * flicker);
    glBegin(GL_TRIANGLES);
    glVertex2f(meteorX4, meteorY4);
    glVertex2f(meteorX4 - 0.15f, meteorY4 + 0.3f);
    glVertex2f(meteorX4 + 0.15f, meteorY4 + 0.3f);
    glEnd();
    glDisable(GL_BLEND);
}
void drawDebris4() {

    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glPointSize(3);
    glBegin(GL_POINTS);
    for (int i = 0; i < 100; i++) {
        glColor4f(0.5f, 0.5f, 0.5f, 0.8f);
        glVertex2f(debrisX4[i], debrisY4[i]);
    }
    glEnd();
    glDisable(GL_BLEND);
}
void drawSmoke4() {

    // ...
     glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    for (int i = 0; i < 50; i++) {
        glColor4f(0.3f, 0.3f, 0.3f, smokeAlpha4[i] * (0.8f + 0.2f * sin(timeCounter + i)));
        drawCircle(smokeX4[i], smokeY4[i], 0.1f + sin(timeCounter + i) * 0.02f, 30); // Puffing effect
    }
    glDisable(GL_BLEND);
}
void drawFire4() {

    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 3.0f);
```

```
    for (int i = 0; i < 50; i++) {
        float size = 0.05f + (rand() % 20) / 100.0f * 0.03f + sin(timeCounter + i) * 0.015f; // Varied size
        // Core flame (yellowish)
        glColor4f(1.0f, 0.9f, 0.3f, fireAlpha4[i] * flicker * 0.8f);
        drawCircle(fireX4[i], fireY4[i], size * 0.6f, 30);
        // Inner flame (orange)
        glColor4f(fireColorR4[i], fireColorG4[i], 0.0f, fireAlpha4[i] * flicker);
        drawCircle(fireX4[i], fireY4[i] + size * 0.2f, size * 0.8f, 30);
        // Outer flame (redish, multiple tongues for realism)
        glColor4f(fireColorR4[i] * 0.8f, fireColorG4[i] * 0.5f, 0.1f, fireAlpha4[i] * flicker * 0.6f);
        for (int j = 0; j < 3; j++) { // Multiple flame tongues
            float offsetX = (rand() % 20 - 10) / 100.0f * size * 0.5f;
            float flameHeight = size * (1.2f + 0.3f * sin(timeCounter + i + j));
            glBegin(GL_TRIANGLES);
            glVertex2f(fireX4[i] + offsetX - 0.02f, fireY4[i]);
            glVertex2f(fireX4[i] + offsetX + 0.02f, fireY4[i]);
            glVertex2f(fireX4[i] + offsetX + cos(timeCounter + i + j) * 0.01f, fireY4[i] + flameHeight);
            glEnd();
        }
        // Sparks (more for realism)
        glColor4f(1.0f, 0.9f, 0.5f, fireAlpha4[i] * flicker * 0.5f);
        glPointSize(2.0f + sin(timeCounter + i) * 1.0f);
        glBegin(GL_POINTS);
        for (int j = 0; j < 3; j++) { // More sparks
            glVertex2f(fireX4[i] + (rand() % 20 - 10) / 100.0f * size, fireY4[i] + (rand() % 30) / 100.0f * size * 1.2f);
        }
        glEnd();
    }
    glDisable(GL_BLEND);
}
void drawDrone4(float x, float y, float angle, float armAngle) {

    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glPushMatrix();
    glTranslatef(x, y, 0.0f);
    glRotatef(angle * 180.0f / M_PI, 0.0f, 0.0f, 1.0f);
    // Drone body (sleeker, hexagonal shape)
    glBegin(GL_POLYGON);
    glColor4f(0.8f, 0.8f, 0.9f, 0.9f); // Metallic silver
    for (int i = 0; i < 6; i++) {
        float theta = 2.0f * M_PI * i / 6.0f;
        glVertex2f(0.035f * cos(theta), 0.025f * sin(theta));
    }
    glEnd();
    // Central core with glowing sensor
    float flicker = 0.8f + 0.2f * sin(timeCounter * 5.0f);
    glColor4f(0.0f, 0.9f, 1.0f, 0.9f * flicker);
    drawCircle(0.0f, 0.0f, 0.015f, 20);
    // Propellers (four, with more detail)
    glColor4f(0.6f, 0.6f, 0.7f, 0.9f);
    for (int i = 0; i < 4; i++) {
        float theta = i * M_PI / 2 + timeCounter * 10.0f; // Faster rotation
        glPushMatrix();
        glTranslatef(0.04f * cos(i * M_PI / 2), 0.04f * sin(i * M_PI / 2), 0.0f);
        glRotatef(theta * 180.0f / M_PI, 0.0f, 0.0f, 1.0f);
        glBegin(GL_QUADS);
        glVertex2f(-0.02f, -0.005f);
        glVertex2f(0.02f, -0.005f);
```

```
        glVertex2f(0.02f, 0.005f);
        glVertex2f(-0.02f, 0.005f);
        glEnd();
        glPopMatrix();
    }
    // Articulated repair arms (two arms)
    glColor4f(0.7f, 0.7f, 0.8f, 0.9f);
    for (int i = -1; i <= 1; i += 2) {
        glPushMatrix();
        glTranslatef(0.02f * i, 0.0f, 0.0f);
        glRotatef(armAngle * 180.0f / M_PI * i, 0.0f, 0.0f, 1.0f);
        // Upper arm
        glBegin(GL_QUADS);
        glVertex2f(0.0f, -0.005f);
        glVertex2f(0.03f, -0.005f);
        glVertex2f(0.03f, 0.005f);
        glVertex2f(0.0f, 0.005f);
        glEnd();
        // Lower arm with tool
        glPushMatrix();
        glTranslatef(0.03f, 0.0f, 0.0f);
        glRotatef(30.0f * sin(timeCounter) * i, 0.0f, 0.0f, 1.0f);
        glBegin(GL_QUADS);
        glVertex2f(0.0f, -0.004f);
        glVertex2f(0.02f, -0.004f);
        glVertex2f(0.02f, 0.004f);
        glVertex2f(0.0f, 0.004f);
        glEnd();
        // Tool tip (glowing)
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f * flicker);
        drawCircle(0.02f, 0.0f, 0.005f, 15);
        glPopMatrix();
        glPopMatrix();
    }
    glPopMatrix();
    glDisable(GL_BLEND);
}
void drawSurvivor4(const Survivor4& s) {

    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    // Astronaut suit body (white with orange accents)
    glColor4f(0.9f, 0.9f, 0.9f, 0.9f); // White suit
    glBegin(GL_QUADS);
    glVertex2f(s.x - 0.025f, s.y - 0.02f);
    glVertex2f(s.x + 0.025f, s.y - 0.02f);
    glVertex2f(s.x + 0.025f, s.y + 0.1f);
    glVertex2f(s.x - 0.025f, s.y + 0.1f);
    glEnd();
    // Orange stripes
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(s.x - 0.025f, s.y + 0.05f);
    glVertex2f(s.x + 0.025f, s.y + 0.05f);
    glVertex2f(s.x + 0.025f, s.y + 0.07f);
    glVertex2f(s.x - 0.025f, s.y + 0.07f);
    glEnd();
    // Legs (suit pants)
    glColor4f(0.9f, 0.9f, 0.9f, 0.9f);
```

```
    glBegin(GL_QUADS);
    glVertex2f(s.x - 0.02f, s.y - 0.07f);
    glVertex2f(s.x - 0.005f, s.y - 0.07f);
    glVertex2f(s.x - 0.005f, s.y - 0.02f);
    glVertex2f(s.x - 0.02f, s.y - 0.02f);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(s.x + 0.005f, s.y - 0.07f);
    glVertex2f(s.x + 0.02f, s.y - 0.07f);
    glVertex2f(s.x + 0.02f, s.y - 0.02f);
    glVertex2f(s.x + 0.005f, s.y - 0.02f);
    glEnd();
    // Boots
    glColor4f(0.3f, 0.3f, 0.3f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(s.x - 0.02f, s.y - 0.07f);
    glVertex2f(s.x - 0.005f, s.y - 0.07f);
    glVertex2f(s.x - 0.005f, s.y - 0.09f);
    glVertex2f(s.x - 0.02f, s.y - 0.09f);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(s.x + 0.005f, s.y - 0.07f);
    glVertex2f(s.x + 0.02f, s.y - 0.07f);
    glVertex2f(s.x + 0.02f, s.y - 0.09f);
    glVertex2f(s.x + 0.005f, s.y - 0.09f);
    glEnd();
    // Helmet
    glColor4f(1.0f, 1.0f, 1.0f, 0.5f); // Semi-transparent glass
    drawCircle(s.x, s.y + 0.12f, 0.03f, 30);
    glColor4f(0.8f, 0.8f, 0.8f, 0.9f); // Helmet frame
    glLineWidth(2.0f);
    glBegin(GL_LINE_LOOP);
    for (int i = 0; i < 30; i++) {
        float theta = 2.0f * M_PI * i / 30.0f;
        glVertex2f(s.x + 0.03f * cos(theta), s.y + 0.12f + 0.03f * sin(theta));
    }
    glEnd();
    // Visor reflection
    glColor4f(0.0f, 0.9f, 1.0f, 0.3f);
    drawCircle(s.x, s.y + 0.12f, 0.025f, 30);
    // Face inside helmet
    glColor4f(0.9f, 0.8f, 0.7f, 0.9f);
    drawCircle(s.x, s.y + 0.12f, 0.02f, 20);
    glColor4f(0.0f, 0.0f, 0.0f, 0.9f);
    drawCircle(s.x - 0.01f, s.y + 0.12f, 0.002f, 10); // Eye
    drawCircle(s.x + 0.01f, s.y + 0.12f, 0.002f, 10); // Eye
    // Moving arm (with suit sleeve)
    glPushMatrix();
    glTranslatef(s.x + 0.025f, s.y + 0.08f, 0.0f);
    glRotatef(30.0f * sin(s.anim), 0.0f, 0.0f, 1.0f);
    glColor4f(0.9f, 0.9f, 0.9f, 0.9f); // Suit arm
    glBegin(GL_QUADS);
    glVertex2f(0.0f, -0.012f);
    glVertex2f(0.045f, -0.012f);
    glVertex2f(0.045f, 0.012f);
    glVertex2f(0.0f, 0.012f);
    glEnd();
    glPopMatrix();
    glDisable(GL_BLEND);
}
```

```
void drawMetroPillar4(float x, float yBottom, float width, float height, bool destroyed) {

    // ...
     glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float topWidth = width;
    if (destroyed) {
        height *= 0.6f; // Shorter
        glColor4f(0.5f, 0.5f, 0.6f, 0.95f);
        glBegin(GL_QUADS);
        glVertex2f(x - width / 2, yBottom);
        glVertex2f(x + width / 2, yBottom);
        glVertex2f(x + topWidth / 2 * 0.8f, yBottom + height);
        glVertex2f(x - topWidth / 2 * 0.8f, yBottom + height);
        glEnd();
        // Cracks
        glColor3f(0.2f, 0.2f, 0.2f);
        glBegin(GL_LINES);
        glVertex2f(x - width / 2, yBottom + height * 0.3f);
        glVertex2f(x + width / 2, yBottom + height * 0.7f);
        glEnd();
    } else {
        glBegin(GL_QUADS);
        glColor4f(0.7f, 0.7f, 0.8f, 0.95f);
        glVertex2f(x - width / 2, yBottom);
        glVertex2f(x + width / 2, yBottom);
        glColor4f(0.8f, 0.8f, 0.9f, 0.95f);
        glVertex2f(x + topWidth / 2, yBottom + height);
        glVertex2f(x - topWidth / 2, yBottom + height);
        glEnd();
        glColor4f(0.5f, 0.5f, 0.6f, 0.8f);
        glLineWidth(1.5f);
        glBegin(GL_LINES);
        for (float offset = -width / 2 + 0.01f; offset < width / 2; offset += 0.02f) {
            glVertex2f(x + offset, yBottom);
            glVertex2f(x + offset, yBottom + height);
        }
        glEnd();
        glColor4f(0.0f, 1.0f, 1.0f, 0.8f); // Neon cyan accents
        glBegin(GL_QUADS);
        glVertex2f(x - width / 2, yBottom);
        glVertex2f(x - width / 2 + 0.005f, yBottom);
        glVertex2f(x - topWidth / 2 + 0.005f, yBottom + height);
        glVertex2f(x - topWidth / 2, yBottom + height);
        glEnd();
        glBegin(GL_QUADS);
        glVertex2f(x + width / 2 - 0.005f, yBottom);
        glVertex2f(x + width / 2, yBottom);
        glVertex2f(x + topWidth / 2, yBottom + height);
        glVertex2f(x + topWidth / 2 - 0.005f, yBottom + height);
        glEnd();
        glColor4f(0.6f, 0.6f, 0.7f, 0.9f);
        glBegin(GL_QUADS);
        glVertex2f(x - width / 2, yBottom + height * 0.3f);
        glVertex2f(x - width / 2 + 0.01f, yBottom + height * 0.3f);
        glVertex2f(x + topWidth / 2 - 0.01f, yBottom + height * 0.7f);
        glVertex2f(x + topWidth / 2, yBottom + height * 0.7f);
        glEnd();
        glBegin(GL_QUADS);
        glVertex2f(x + width / 2 - 0.01f, yBottom + height * 0.3f);
```

```
        glVertex2f(x + width / 2, yBottom + height * 0.3f);
        glVertex2f(x - topWidth / 2, yBottom + height * 0.7f);
        glVertex2f(x - topWidth / 2 + 0.01f, yBottom + 0.7f * height);
        glEnd();
        glColor4f(0.5f, 0.5f, 0.6f, 0.9f);
        glBegin(GL_QUADS);
        glVertex2f(x - topWidth / 2 - 0.05f, yBottom + height - 0.03f);
        glVertex2f(x + topWidth / 2 + 0.05f, yBottom + height - 0.03f);
        glVertex2f(x + topWidth / 2 + 0.05f, yBottom + height + 0.03f);
        glVertex2f(x - topWidth / 2 - 0.05f, yBottom + height + 0.03f);
        glEnd();
    }
    glDisable(GL_BLEND);
}
void drawVacuumTube4(float xStart, float xEnd, float y, float height, bool destroyed) {

    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    if (destroyed) {
        // Broken tube
        glColor4f(0.55f, 0.55f, 0.65f, 0.5f);
        glBegin(GL_POLYGON);
        glVertex2f(xStart, y - height / 2);
        glVertex2f(meteorX4 - 0.1f, y - height / 2);
        for (int i = 0; i <= 10; i++) {
            float t = float(i) / 10.0f;
            float x = xStart + (meteorX4 - 0.1f - xStart) * t;
            float yOffset = height / 2 * cos(M_PI * t) * (1.0f - t * 0.5f); // Distorted
            glVertex2f(x, y + yOffset);
        }
        glEnd();
        glBegin(GL_POLYGON);
        glVertex2f(meteorX4 + 0.1f, y - height / 2);
        glVertex2f(xEnd, y - height / 2);
        for (int i =0; i <= 10; i++) {
            float t = float(i) / 10.0f;
            float x = meteorX4 + 0.1f + (xEnd - meteorX4 - 0.1f) * t;
            float yOffset = height / 2 * cos(M_PI * t) * (1.0f - (1.0f - t) * 0.5f);
            glVertex2f(x, y + yOffset);
        }
        glEnd();
    } else {
        glColor4f(0.75f, 0.75f, 0.85f, 0.5f);
        glBegin(GL_POLYGON);
        glVertex2f(xStart, y - height / 2);
        glVertex2f(xEnd, y - height / 2);
        for (int i = 0; i <= 20; i++) {
            float t = float(i) / 20.0f;
            float x = xStart + (xEnd - xStart) * t;
            float yOffset = height / 2 * cos(M_PI * t);
            glVertex2f(x, y + yOffset);
        }
        glEnd();
        glColor4f(0.2f, 0.2f, 0.3f, 0.9f);
        for (float x = -1.4f; x <= 1.4f; x += 0.4f) {
            glBegin(GL_QUADS);
            glVertex2f(x - 0.06f, y - height / 2);
            glVertex2f(x + 0.06f, y - height / 2);
            glVertex2f(x + 0.06f, y + height / 2);
```

```
            glVertex2f(x - 0.06f, y + height / 2);
            glEnd();
        }
        glColor4f(1.0f, 0.5f, 0.0f, 0.7f);
        for (float x = -1.4f; x <= 1.4f; x += 0.4f) {
            glBegin(GL_QUADS);
            glVertex2f(x - 0.06f, y + height / 2 - 0.01f);
            glVertex2f(x + 0.06f, y + height / 2 - 0.01f);
            glVertex2f(x + 0.06f, y + height / 2);
            glVertex2f(x - 0.06f, y + height / 2);
            glEnd();
        }
    }
    glDisable(GL_BLEND);
}
void drawMetroRail4(float xStart, float xEnd, float y, float thickness, bool destroyed) {

    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    if (destroyed) {
        // Broken rail
        glColor4f(0.1f, 0.1f, 0.15f, 0.9f);
        glBegin(GL_QUADS);
        glVertex2f(xStart, y - thickness / 2);
        glVertex2f(meteorX4 - 0.2f, y - thickness / 2);
        glVertex2f(meteorX4 - 0.2f, y + thickness / 2 - 0.02f);
        glVertex2f(xStart, y + thickness / 2 - 0.02f);
        glEnd();
        glBegin(GL_QUADS);
        glVertex2f(meteorX4 + 0.2f, y - thickness / 2 + 0.02f);
        glVertex2f(xEnd, y - thickness / 2 + 0.02f);
        glVertex2f(xEnd, y + thickness / 2);
        glVertex2f(meteorX4 + 0.2f, y + thickness / 2);
        glEnd();
    } else {
        glColor4f(0.2f, 0.2f, 0.3f, 0.9f);
        glBegin(GL_QUADS);
        glVertex2f(xStart, y - thickness / 2);
        glVertex2f(xEnd, y - thickness / 2);
        glVertex2f(xEnd, y + thickness / 2);
        glVertex2f(xStart, y + thickness / 2);
        glEnd();
        glColor4f(1.0f, 0.5f, 0.0f, 0.6f);
        glBegin(GL_QUADS);
        glVertex2f(xStart, y - thickness / 4);
        glVertex2f(xEnd, y - thickness / 4);
        glVertex2f(xEnd, y + thickness / 4);
        glVertex2f(xStart, y + thickness / 4);
        glEnd();
    }
    glDisable(GL_BLEND);
}
void drawMetroTrain4(float x, float y, float width, float height, bool destroyed) {

    // ...
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    float flicker = 0.8f + 0.2f * sin(timeCounter * 2.0f);
    float segmentWidth = width / 4.0f;
```

```
float segmentHeight = height * 1.2f;
if (destroyed) {
    // Draw broken train parts
    glColor4f(0.7f, 0.3f, 0.0f, 0.9f);
    for (int i = 0; i < 4; i++) {
        float segX = x - width / 2 + segmentWidth * i + segmentWidth / 2;
        glPushMatrix();
        glTranslatef(segX, y, 0.0f);
        glRotatef((i - 1.5f) * 15.0f, 0.0f, 0.0f, 1.0f); // Tilted segments
        glBegin(GL_POLYGON);
        glVertex2f(-segmentWidth / 2, -segmentHeight / 2);
        glVertex2f(segmentWidth / 2, -segmentHeight / 2);
        glVertex2f(segmentWidth / 2 * 0.8f, segmentHeight / 2 * 0.7f);
        glVertex2f(-segmentWidth / 2 * 0.8f, segmentHeight / 2 * 0.7f);
        glEnd();
        glPopMatrix();
    }
    // Add debris
    glColor4f(0.5f, 0.5f, 0.5f, 0.9f);
    drawCircle(x - 0.2f, y - 0.1f, 0.03f, 20);
    drawCircle(x + 0.3f, y - 0.05f, 0.04f, 20);
} else {
    // Continuous body
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
    glBegin(GL_QUADS);
    glVertex2f(x - width / 2, y - segmentHeight / 2);
    glVertex2f(x + width / 2, y - segmentHeight / 2);
    glVertex2f(x + width / 2, y + segmentHeight / 2);
    glVertex2f(x - width / 2, y + segmentHeight / 2);
    glEnd();
    // Compartments
    for (int i = 0; i < 4; i++) {
        float segX = x - width / 2 + segmentWidth * i + segmentWidth / 2;
        glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
        glBegin(GL_POLYGON);
        glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 2);
        glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 2);
        glVertex2f(segX + segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
        glVertex2f(segX - segmentWidth / 2 * 0.8f, y + segmentHeight / 2);
        glEnd();
        // Neon accent
        glColor4f(1.0f, 0.7f, 0.2f, 0.7f * flicker);
        glBegin(GL_QUADS);
        glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 4);
        glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 4);
        glVertex2f(segX + segmentWidth / 2, y - segmentHeight / 4 + 0.01f);
        glVertex2f(segX - segmentWidth / 2, y - segmentHeight / 4 + 0.01f);
        glEnd();
        // Windows
        glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
        int numWindows = (i == 0 || i == 3) ? 2 : 3;
        float startX = segX - segmentWidth / 2 + 0.03f;
        float stepX = (segmentWidth - 0.06f) / (numWindows - 1);
        for (int j = 0; j < numWindows; j++) {
            drawCircle(startX + j * stepX, y, 0.02f, 20);
        }
    }
    // Front and back noses
    glColor4f(1.0f, 0.5f, 0.0f, 0.9f);
    glBegin(GL_POLYGON);
```

```
        glVertex2f(x + width / 2, y - segmentHeight / 3);
        glVertex2f(x + width / 2 + 0.1f, y - segmentHeight / 3);
        glVertex2f(x + width / 2 + 0.1f, y + segmentHeight / 3);
        glVertex2f(x + width / 2, y + segmentHeight / 3);
        glEnd();
        glPushMatrix();
        glTranslatef(x + width / 2 + 0.1f, y, 0.0f);
        glScalef(0.6f, 0.6f, 1.0f);
        drawCircle(0.0f, 0.0f, 0.1f, 30);
        glPopMatrix();
        glBegin(GL_POLYGON);
        glVertex2f(x - width / 2, y - segmentHeight / 3);
        glVertex2f(x - width / 2 - 0.1f, y - segmentHeight / 3);
        glVertex2f(x - width / 2 - 0.1f, y + segmentHeight / 3);
        glVertex2f(x - width / 2, y + segmentHeight / 3);
        glEnd();
        glPushMatrix();
        glTranslatef(x - width / 2 - 0.1f, y, 0.0f);
        glScalef(-0.6f, 0.6f, 1.0f);
        drawCircle(0.0f, 0.0f, 0.1f, 30);
        glPopMatrix();
        // Windows
        glColor4f(0.0f, 0.9f, 1.0f, 0.9f);
        glBegin(GL_QUADS);
        glVertex2f(x + width / 2 + 0.05f, y - 0.03f);
        glVertex2f(x + width / 2 + 0.15f, y - 0.03f);
        glVertex2f(x + width / 2 + 0.15f, y + 0.03f);
        glVertex2f(x + width / 2 + 0.05f, y + 0.03f);
        glEnd();
        glBegin(GL_QUADS);
        glVertex2f(x - width / 2 - 0.15f, y - 0.03f);
        glVertex2f(x - width / 2 - 0.05f, y - 0.03f);
        glVertex2f(x - width / 2 - 0.05f, y + 0.03f);
        glVertex2f(x - width / 2 - 0.15f, y + 0.03f);
        glEnd();
        // Headlights
        glColor4f(1.0f, 1.0f, 0.8f, 0.9f * flicker);
        drawCircle(x + width / 2 + 0.1f, y, 0.02f, 20);
        drawCircle(x - width / 2 - 0.1f, y, 0.02f, 20);
        // Jet thrusters
        glColor4f(1.0f, 0.7f, 0.2f, 0.7f * flicker);
        float jetPos = commonTrainX > 0 ? (x - width / 2 - 0.1f) : (x + width / 2 + 0.1f);
        glBegin(GL_TRIANGLES);
        glVertex2f(jetPos, y);
        glVertex2f(jetPos + (commonTrainX > 0 ? -0.06f : 0.06f), y + 0.03f);
        glVertex2f(jetPos + (commonTrainX > 0 ? -0.06f : 0.06f), y - 0.03f);
        glEnd();
    }
    glDisable(GL_BLEND);
}
void drawMetro4() {

    float yMid = (-0.05f + -0.60f) / 2.0f;
    float yBottom = yMid - 0.35f;
    float height = 0.76f;
    float railY = yBottom + height;
    float thickness = 0.06f;
    float tubeHeight = 0.12f;
    drawVacuumTube4(-1.5f, 1.5f, railY, tubeHeight, metroDestroyed4);
    drawMetroRail4(-1.5f, 1.5f, railY, thickness, metroDestroyed4);
```

```
    for (float x = -1.4f; x <= 1.4f; x += 0.4f) {
        drawMetroPillar4(x, yBottom, 0.05f, height, metroDestroyed4);
    }
    drawMetroTrain4(commonTrainX, railY, 0.8f, 0.08f, metroDestroyed4);
}
void drawScene4() {
    drawStars4();
    drawPlanets4();
    drawRocket4(rocketX4, 0.7f);
    if (groundDestroyed4) {
        drawDestroyedGround4();
    } else {
        drawGround4();
    }
    drawCity4();
    if (roadDestroyed4) {
        drawDestroyedRoadAndFootpath4();
    } else {
        drawRoadAndFootpath4();
    }
    for (int i = 0; i < 8; i++) {
        if (vehicles4[i].destroyed) {
            drawDestroyedVehicle4(vehicles4[i]);
        } else {
            drawVehicle4(vehicles4[i]);
        }
    }
    drawMetro4();
    drawMeteor4();
    if (meteorHit4) {
        drawDebris4();
        drawSmoke4();
        drawFire4();
        for (int i = 0; i < 10; i++) {
            drawDrone4(drones4[i].x, drones4[i].y, drones4[i].angle, drones4[i].armAngle);
        }
        for (int i = 0; i < 3; i++) {
            drawSurvivor4(survivors4[i]);
        }
    }
}
void updateScene4() {
    timeCounter += 0.05f;
    if (!meteorHit4) {
        for (int i = 0; i < 8; i++) {
            if (vehicles4[i].destroyed) continue;
            float effectiveSpeed = vehicles4[i].speed;
            float newX = vehicles4[i].x + effectiveSpeed * (vehicles4[i].flip ? -1 : 1);
            float minDist = 0.4f;
            for (int j = 0; j < 8; j++) {
                if (i != j && vehicles4[j].y == vehicles4[i].y && vehicles4[i].flip == vehicles4[j].flip &&
!vehicles4[j].destroyed) {
                    float dist = vehicles4[i].flip ? (vehicles4[j].x - newX) : (newX - vehicles4[j].x);
                    if (dist > 0 && dist < minDist) {
                        newX = vehicles4[i].flip ? (vehicles4[j].x - minDist) : (vehicles4[j].x + minDist);
                        break;
                    }
                }
            }
            vehicles4[i].x = newX;
```

```
            if (vehicles4[i].flip && vehicles4[i].x < -1.5f) vehicles4[i].x = 1.5f;
            if (!vehicles4[i].flip && vehicles4[i].x > 1.5f) vehicles4[i].x = -1.5f;
        }
        // Train movement only if !meteorHit4
        if (!meteorHit4) {
            commonTrainX += commonTrainSpeed;
            if (commonTrainX > 2.0f) commonTrainX = -1.5f;
        }
    }
    if (rocketMoving4) {
        rocketX4 += rocketSpeed4;
        if (rocketX4 > 1.5f) rocketX4 = -1.5f;
    }
    if (meteorFalling4 && !meteorPaused4 && !meteorHit4) {
        meteorY4 += meteorSpeed4;
        meteorSize4 += 0.001f;
        if (meteorY4 < -0.63f) {
            meteorHit4 = true;
            cityDestroyed4 = true;
            metroDestroyed4 = true;
            groundDestroyed4 = true;
            roadDestroyed4 = true;
            for (int i = 0; i < 8; i++) {
                if (fabs(vehicles4[i].x - meteorX4) < 1.0f) {
                    vehicles4[i].destroyed = true;
                }
            }
            initDebris4();
            initSmoke4();
            initFire4();
            initDrones4();
            initSurvivors4();
        }
    }
    if (meteorHit4) {
        for (int i = 0; i < 100; i++) {
            debrisY4[i] += debrisSpeed4[i];
            if (debrisY4[i] > 1.0f || debrisY4[i] < -1.0f) {
                debrisY4[i] = -0.63f;
                debrisSpeed4[i] = 0.0f;
            }
        }
        for (int i = 0; i < 50; i++) {
            smokeY4[i] += 0.001f;
            smokeAlpha4[i] -= 0.0005f;
            if (smokeAlpha4[i] < 0) smokeAlpha4[i] = 0;
        }
        for (int i = 0; i < 50; i++) {
            fireAlpha4[i] = 0.5f + 0.5f * sin(timeCounter * 3 + i);
        }
        for (int i = 0; i < 10; i++) {
            drones4[i].angle += drones4[i].speed;
            drones4[i].x += 0.001f * cos(drones4[i].angle);
            drones4[i].y += 0.001f * sin(drones4[i].angle);
            drones4[i].armAngle = sin(timeCounter + i) * M_PI / 4.0f; // Animate arms
        }
        for (int i = 0; i < 3; i++) {
            survivors4[i].anim += 0.05f;
        }
    }
```

```
}
// Main display
void display() {
   glClear(GL_COLOR_BUFFER_BIT);
   glMatrixMode(GL_MODELVIEW);
   glLoadIdentity();
   switch (currentScene) {
      case 1:
         drawScene1();
         break;
      case 2:
         drawScene2();
         break;
      case 3:
         drawScene3();
         break;
      case 4:
         drawScene4();
         break;
   }
   glutSwapBuffers();
}
// Main update
void update(int value) {
   switch (currentScene) {
      case 1:
         updateScene1();
         break;
      case 2:
         updateScene2();
         break;
      case 3:
         updateScene3();
         break;
      case 4:
         updateScene4();
         break;
   }
   glutPostRedisplay();
   glutTimerFunc(16, update, 0);
}
// Reshape (common, from scene1)
void reshape(GLsizei width, GLsizei height) {
   if (height == 0) height = 1;
   glViewport(0, 0, width, height);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(-1.5, 1.5, -1.0, 1.0);
}
// Keyboard (union of all, with prefix)
void keyboard(unsigned char key, int x, int y) {
   if (currentScene == 1) {
      switch (key) {
         case 'q':
            astronautVel1 = -0.005f;
            break;
         case 'e':
            astronautVel1 = 0.005f;
            break;
         case 's':
```

```
            dogVel1 = -0.005f;
            break;
        case 'd':
            dogVel1 = 0.005f;
            break;
        case 'r':
            robot1XVel1 = -0.005f;
            break;
        case 't':
            robot1XVel1 = 0.005f;
            break;
        case 'j':
            if (robot1YOffset1 == 0.0f) {
                robot1JumpVel1 = 0.02f;
            }
            break;
        case 'n':
        case 'N':
            nightMode1 = !nightMode1;
            break;
        case ' ':
            trainPaused1 = !trainPaused1;
            long long now = glutGet(GLUT_ELAPSED_TIME);
            if (trainPaused1) {
                pauseStartTime1 = now;
            } else {
                totalPausedTime1 += now - pauseStartTime1;
            }
            break;
    }
} else if (currentScene == 2) {
    switch (key) {
        case '1':
        case '!':
            enableWaterDrone2[0] = !enableWaterDrone2[0];
            break;
        case '2':
        case '@':
            enableWaterDrone2[1] = !enableWaterDrone2[1];
            break;
        case '3':
        case '#':
            enableWaterDrone2[2] = !enableWaterDrone2[2];
            break;
        case 'q':
        case 'Q':
            enableSky12 = !enableSky12;
            break;
        case 'w':
        case 'W':
            enableSky22 = !enableSky22;
            break;
        case '[':
            rainIntensity2 -= 6;
            if(rainIntensity2 < 6) rainIntensity2 = 6;
            break;
        case ']':
            rainIntensity2 += 6;
            if(rainIntensity2 > 120) rainIntensity2 = 120;
            break;
```

```
            case 'R':
                for(int p = 0; p < NUM_PATCHES2; p++)
                    wetness2[p] = 0.0f;
                break;
        }
    } else {
        switch (key) {
            case 'x':
            case 'X':
                rocketMoving3 = false;
                rocketMoving4 = false;
                break;
            case 'y':
            case 'Y':
                rocketMoving3 = true;
                rocketMoving4 = true;
                break;
            case 'z':
            case 'Z':
                rocketSpeed3 += 0.005f;
                if (rocketSpeed3 > 0.05f) rocketSpeed3 = 0.05f;
                rocketSpeed4 += 0.005f;
                if (rocketSpeed4 > 0.05f) rocketSpeed4 = 0.05f;
                break;
            case 'v':
            case 'V':
                rocketSpeed3 -= 0.005f;
                if (rocketSpeed3 < 0.001f) rocketSpeed3 = 0.001f;
                rocketSpeed4 -= 0.005f;
                if (rocketSpeed4 < 0.001f) rocketSpeed4 = 0.001f;
                break;
            case 'W':
                windowsOn3 = !windowsOn3;
                windowsOn4 = !windowsOn4;
                break;
        }
    }
    switch (key) {
        case 'k':
            exit(0);
            break;
        case 27: // Escape key
            exit(0);
            break;
    }
}
void keyboardUp(unsigned char key, int x, int y) {
    // from scene1
    switch (key) {
        case 'q':
        case 'e':
            astronautVel1 = 0.0f;
            break;
        case 's':
        case 'd':
            dogVel1 = 0.0f;
            break;
        case 'r':
        case 't':
            robot1XVel1 = 0.0f;
```

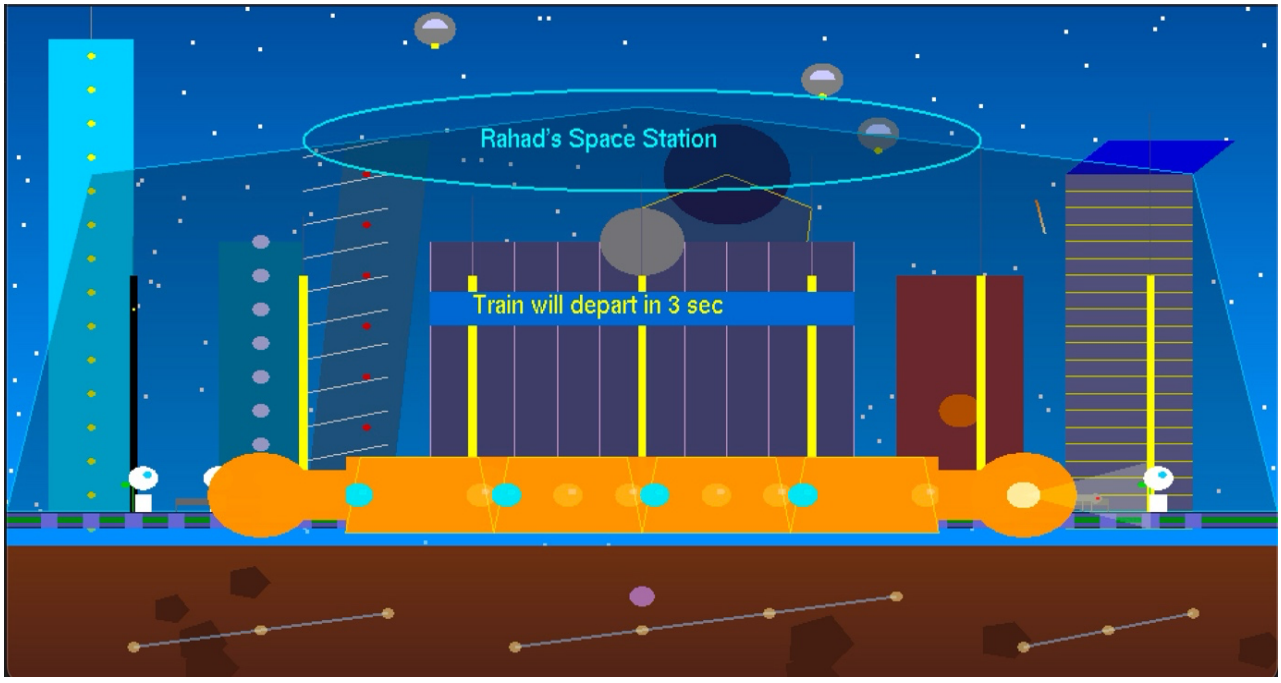
```
            break;
        }
    }
    void mouse(int button, int state, int x, int y) {
        // from scene1
        if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
            commonTrainSpeed += 0.002f;
        if (button == GLUT_RIGHT_BUTTON && state == GLUT_DOWN)
            commonTrainSpeed = (commonTrainSpeed > 0.002f) ? commonTrainSpeed - 0.002f : 0.002f;

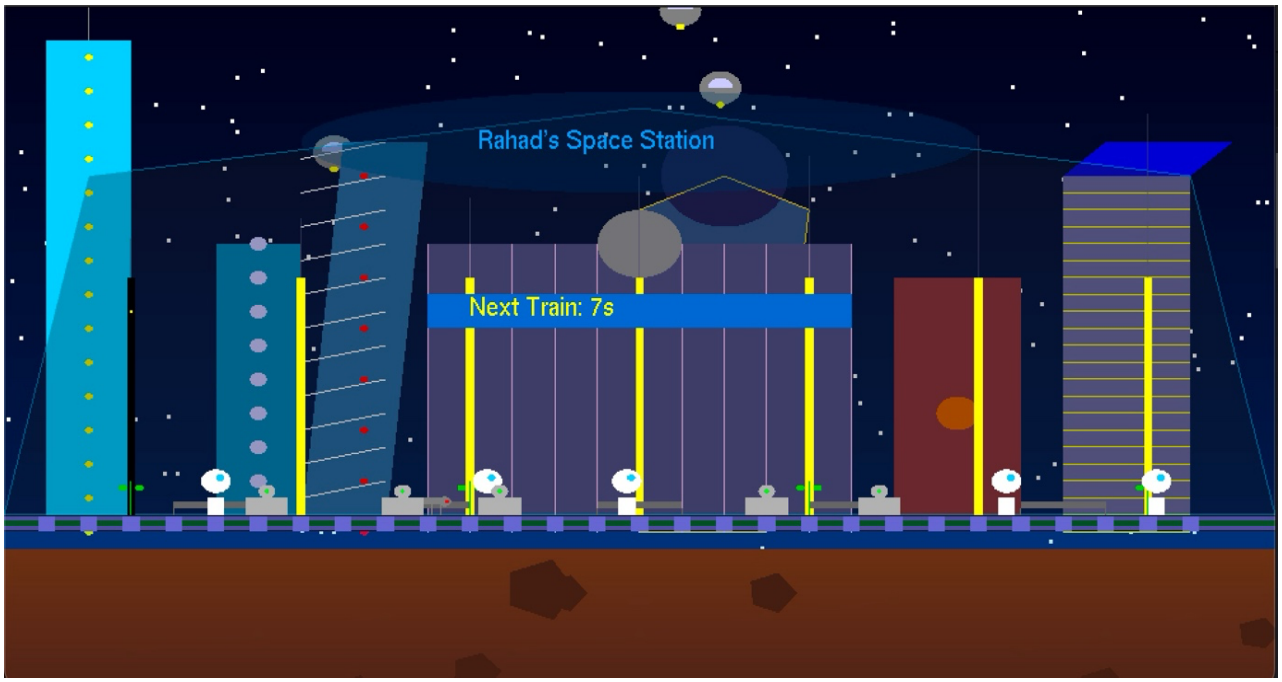
        // from scene3
        if (state == GLUT_DOWN) {
            if (button == GLUT_LEFT_BUTTON) {
                ferrariMoving3 = true;
            } else if (button == GLUT_RIGHT_BUTTON) {
                ferrariMoving3 = false;
            }
        }
        // from scene4
        if (state == GLUT_DOWN) {
            if (button == GLUT_LEFT_BUTTON && meteorFalling4 && meteorPaused4 && !meteorHit4) {
                meteorPaused4 = false;
            } else if (button == GLUT_RIGHT_BUTTON && meteorFalling4 && !meteorPaused4 && !meteorHit4) {
                meteorPaused4 = true;
            }
        }
    }
    int main(int argc, char** argv) {
        srand((unsigned)time(0));
        init1();
        initStars2();
        initGrains2();
        initStars3();
        initStars4();
        // Initialize vehicles for scene3 and 4
        float initialPositions[8] = {-1.4f, -1.0f, -0.6f, -0.2f, 0.2f, 0.6f, 1.0f, 1.4f};
        for (int i = 0; i < 8; i++) {
            vehicles3[i].x = initialPositions[i];
            vehicles3[i].y = laneY3[i % 4];
            vehicles3[i].flip = (i % 2 == 0);
            vehicles3[i].speed = 0.008f;
            vehicles3[i].scale = 0.7f + (rand() % 3)/10.0f;
            vehicles3[i].type = i % 4;
            vehicles3[i].r = 0.3f + (i%3)*0.2f;
            vehicles3[i].g = 0.3f + ((i+1)%3)*0.2f;
            vehicles3[i].b = 0.7f;
            vehicles3[i].isBus = false;
        }
        vehicles3[7].isBus = true;
        vehicles3[7].r = 0.5f; vehicles3[7].g = 0.5f; vehicles3[7].b = 0.5f;
        vehicles3[7].type = 0;
        // Copy for scene4
        for (int i = 0; i < 8; i++) {
            vehicles4[i].x = vehicles3[i].x;
    vehicles4[i].y = vehicles3[i].y;
    vehicles4[i].speed = vehicles3[i].speed;
    vehicles4[i].flip = vehicles3[i].flip;
    vehicles4[i].scale = vehicles3[i].scale;
    vehicles4[i].r = vehicles3[i].r;
    vehicles4[i].g = vehicles3[i].g;
```

```
vehicles4[i].b = vehicles3[i].b;
vehicles4[i].isBus = vehicles3[i].isBus;
vehicles4[i].type = vehicles3[i].type;
vehicles4[i].destroyed = false;
    }
    // Start meteor for scene4
    meteorFalling4 = true;
    meteorX4 = (rand() % 100 + 50) / 100.0f;
    meteorY4 = 1.0f;
    meteorSize4 = 0.01f;
    meteorHit4 = false;
    meteorPaused4 = false;
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_ALPHA);
    glutInitWindowSize(1000, 700);
    glutCreateWindow("Group 9 Project");
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutKeyboardFunc(keyboard);
    glutKeyboardUpFunc(keyboardUp);
    glutMouseFunc(mouse);
    glutTimerFunc(0, update, 0);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glutMainLoop();
    return 0;
}
```

Screenshots of the project (For each scenario)

Scenario 1: Space Metro Station





Scenario 2: Space City Farmland



Scenario 3: Futuristic City and Destroyed City

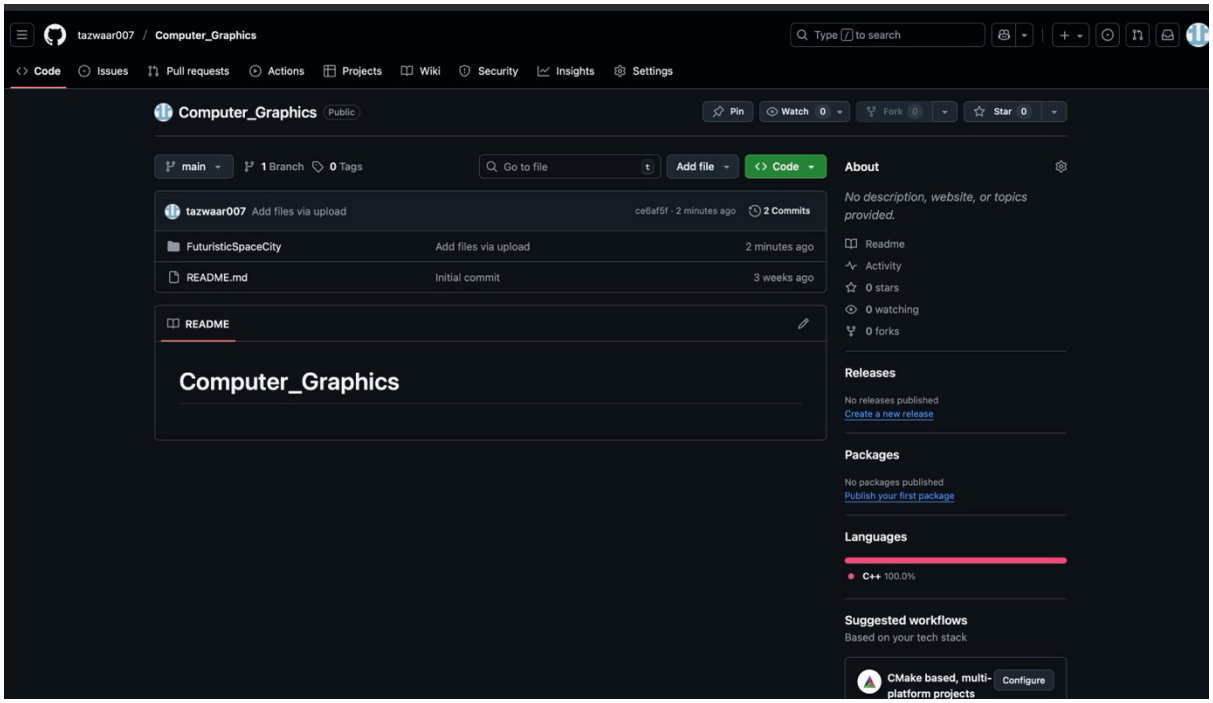


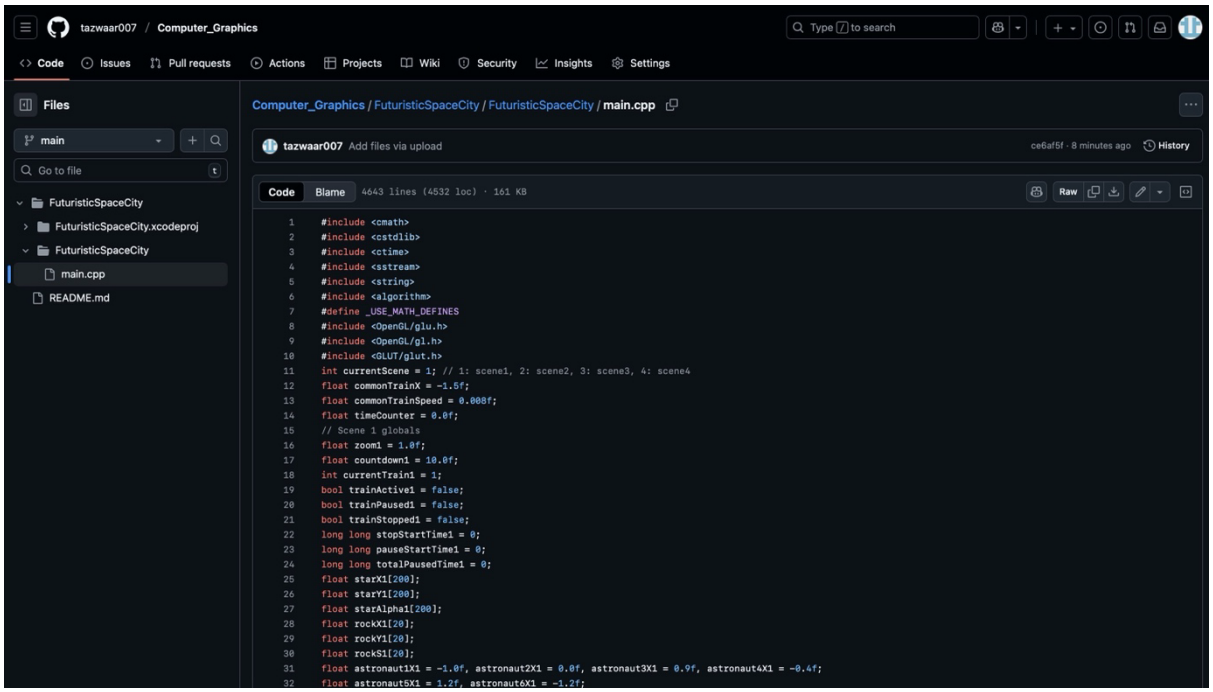


Link of the implementation in Github

Repository link: https://github.com/tazwaar007/Computer_Graphics

Screenshots:





References

[1] N. Crowley, "Production Design," *Interstellar (film)*. [Online]. Available: https://en.wikipedia.org/wiki/Interstellar_(film)#Production. [Accessed: Oct. 8, 2025].

[2] "Inside the Design of Cyberpunk 2077's Urban Dystopia," *Architectural Digest*. [Online]. Available: https://www.architecturaldigest.com/story/inside-the-design-of-cyberpunk-2077s-urban-dystopia. [Accessed: Oct. 8, 2025].

[3] Parametric Architecture, "A Guide to Identifying Futurist Architecture in Modern Cities." [Online]. Available: https://parametric-architecture.com/futurist-architecture-modern-cities. [Accessed: Oct. 8, 2025].

[4] NASA, "Human Exploration and Operations Mission Directorate," *National Aeronautics and Space Administration*, 2023. [Online]. Available: https://www.nasa.gov/exploration/

[5] "Blade Runner 2049," *Wikipedia*, https://en.wikipedia.org/wiki/Blade_Runner_2049, Accessed: Oct. 8, 2025.